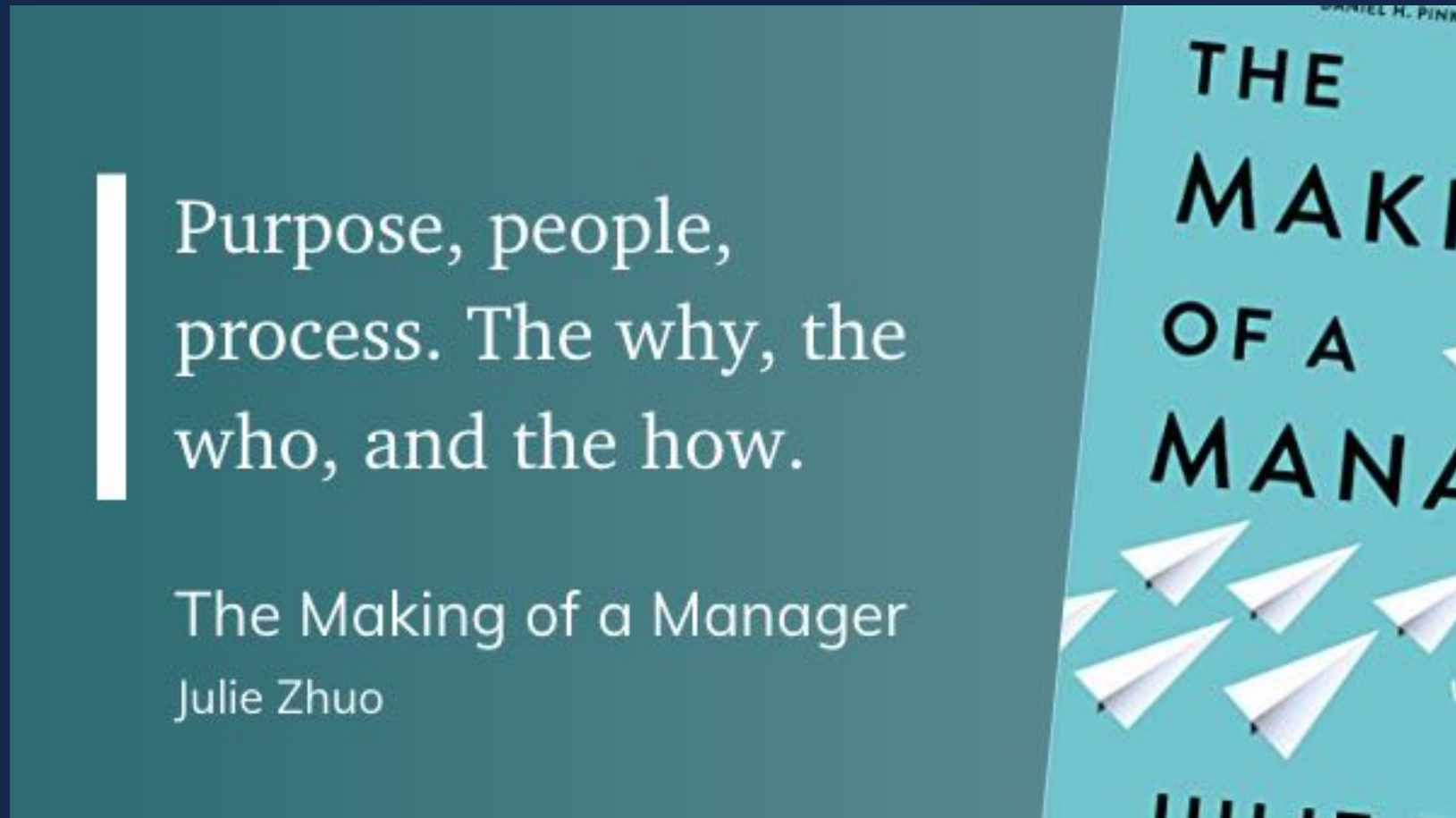


```
//CSE  
110;
```

Process Models Intro

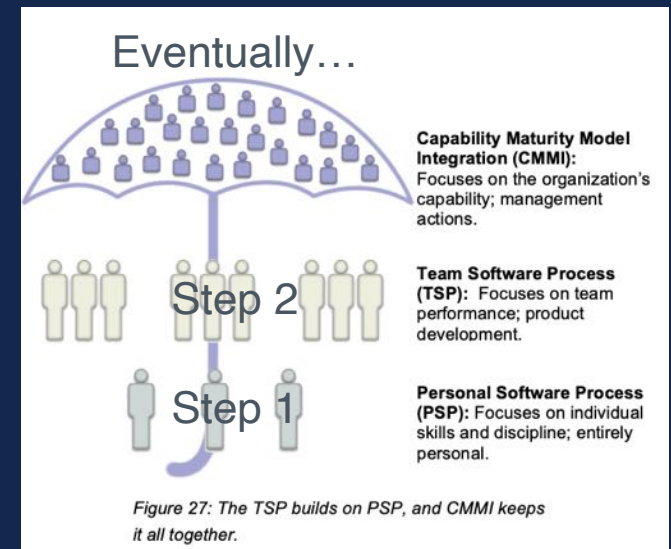
with Thomas Powell

Two out of three down...next up process



Reminders on where we have been...

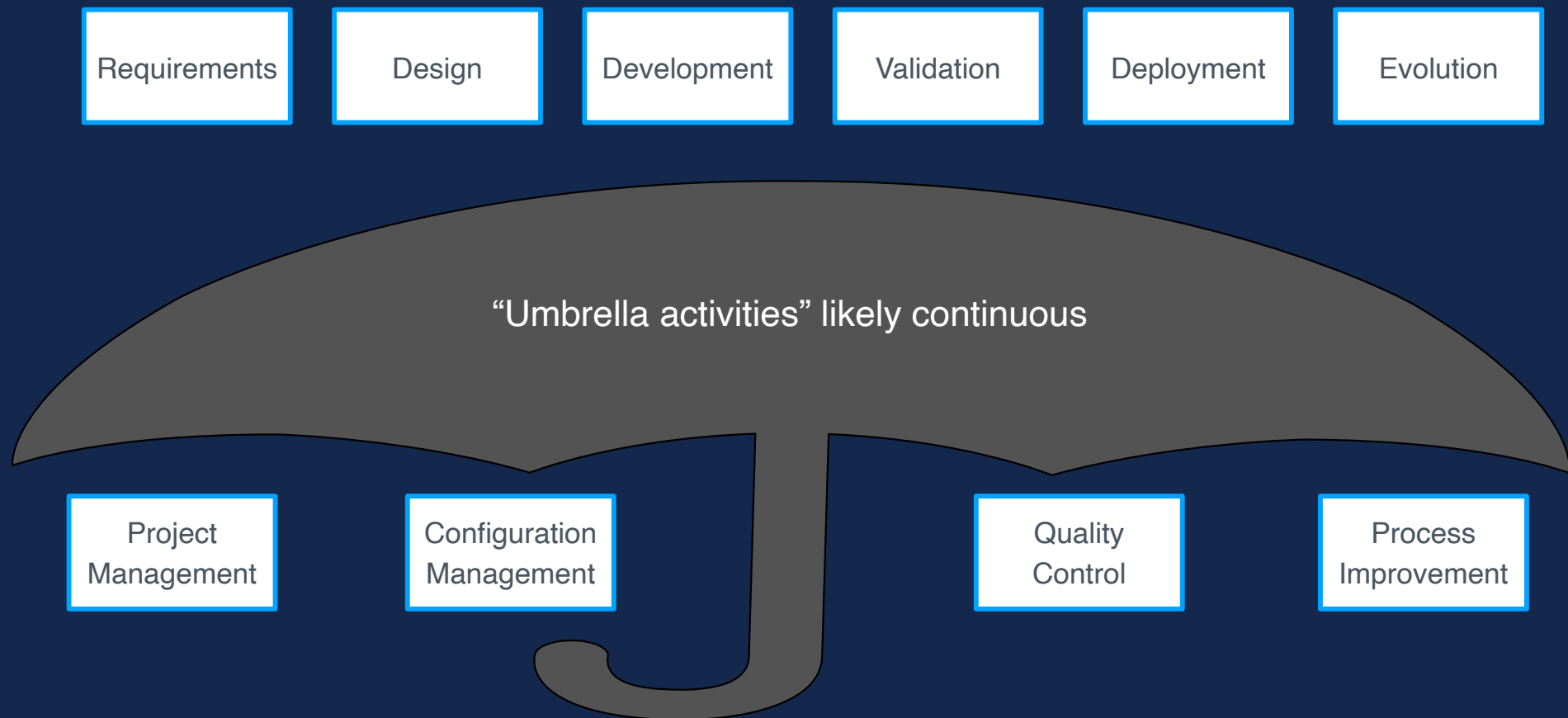
- Do you know what the CMMI is?
 - https://en.wikipedia.org/wiki/Capability_Maturity_Model_Integration
- Are we doing anything that serious?
- Of course not, but we are laying the foundation and building you up towards getting there...someday



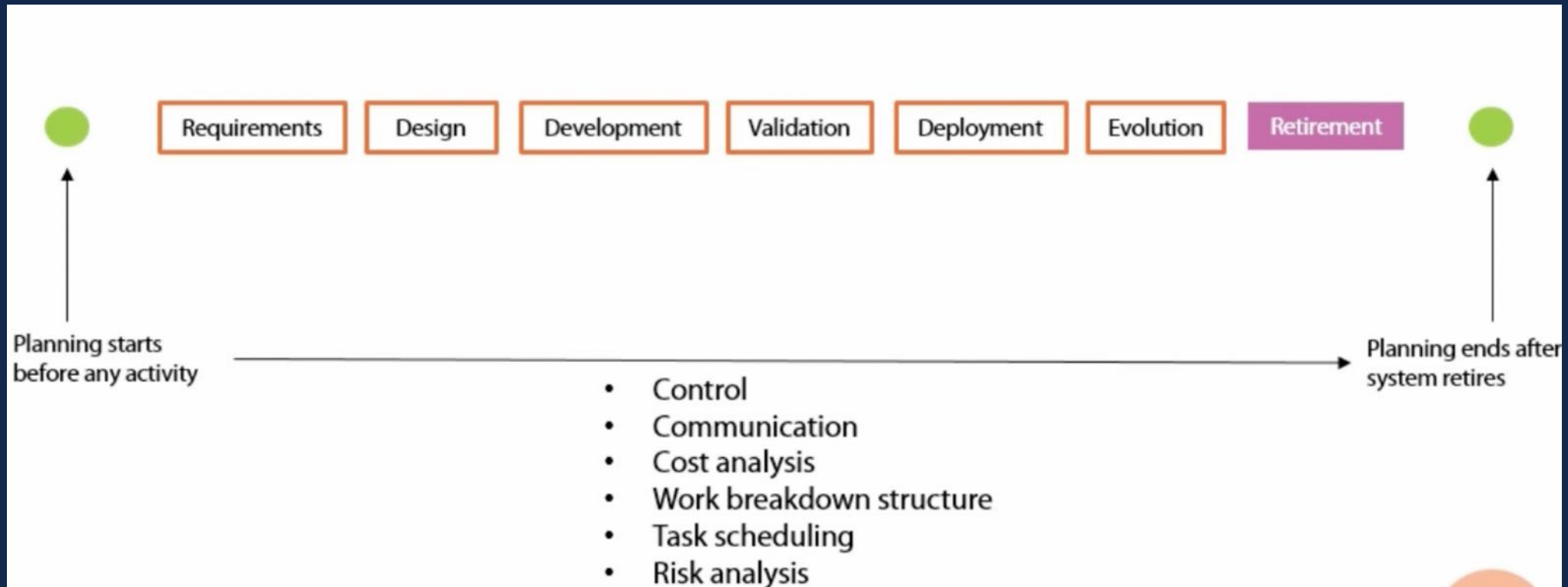
*Yep! Those darn soft skills
are hard engineering recs!*

Software Activities

General Software Lifecycle Activities (May be discrete)



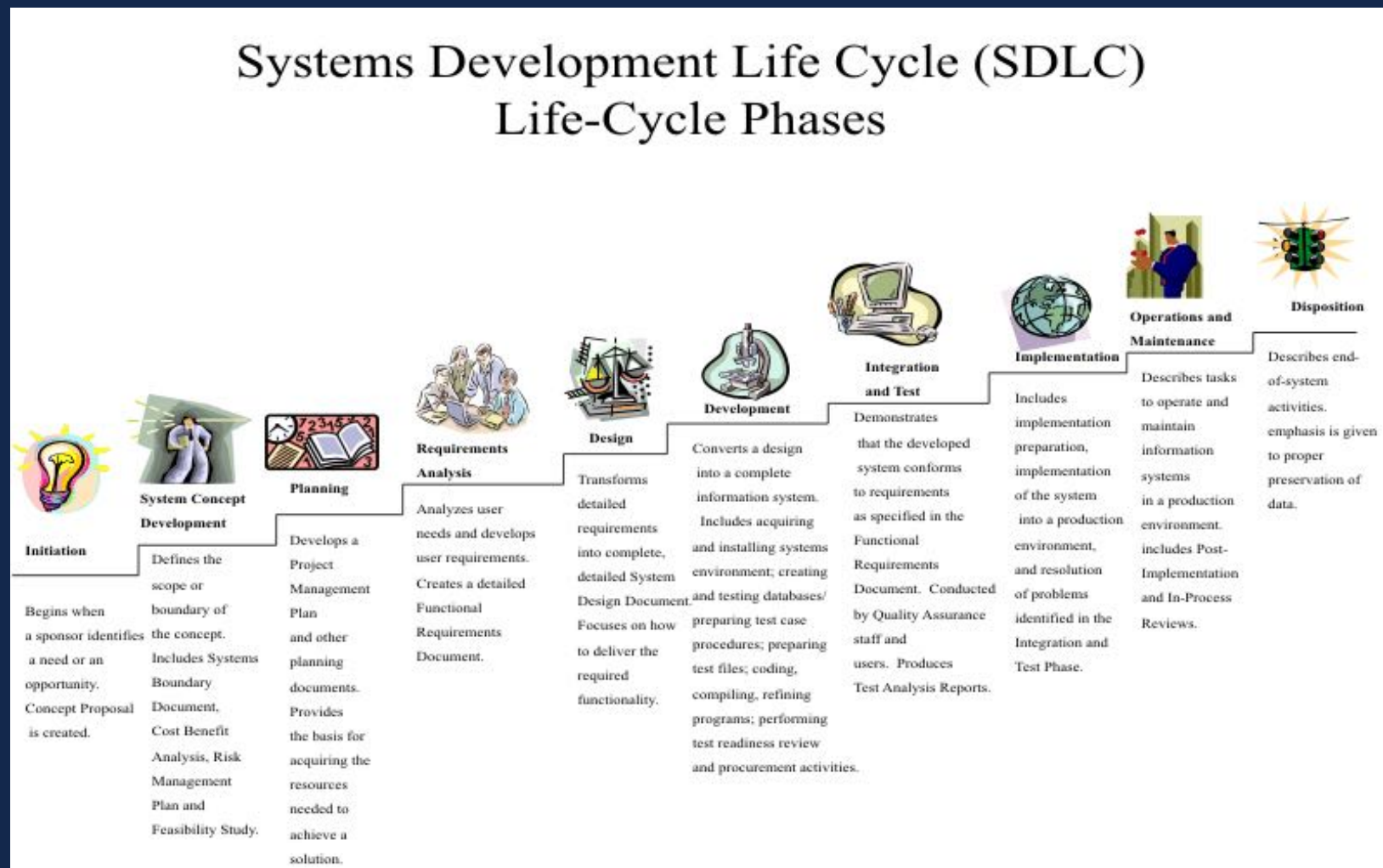
Umbrella Activity Explained



“What to be done” vs “How to do it”

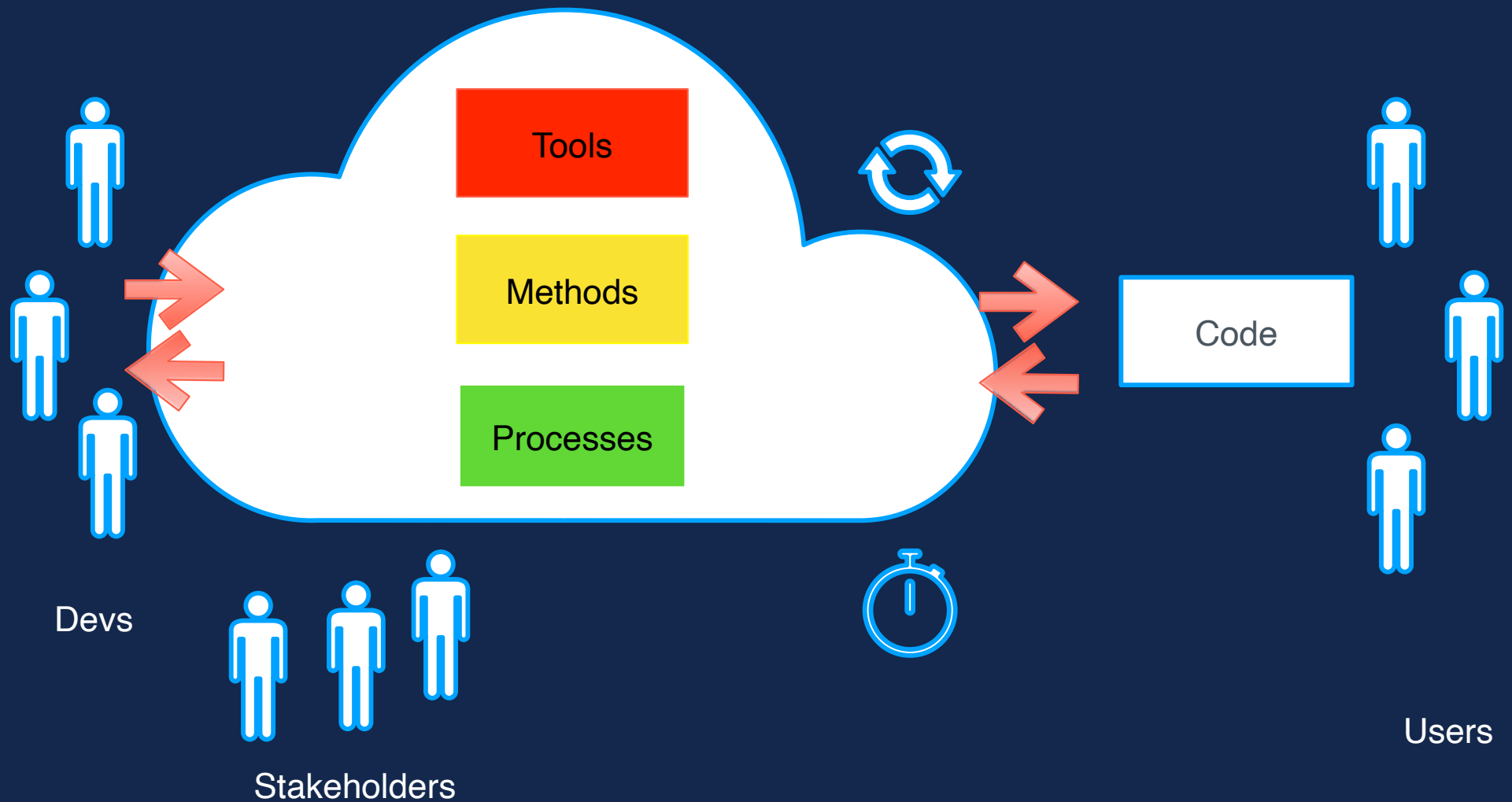
- Activities are independent of the process model we adopt.
 - Despite what some believe all steps should be present in the various approaches to SE, though the emphasis, amount and approach clearly varies from system to system
- Process models define the emphasize upon and the style of execution (ceremonies and structure) of the software activities performed
 - Ex: Order (liner or iterative)
 - Ex: Scope (incremental or whole)

SDLC (Systems Development Life Cycle)



https://en.wikipedia.org/wiki/Systems_development_life_cycle

Software Engineering at 10K Feet



Landscape Break Down

Processes

Framework of how SE activities are performed

Specifically how are requirements, design, dev, testing, deployment performed

Methods

Accepted (and maybe proven) techniques to perform SE activities

Specifically how we run a design process, approach testing throughout, etc.

Tools

The tools used to automate and improve the SE process

Tools improve the systematic and efficiency aspects of software engineering

Factory Thinking Redux

- Is software engineering about factory or is it about the resulting product in the form of software?
- Reality it is about both
 - The product is obviously what the user cares about and is the true measure of success or failure
 - However, the repeatability and measuring issues would suggest that without process we really aren't doing things predictably
- I am concerned that process without awareness of economic and situational drivers is theory truly without pragmatic practice.

Truth: Our Software Thinking Must Think of the WHOLE SYSTEM

Basics Level 1

Top-Down Design

- Break down a big idea into smaller parts
 - Think starting with a outcome vision and breaking into appropriate lego parts to make the model
 - Alternatively “How to eat an elephant? One bite at a time”
- Reductionist at its heart
 - Challenge how do we know what constitutes a small enough part and is it is easy to decompose
 - May suffer from gaps and emergence of sum of parts effects
 - Top level mistakes can cascade problems widely



Bottom-up Design

- Piece together small understood components to build a larger system
 - Think “building up with Legos” to our vision, but without a set plan
 - Organic and evolutionary
- A bottom up strategy is useful to get started quickly and deliver some minimal value
 - Can get messy as the path from small pieces to larger outcome may not be that easy to find.



0000 98 0000

BEZOS MUSK

FIGHT

... began product work by having his employees write press releases about a new product or feature before it even entered development.

...reduced the problem to first principles and worked it out from there

VOTE!

Special Move: Work From Outcome Backwards

Special Move: Work From Core Problem Forwards

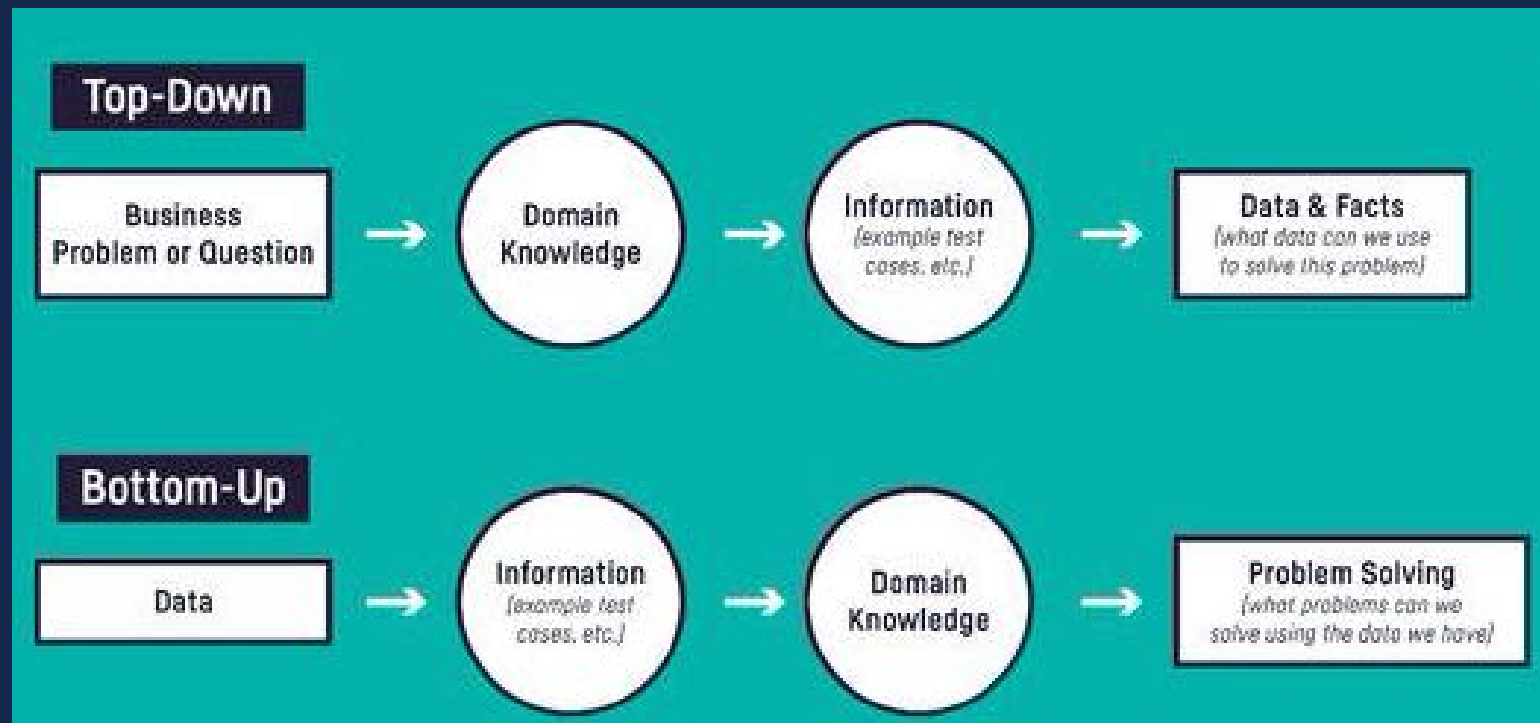
BOTH MADE TONS OF \$\$\$

**WHAT KIND OF
MORTAL KOMBAT IS THIS?**

Either Or AND Top and Bottom

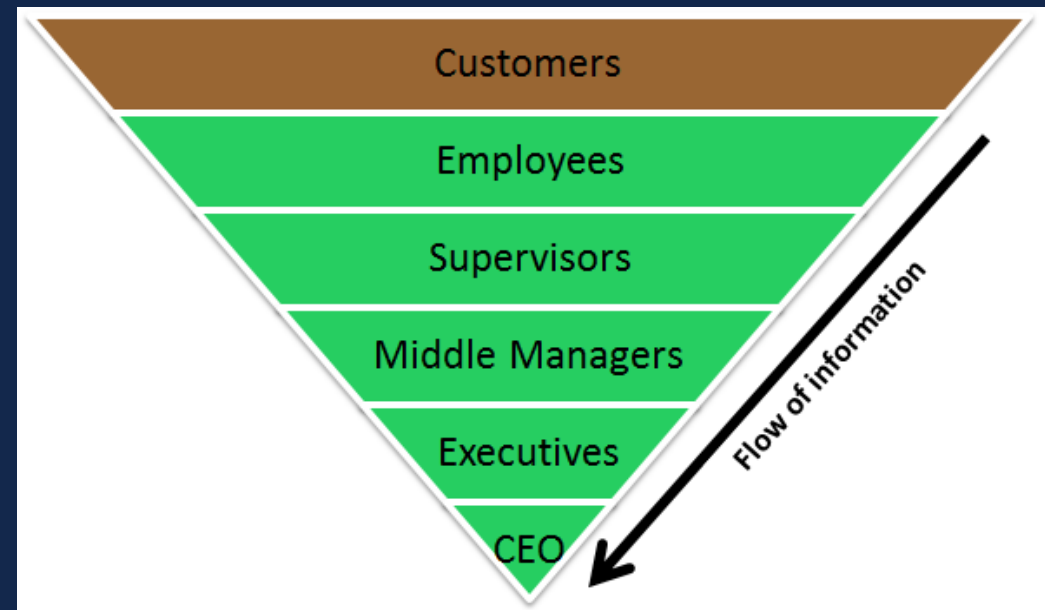
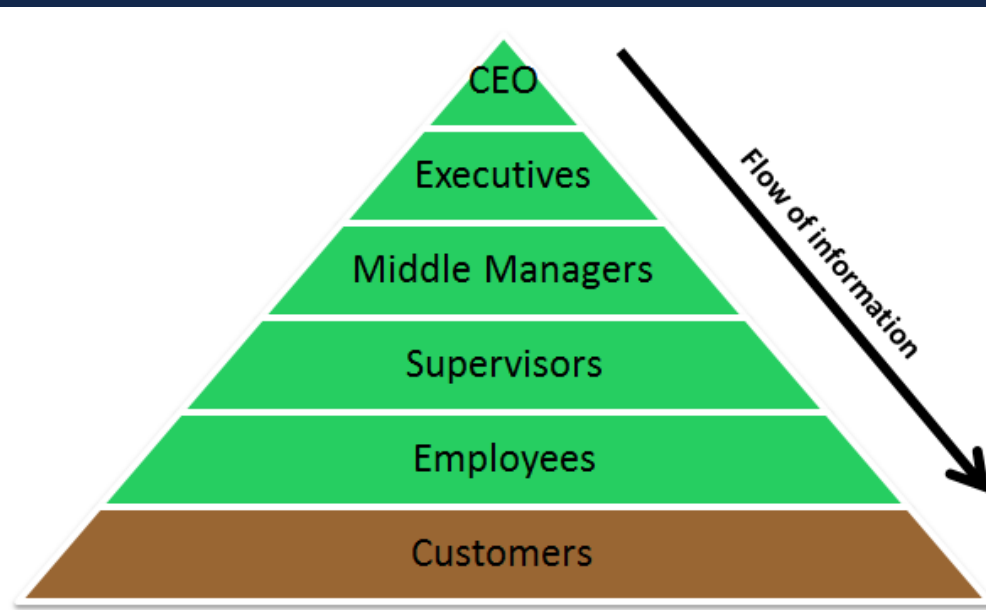
- Reality is when approaching problems you often need both approaches.
 - High level planning and work it out as you go!
 - Example: Writing my book(s)
- We will see a tension between the extremes in our SE models which suggest a continuum as opposed to an absolute blend
- Pragmatically I think there is not really a tension here but rather a sense of appropriateness
 - Which strategy to employ when and for how long

Top and Bottom



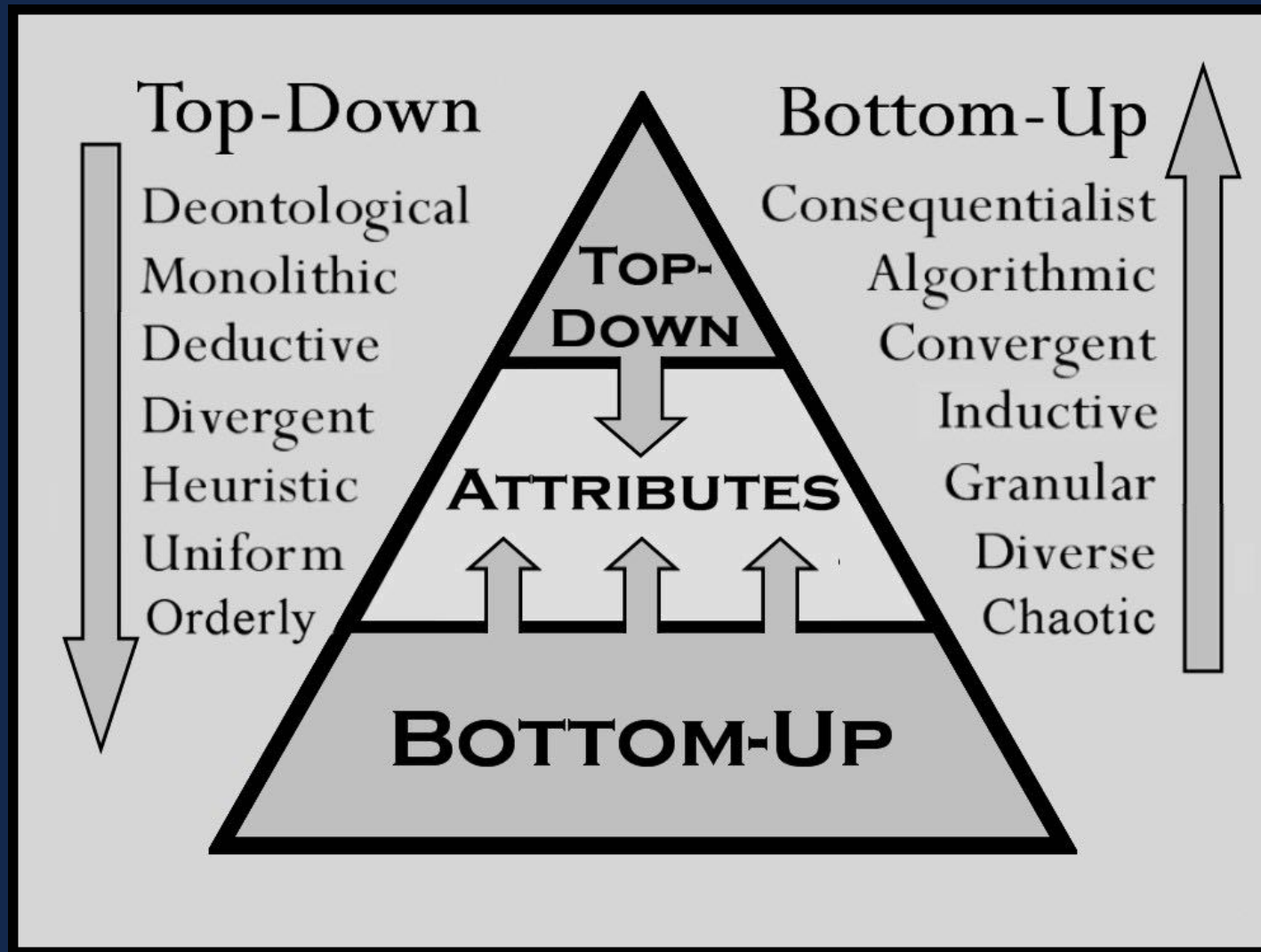
Slide Bonus: When you are making diagrams and data viz JPEG = FAIL

Top Down and Bottom Up in Biz

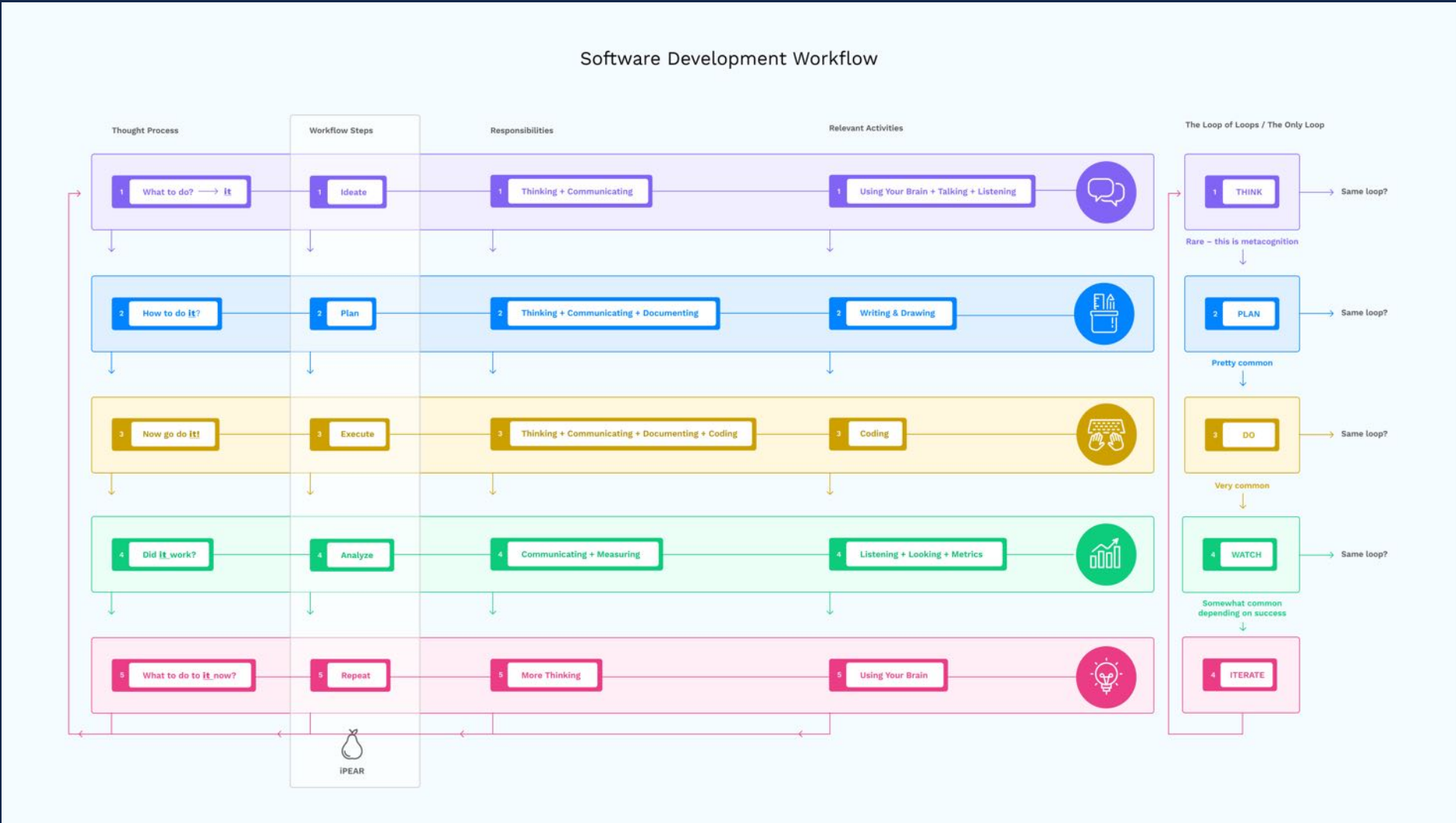


- Question: Why did we flip the pyramid instead of just adding an arrow?

Top and Bottom



The High-Level in SWE Summarized



Basics Level 2

Iterative and Linear Approach

Linear



Iterative



All At Once Delivery



Incremental or Iterative Delivery

Incremental
delivery



Increment 1



Increment 2



Full version

Iterative
development



Iteration 1

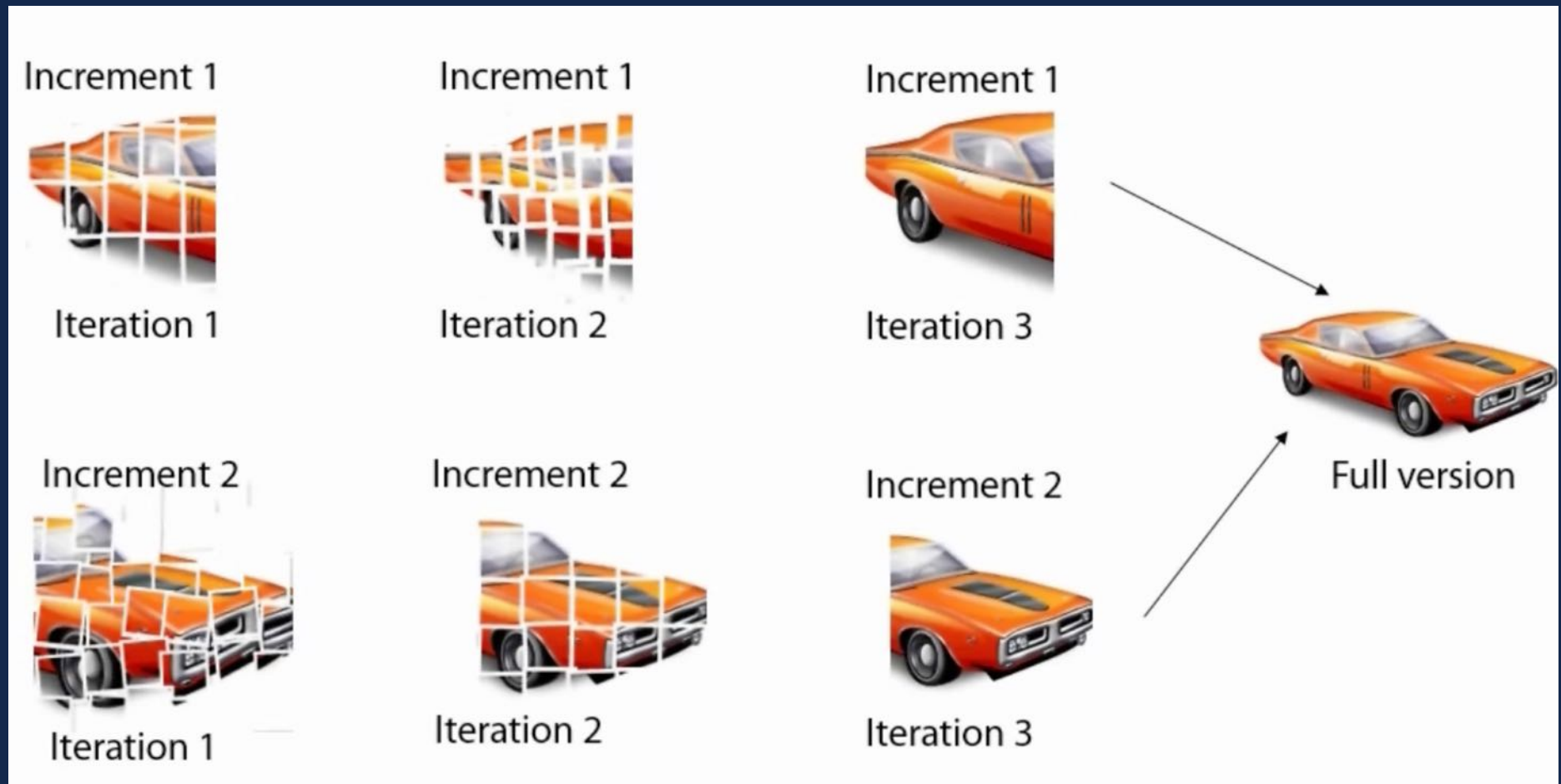


Iteration 2

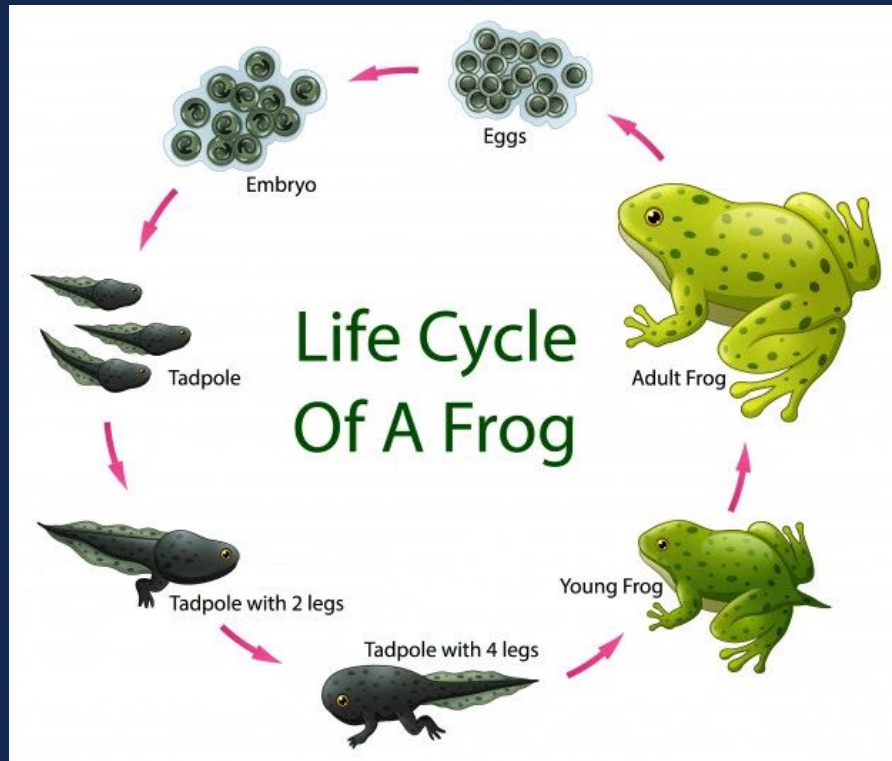


Final Iteration

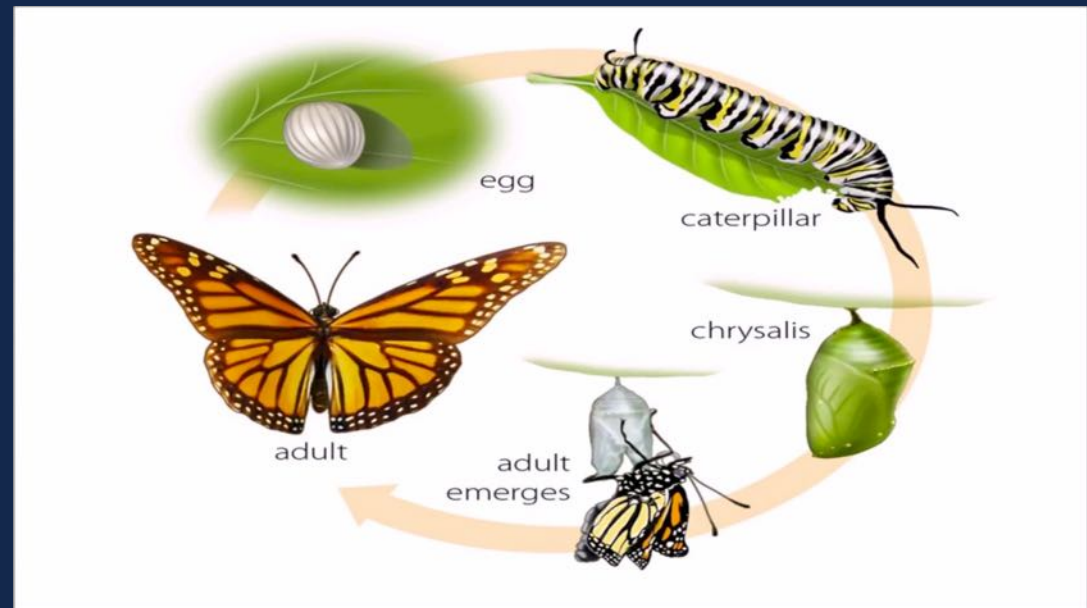
Incremental and Iterative?



Alternative Metaphors

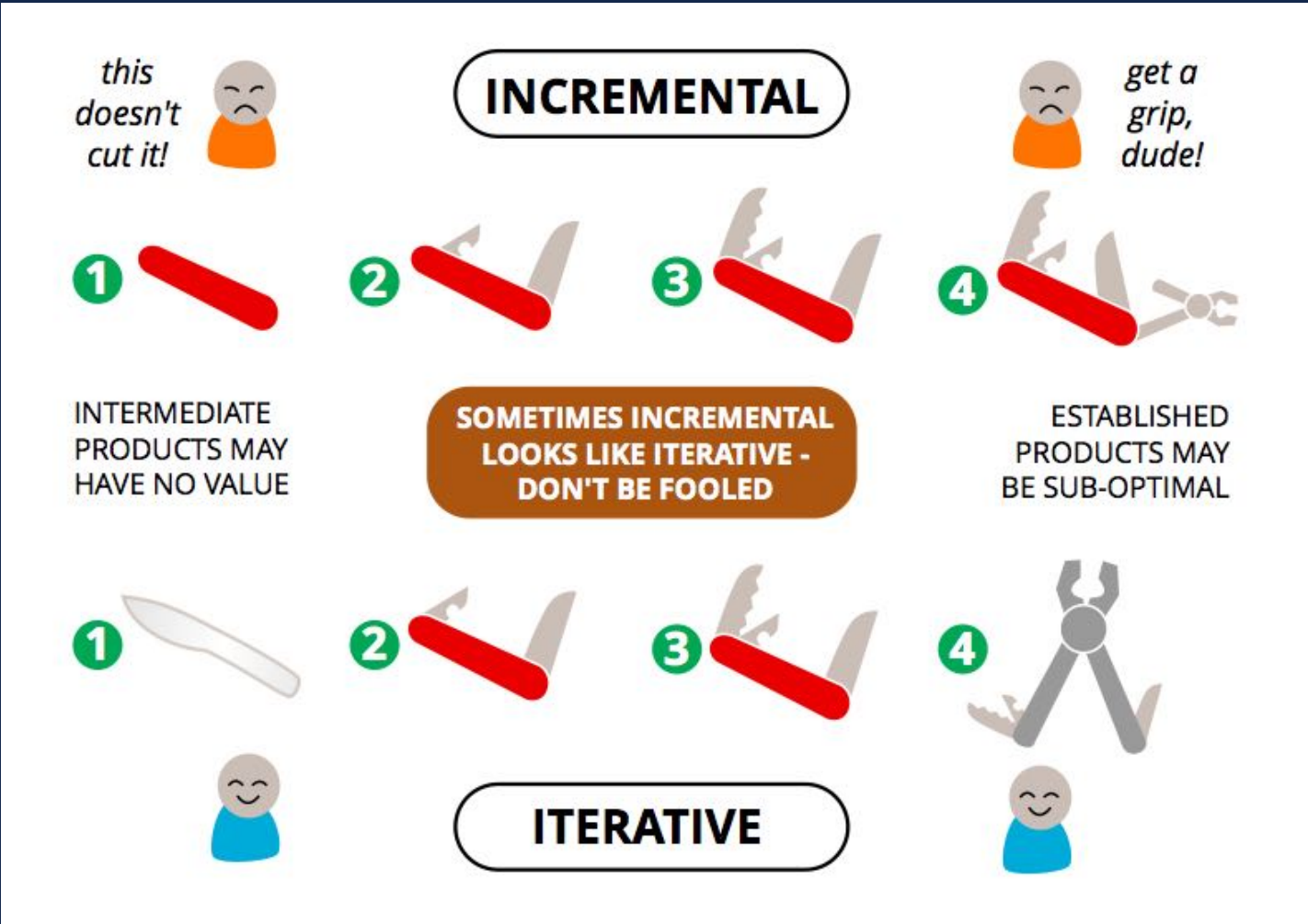


Reaction 1

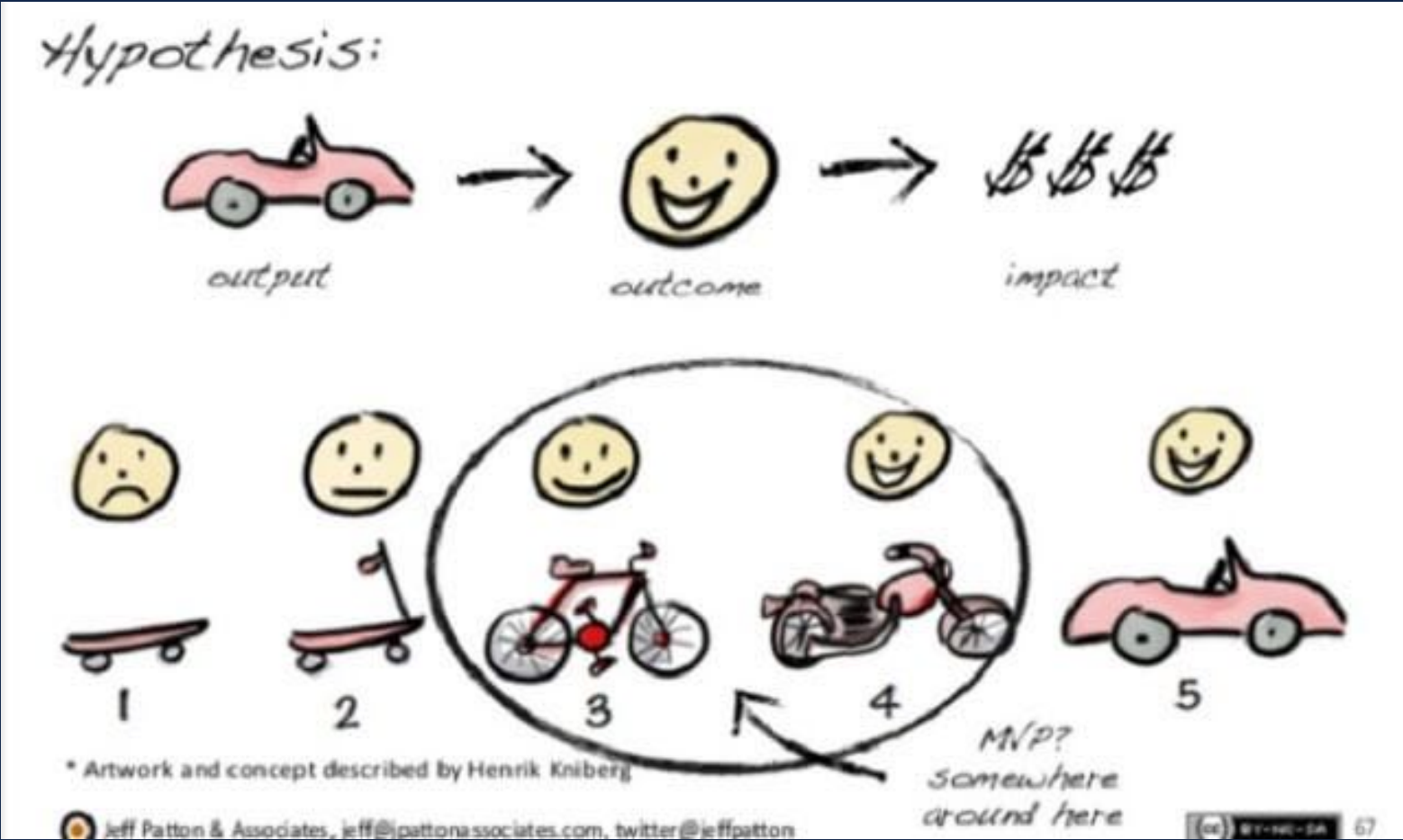


Reaction 2

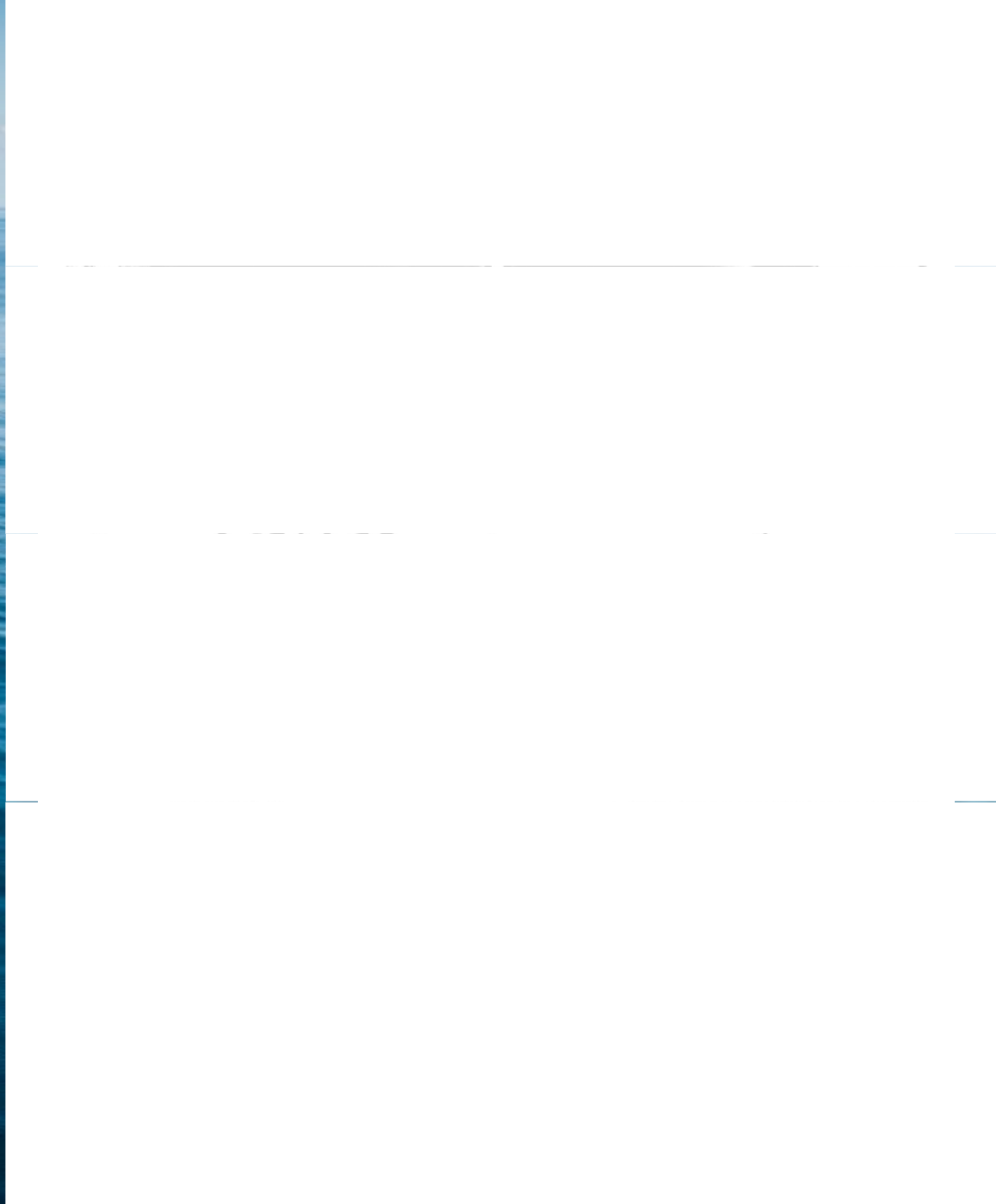
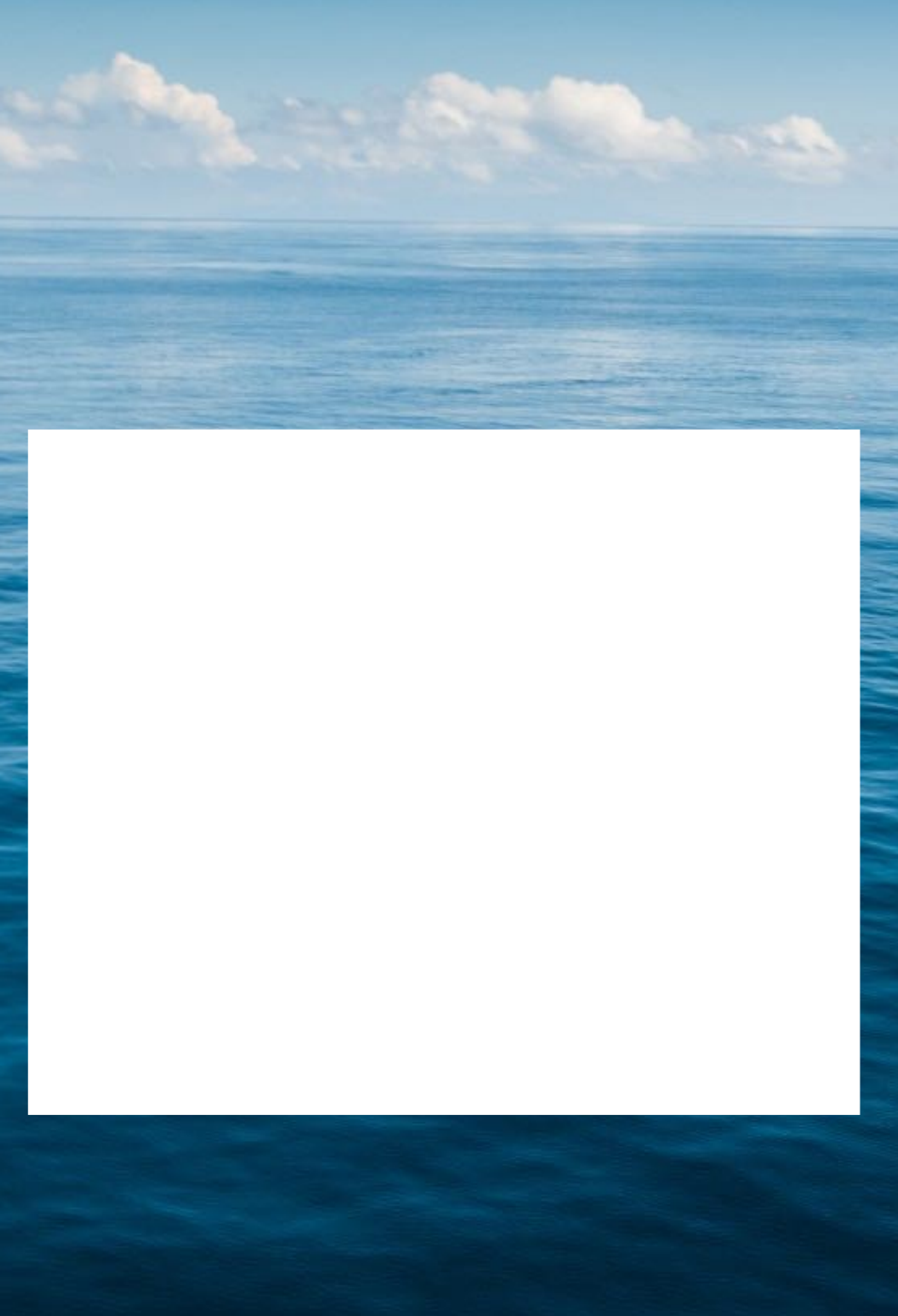
Take 2



Point of Viability Reality



All Aboard U.S.S. Wrong Approach?



It Always Depends

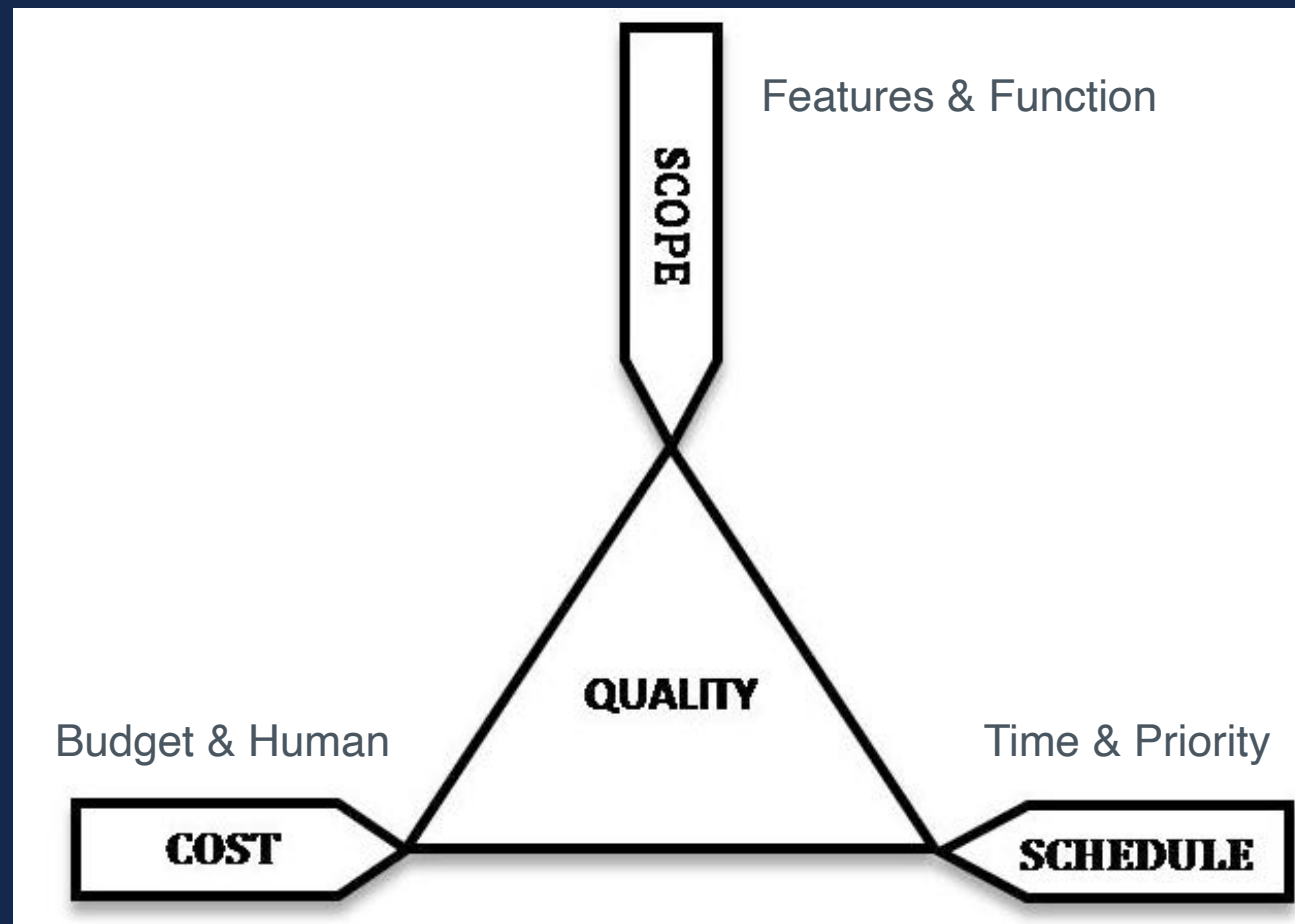
- In practice my project and product experiences suggests there is no one true approach
 - top down/bottom up, iterative, incremental, formal, informal, etc. it always depends
- Example of early release and incrementalism doom
 - Politics, significant minimum requirements, etc. (think appropriate time for MVP idea on last slide)
- Example of inefficiency of iteration and incrementalism
 - Often the iteration approach results in an initial architecture that may not be appropriate long term. An easy evolution may not be possible and you may need a chrysalis approach to do the re-architecture necessary to support change

One Size Rarely Fits All Contd.

- Depending on situation, circumstance, type of product, people involved, etc. SE methodologies make no (or make perfect) sense
 - Airplane control software
 - Concerned Dev - “Maybe we should really plan this out”
 - My hot new social software for dogs
 - CEO - “My gut says this is a winner! We need to get it out there!!!!”
 - A company that is extremely fiscally driven or has a very command and control structure
 - “Agile won’t fly here - they just don’t buy into it”

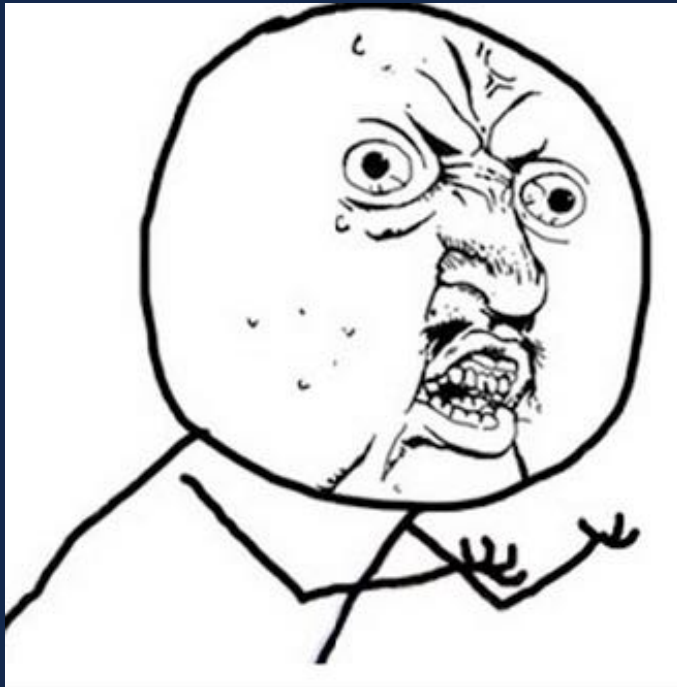
Basics Level 3

An Iron Triangle Appears

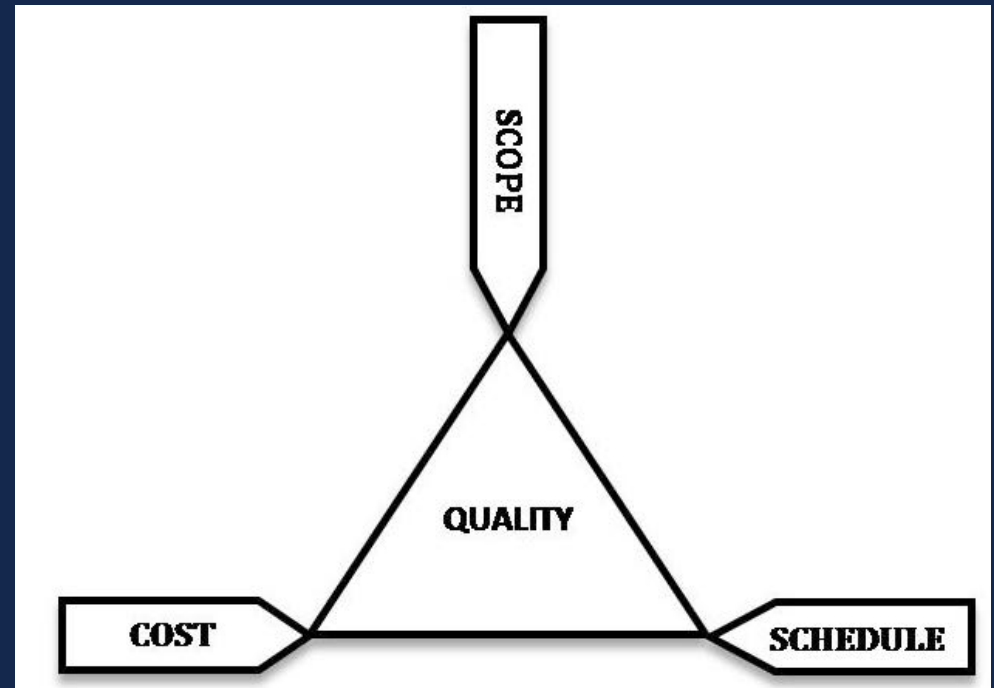


Pull a handle it effects other and of course interior

Yet for many...



Y U No Give Me ALL!?



the Iron Triangle represents a kind of “*project management physics*”. That is, *your project is going to follow the laws of the Iron Triangle whether you’re conscious of it or not.*

Iron Triangle Demonstration



Keep the time fixed and the feature goals the same and observe how the quality is effected as one would expect

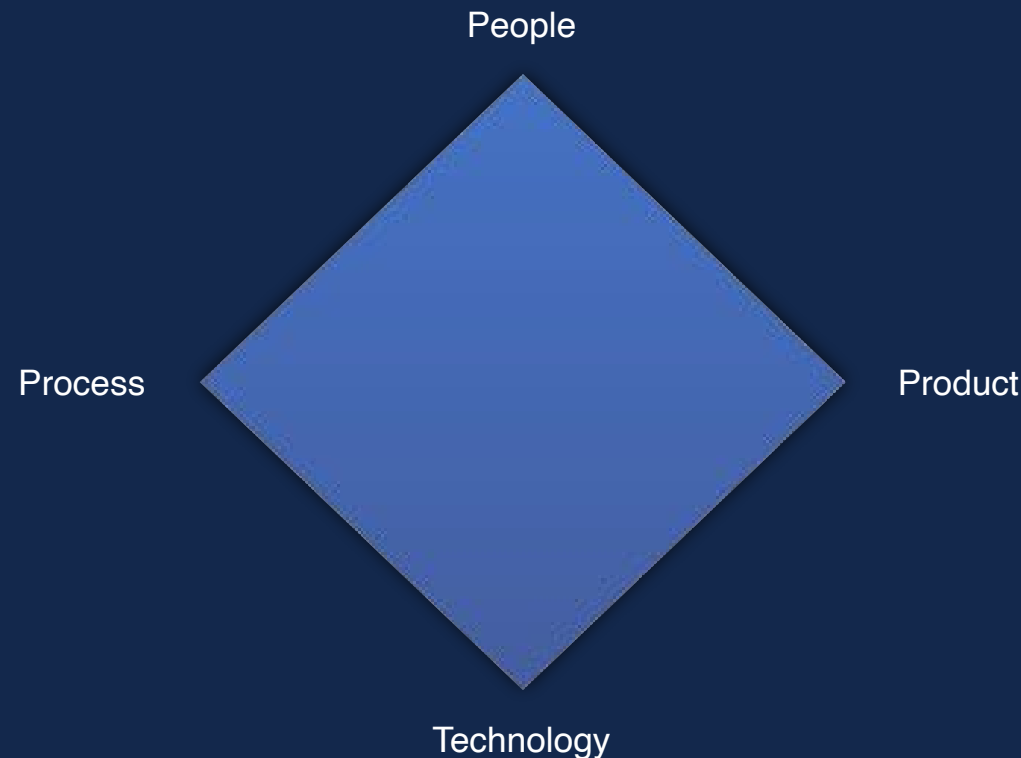
It's Triangle Trade-offs

The reality is that a good project manager understands that these four attributes have coefficients. A good manager drives a project to be good enough, fast enough, cheap enough, and done as much as necessary. A good manager manages the coefficients on these attributes rather than demanding that all those coefficients are 100%. It is this kind of management that Agile strives to enable.

Clean Agile
Robert C. Martin

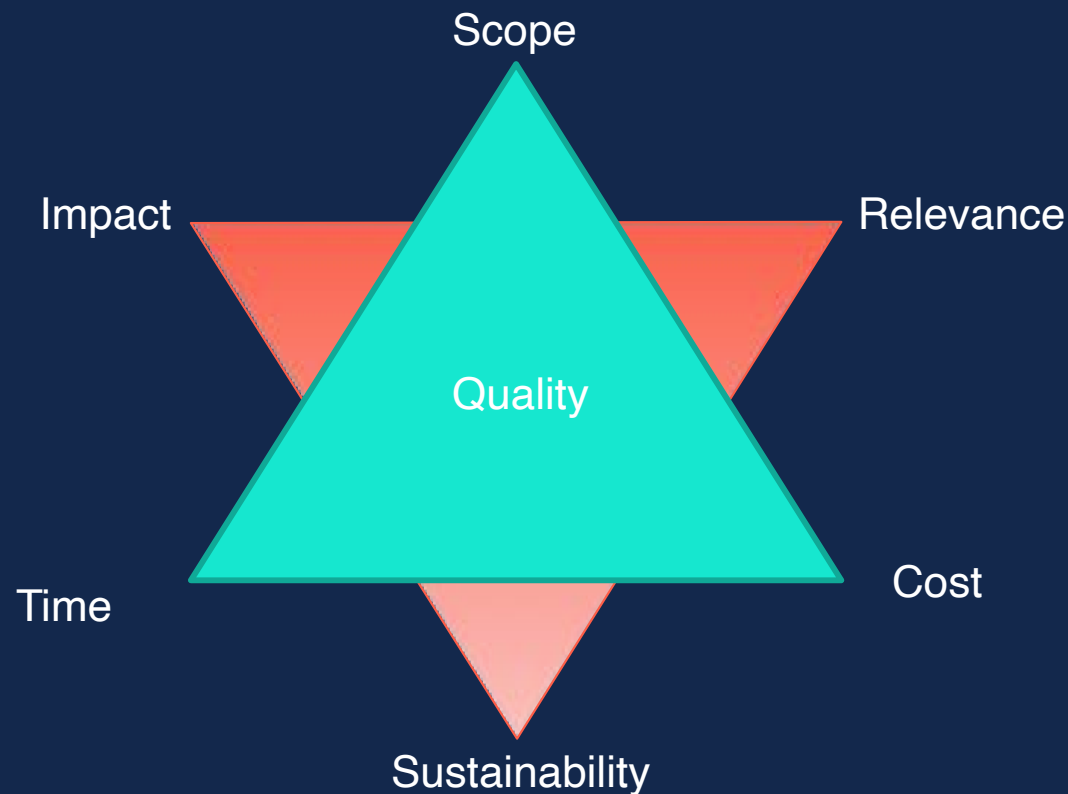


Change shapes or increase model complexity, but tensions remains - Part 1

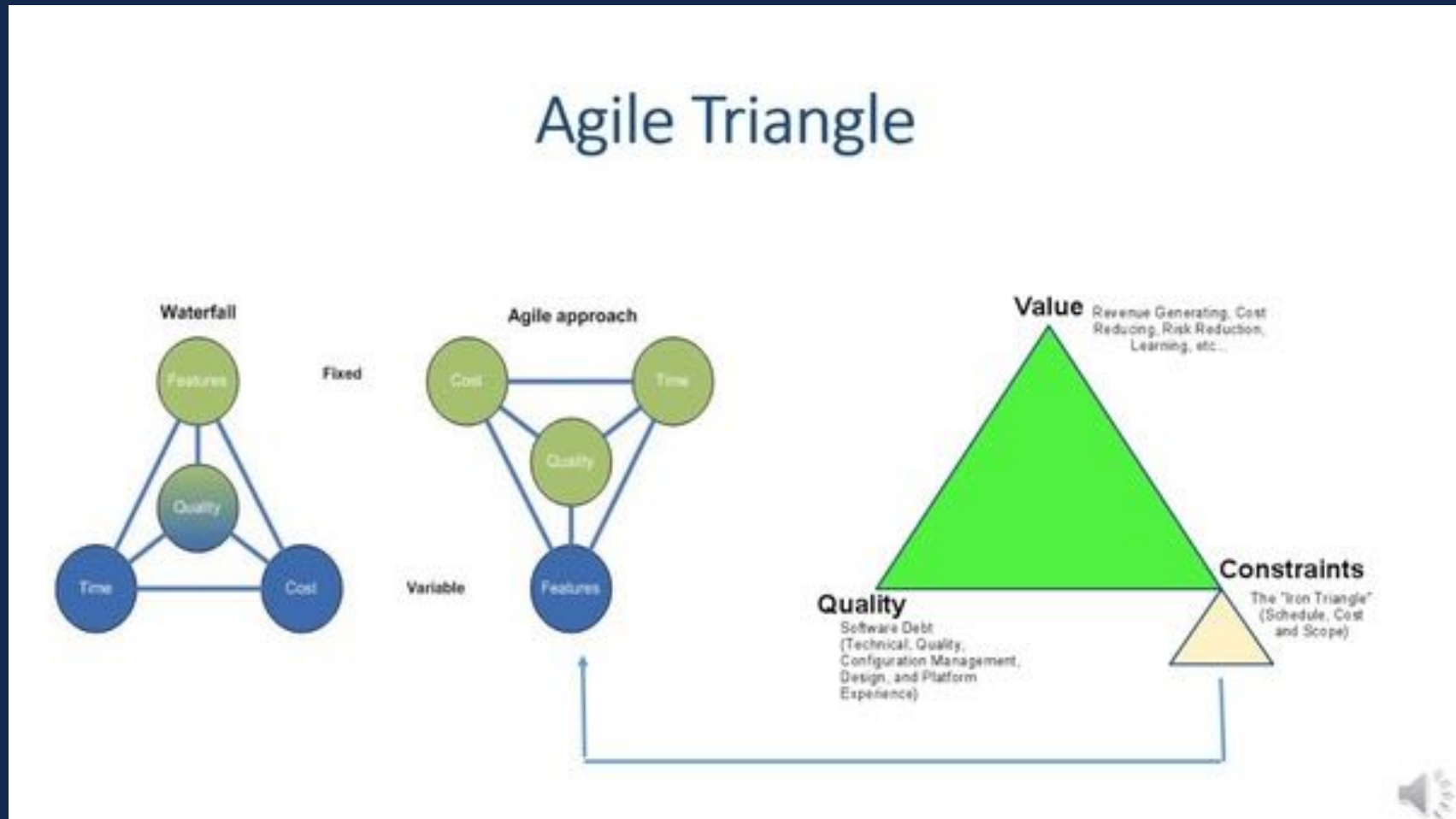


Easy Question I hope: *Which might be the hardest to deal with?*

Change shapes or increase model complexity, but tensions remain - Part 2



Change shapes or increase model complexity, but the tensions remain - Part 3



Professor's Favorite Word

- The Professor's favorite word that is the law above all law's in SE (or maybe life)
- There are always _____
- Magical thinking
 - “Middle Earth Engineering” – thinking there is one master solution
 - Gandalf (and me) says “You shall not pass!!!”
 - Also you might not ship and if you do it is likely pretty awful

Methodology/Tech/PL will Solve This!

- Pretty likely to be encountered
 - What we do is hard and when frustrated we might be quite susceptible to poor analysis and wishful thinking
- Current Internet Risk Level - Extreme
 - Be careful of falling into this thinking “X employ Y and X is big/rich/popular so I should do/use Y”



Social is not the best source and contributes to a frequency illusion



Nobody legit uses Y (per the tweets)



> 70% of sites use X (per the data at scale)

Does this hold outside of tech preferences? YEEEESSSS!!!!!!

Warnings About Agile Process Before We Start

- The last slide wasn't about tech it started there to show the broad thinking which applies to process models as well which are just as plagued with magically positive (and negative) thinking
- Traditionally I could summarize such simplistic thinking as “waterfall = old, slow, dumb and agile = new, fast, smart”
 - So let's take a tour of the headlines in my Medium search....WUT!?!
 - It seems to be filled with more negative than positive now! Why? Clickbait, smoke/fire, or it always depends and maybe this is trade-off hangover here for the “one true process” pitch is the most likely answer.

Basics Level 4

Exploring the Tensions

- Each aspect of our triangle/diamond/⟨shape of choice⟩ has dimensions we might explore
- We can look at time, scope, people, quality, etc. and see if we can notice tensions within the tensions

Time: Respect the Law

4: Design

Staging
Setup

API
Goals

- Laravel + Bootstrap

Initialize
Database

Normalize
Database

Create
Fixtures
for
testing

← Create Fixtures

Hofstadter's Law

It always takes longer than you think

X 6: Dev

B/E Sorting
(for All
Dashboard
Views)
w/ pagination

B/E Sorting
(for all views)

Create
all DB
Tables

CRON

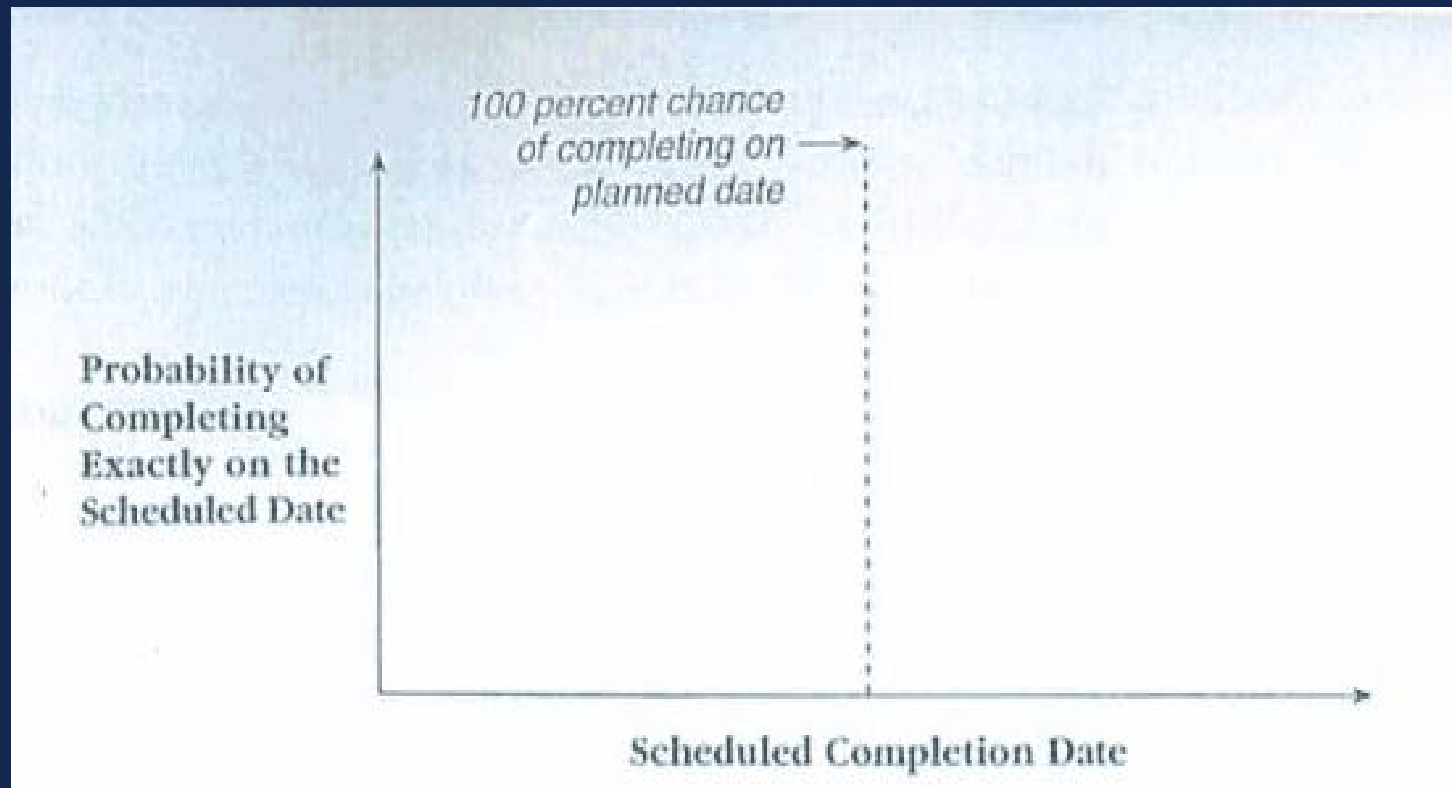
Automated
(Send)
email
B/E

ADMIN

API

Time: Scheduled Insanity

It's never this...



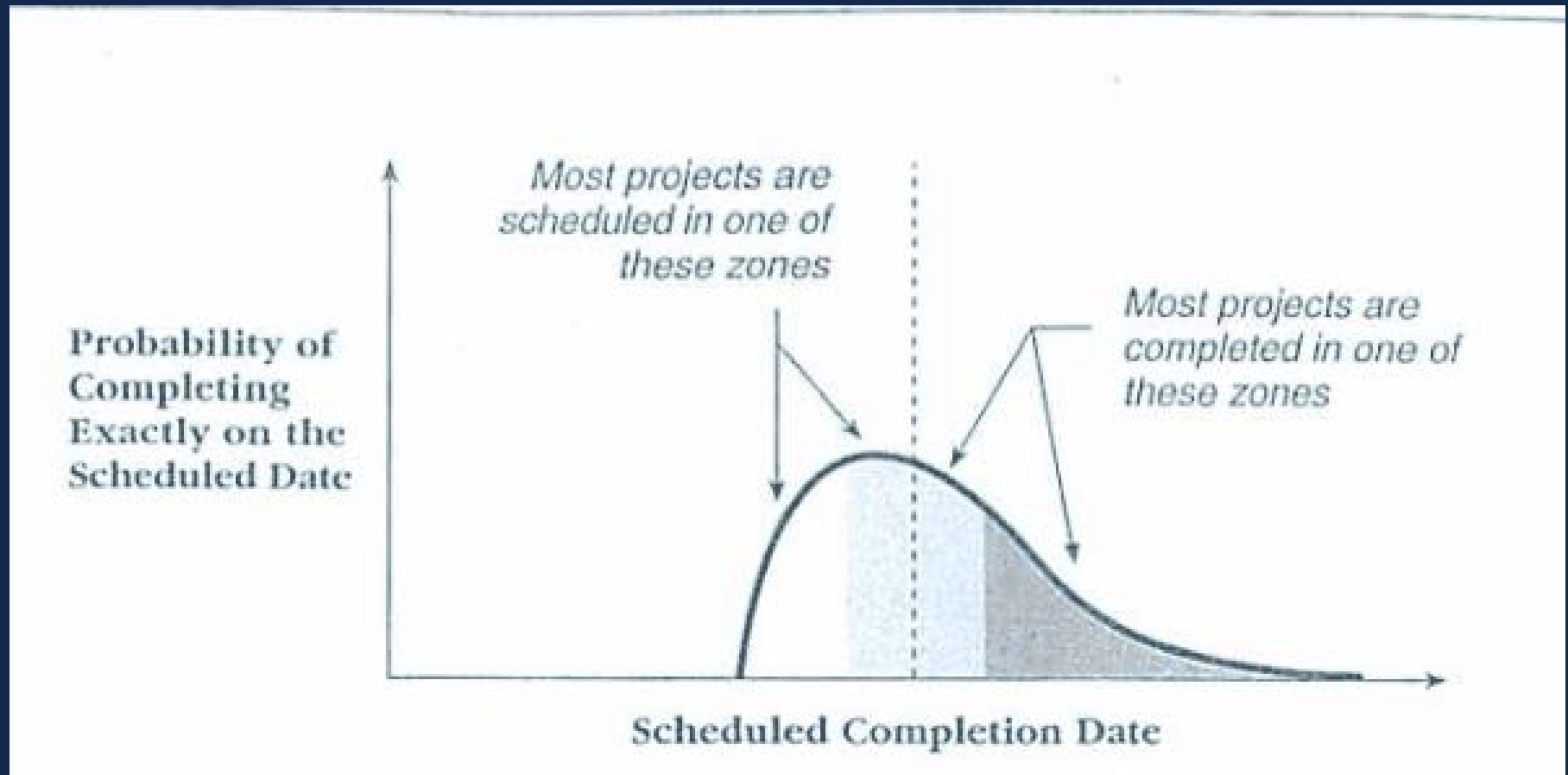
Ch 6, Rapid Development, Steve McConnell

Time: It's Done When It's Done

- Blizzard approach
 - Nice if you have the capital and buy-in from stakeholders and customers to act like this
- Can lead to laziness and vaporware
 - http://en.wikipedia.org/wiki/Duke_Nukem_Forever
- Generally a no schedule approach just won't work
 - ...but loved to see you be in the position to do this (nice work if you can get it!)

TL;DR - Also as insane as the previous line of doom approach

Time: Reality in Ranges



Ch 6, Rapid Development, Steve McConnell

Time: Pressure Effects

Under pressure, teams give up the wrong things. They don't test enough; they leave the code in poor condition. This reduces value, increases the delay to getting the value, and reduces the value they can deliver later.

Under pressure, teams test less and therefore put more defects in. Some of these defects escape the room and affect customers. Value is directly reduced.

The Nature of Software Development
Ron Jeffries

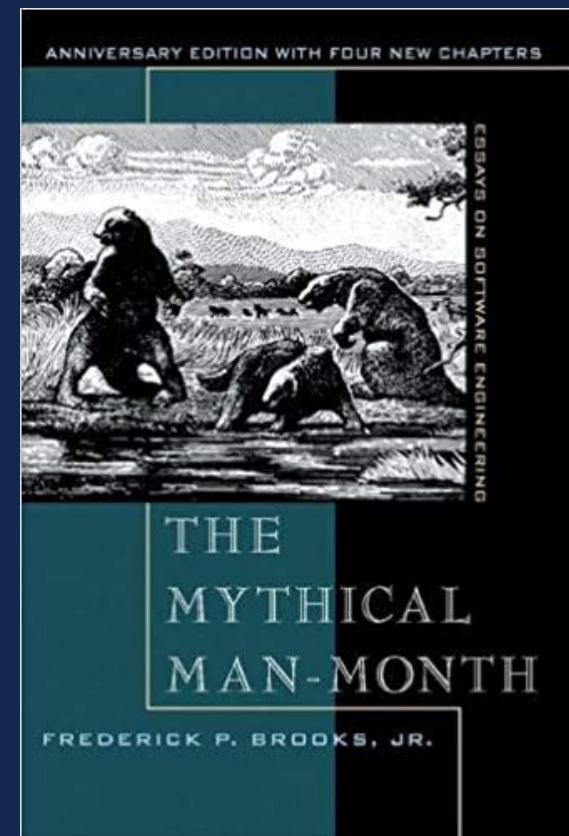


Time: Pressure Solution?



Time: When Late Always Respect Brook's Law

- Adding people to a late project doesn't generally deliver desired results
 - On-boarding time is generally larger than assumed
 - Communication path multiplication
 - We lack the shared hive mind to recover history and details necessary to be productive quick enough

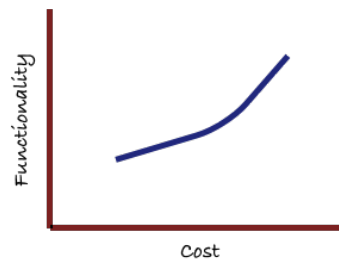




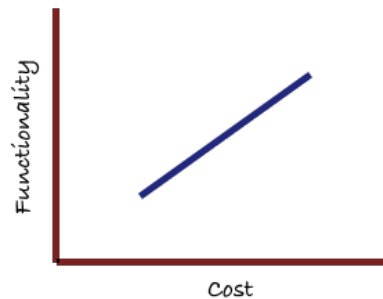
Some people have called the book the bible of software engineering. I would agree with that in one respect: that is, everybody quotes it, some people read it, and a few people go by it.

(Fred Brooks)

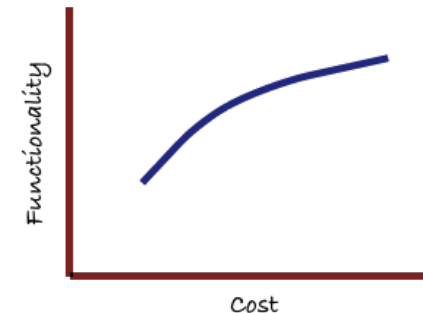
Scope: Do You Need More?



A) Features Cheaper Over Time



B) Features & Cost Stable Over Time



C) Costs Rise For Features Over Time

Actual experience says C, Idealism says A, Realism Strives for B-ish

Scope: Respect the Law

Dietzler's Law

most things
for framework ~~X~~:

80% of what the user wants is fast & easy

the next 10% is possible but difficult

the last 10% is *impossible*

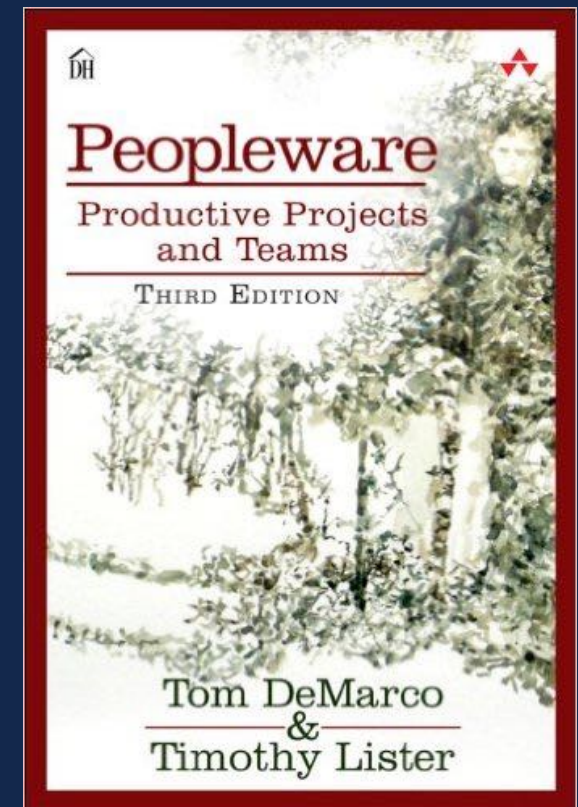
users want 100% of what they want

Scope: Some Common Risks

- Unrealistic expectations
- Too many competing requirements
- Feature creep - “just one more little thing”
- Competitor reactions
 - Checkbox Marketing responses
- Consumer Loss Aversion
- Developer Feature/Tech FOMO

People: The Biggest Tension of All

- Peopleware - a term describing role of people in dev and use of computer systems
 - productivity, teamwork, group dynamics, HCI, social effects, market effects
 - Plain old garden variety irrationality and inconsistency
- The biggest variable in your SE life, hard to learn this without experiencing it
- We've given you a flavor of this immediately in this class and it will continue throughout the course (and your career)



People: Common People Risks

- Weak personnel
- Heroics
- Negative personalities
- Wishful thinking
- Politics
- Inappropriate work space
- Lack of buy-in, patrons, etc.

Team Organization Broad Concerns

- Should we all be equal partners?
- Should we all do same thing or different things?
- Should we have a lead developer with assistants?
 - Master chef, surgeon, etc.
 - Should the “big boss” just direct or work?
- The process models will actually have some things to say about this

Quality: On Good Enough Choices

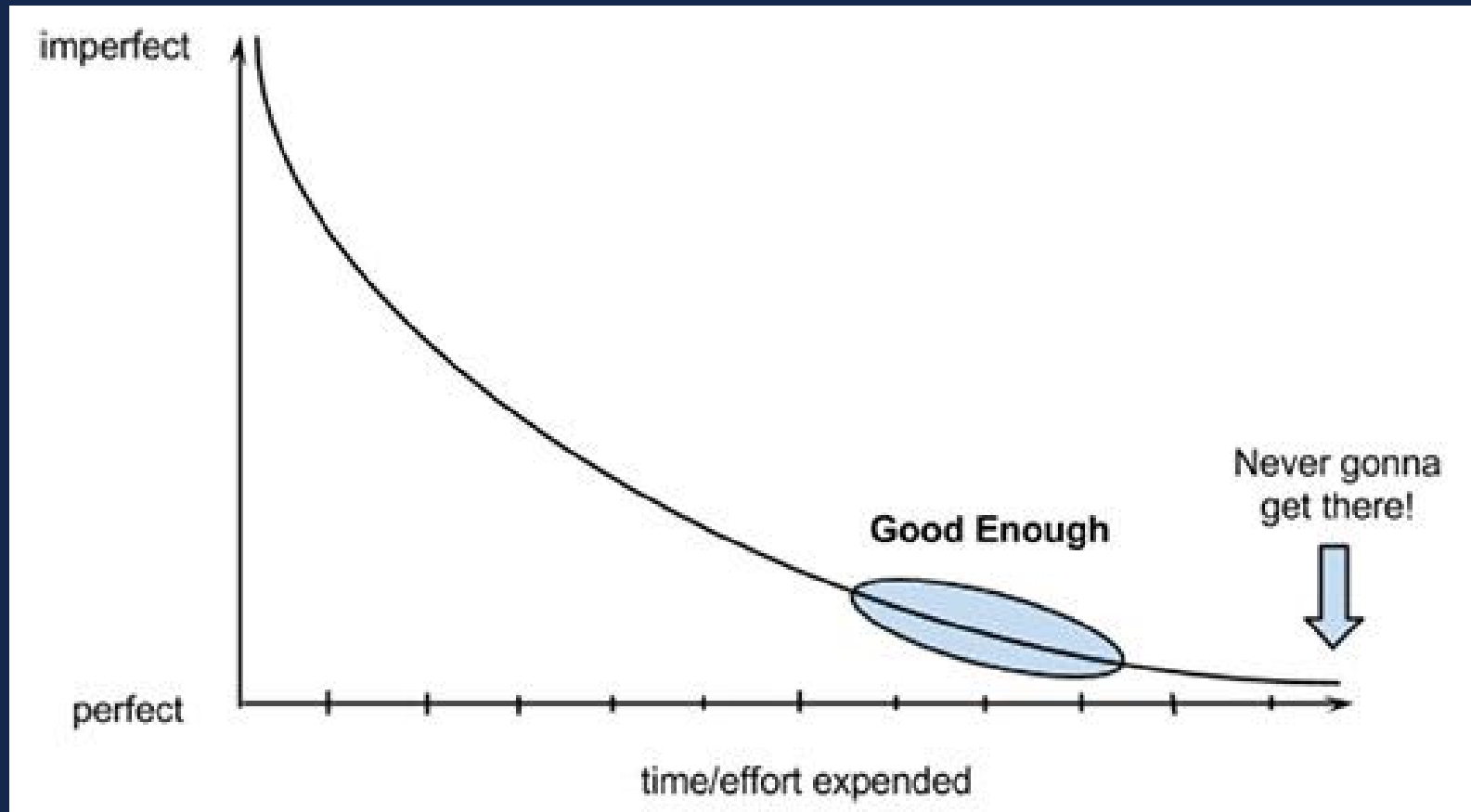
Good choices are just choices that are “good enough” rather than perfect. Such choices likely have cons, but the pros outweigh the cons and the picked solution fits the current situation. If the situation changes then the choice may need to change too. Good enough given what I know. Good enough for now. Just plain good enough, not perfect.

Quality: Obvious and Old Idea



https://en.wikipedia.org/wiki/Perfect_is_the_enemy_of_good

Quality: Pragmatism Visualized

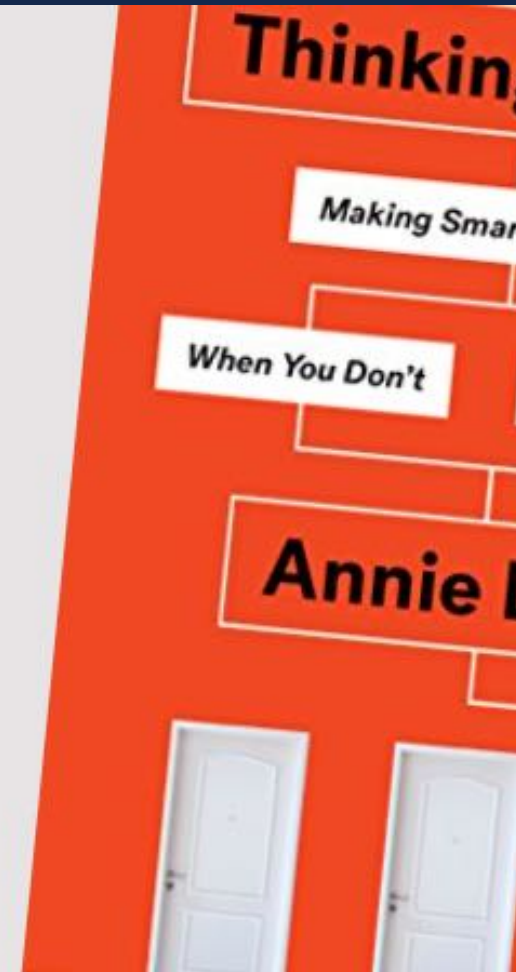


Quality: When Deciding Anything: Mostly Just Avoid Being a “Resultist”

Decisions are bets on the future, and they aren’t “right” or “wrong” based on whether they turn out well on any particular iteration.

Thinking in Bets

Annie Duke



Much can always go wrong

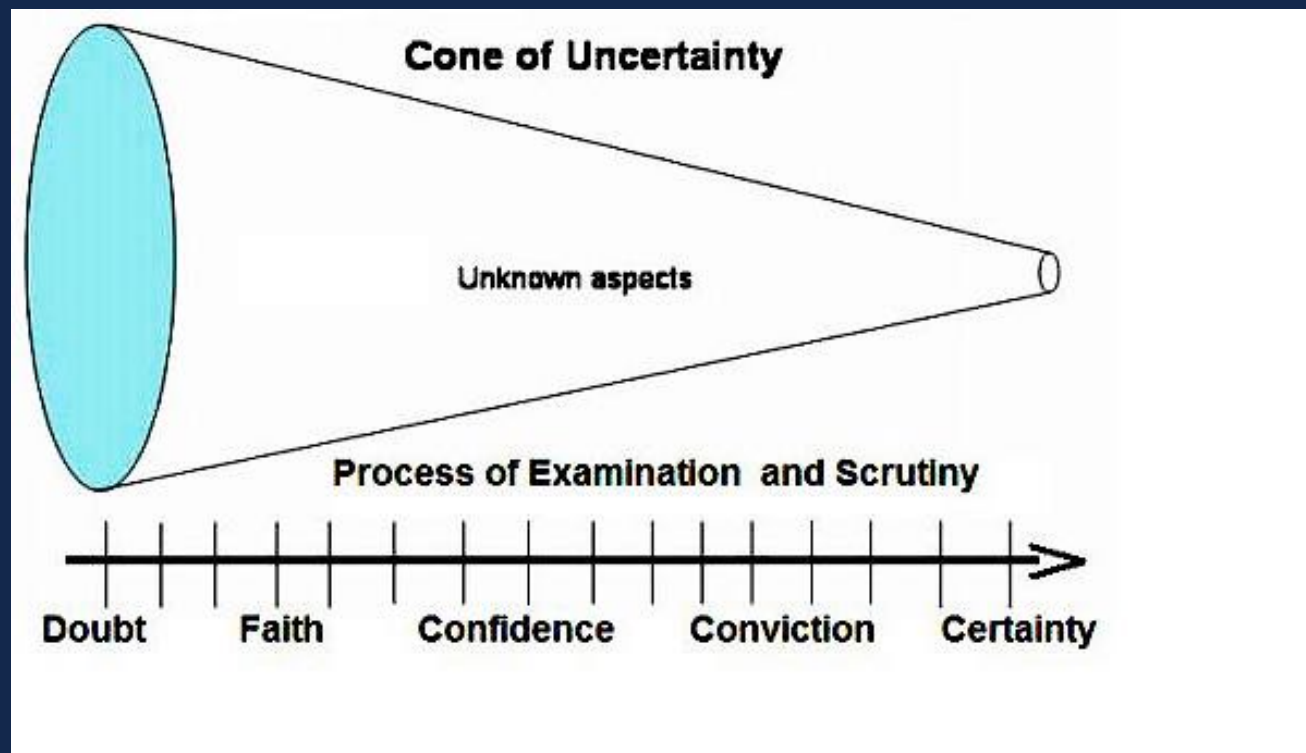
- Some of this can be learned
- Some of this truly must be experienced (sadly) even beyond the peopleware
 - At least know it is coming!
- Have realistic expectations for yourself and relative to others - “SE Nirvana is a myth”
 - Now let’s explore the two general approach types respectively
 - the specifics like XP, Agile, Waterfall, etc. will be instances of these general two approaches we have seen actually already

Basics Level 5

Uncertainty Boss

Facing the Cone of Uncertainty

I like this one because it shows the general nature of the idea as opposed to some specific SE aspect to this point



Exploring Specific Methods

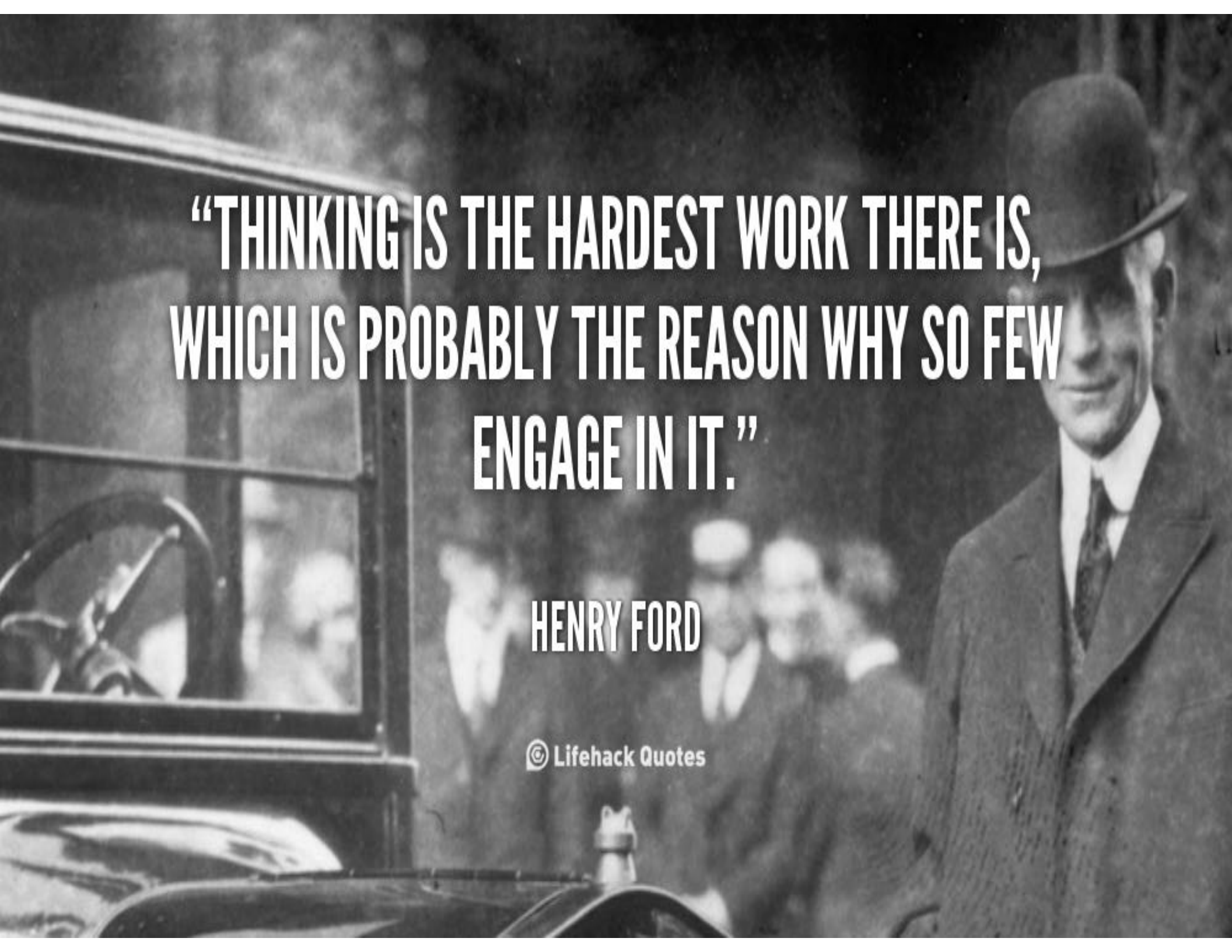
We Must Pick Something

- Software should be built
 - reliably
 - on time
 - on budget
 - with quality
- ...and not have it be a miserable experience

We need some amount and type of process!

Big Design Upfront (BDUF)

- Often when it comes to writing software we wonder should we do a “Big Think” first
- **Theory**: If you big think up front and plan things out the investment will pay off
- **Obvious Concern**: Can we know what is not knowable?
- **Obvious Concern**: Time taken to plan may outweigh time taken to do or fix.

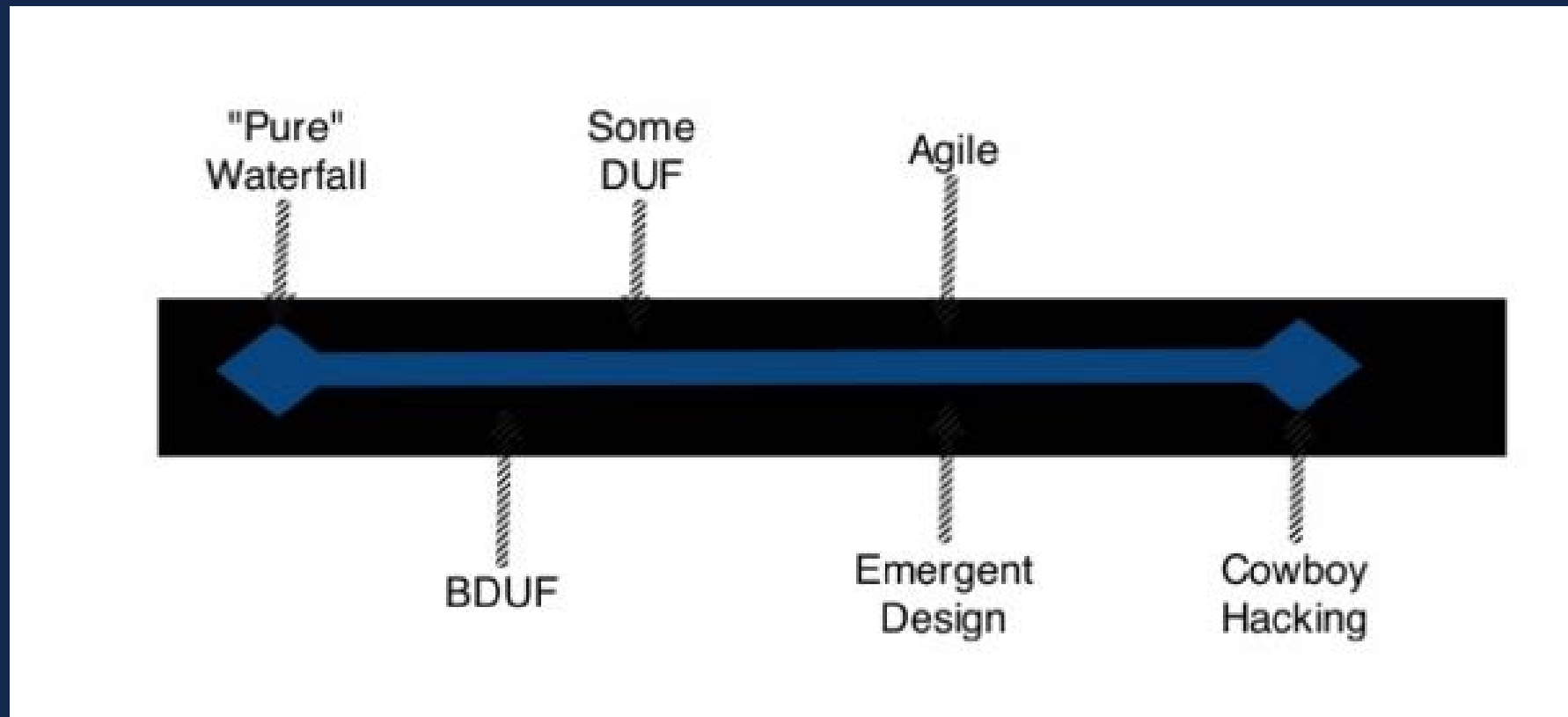
A black and white photograph of Henry Ford. He is wearing a dark suit, a white shirt, a dark tie, and a bowler hat. He is standing next to an early 20th-century automobile. The car's steering wheel and dashboard are visible on the left. In the background, several other people are standing, but they are out of focus. The overall scene is outdoors.

**“THINKING IS THE HARDEST WORK THERE IS,
WHICH IS PROBABLY THE REASON WHY SO FEW
ENGAGE IN IT.”**

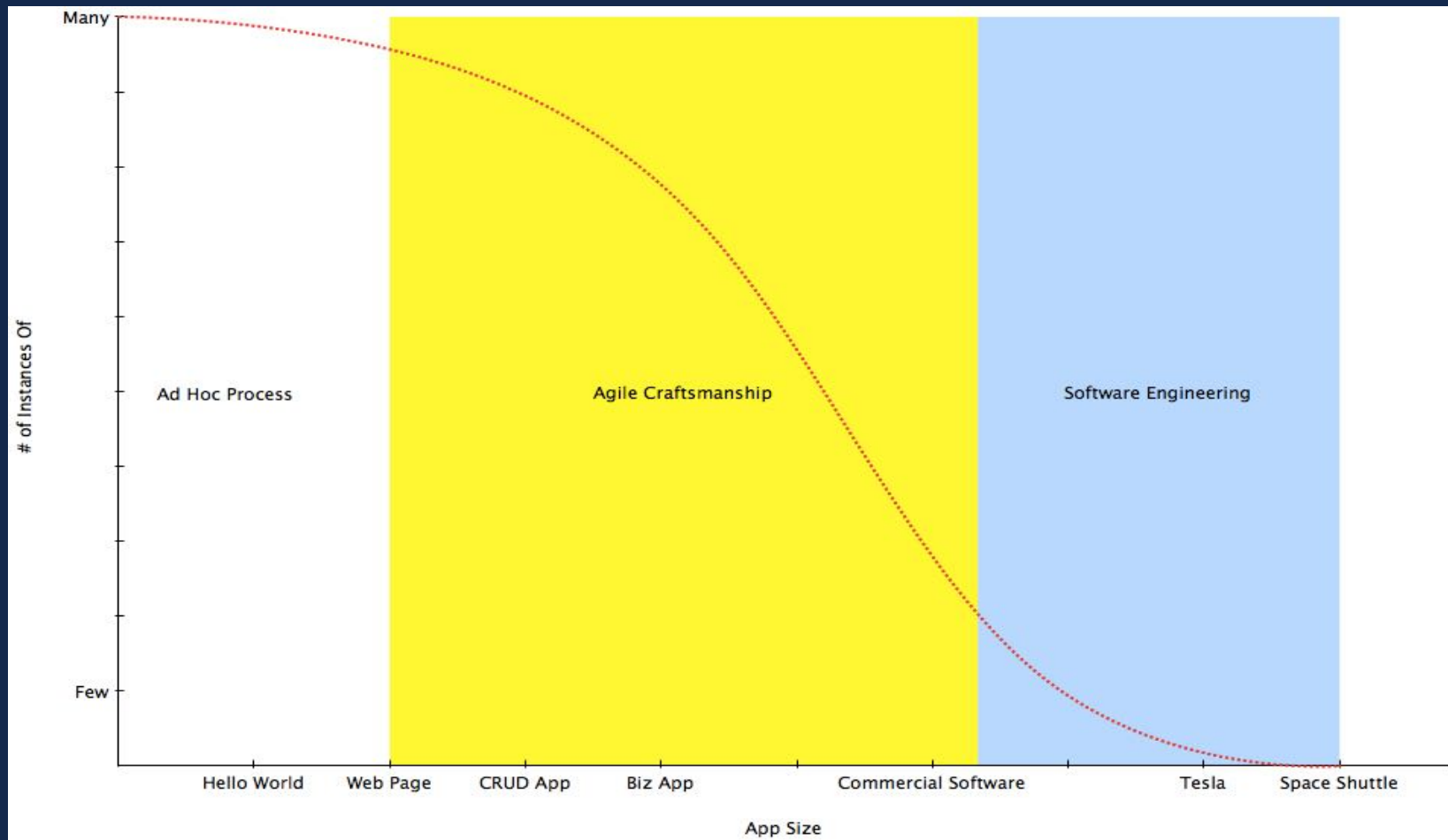
HENRY FORD

© Lifehack Quotes

Spectrum of Process



Meeting the Range of Needs



Ad Hoc Methods

Code and Fix

- Little design, just start coding
 - Comes from naiveté - “This looks easy.”
 - Schedule pressure - “It’s due Monday!”
- Favorite method of Cowboy coders
- Also a bit of fake Agile like the last story we just saw
- Q: Do we use this one at UCSD? Successfully?
 - Actually, yes and also yes*

Cowboys are dangerously cool?

- Code cowboys can be heroes sometimes
 - Freedom can inspire
 - Breaking rules! - introduce useful new tools, langs, ...
 - "Border crossings" from wild cowboys can be useful
 - Sometimes the approach is faster - no boss blockers
- Tremendous room for failure on many levels

Heroes Who Drop Process Become Inadvertent Villains

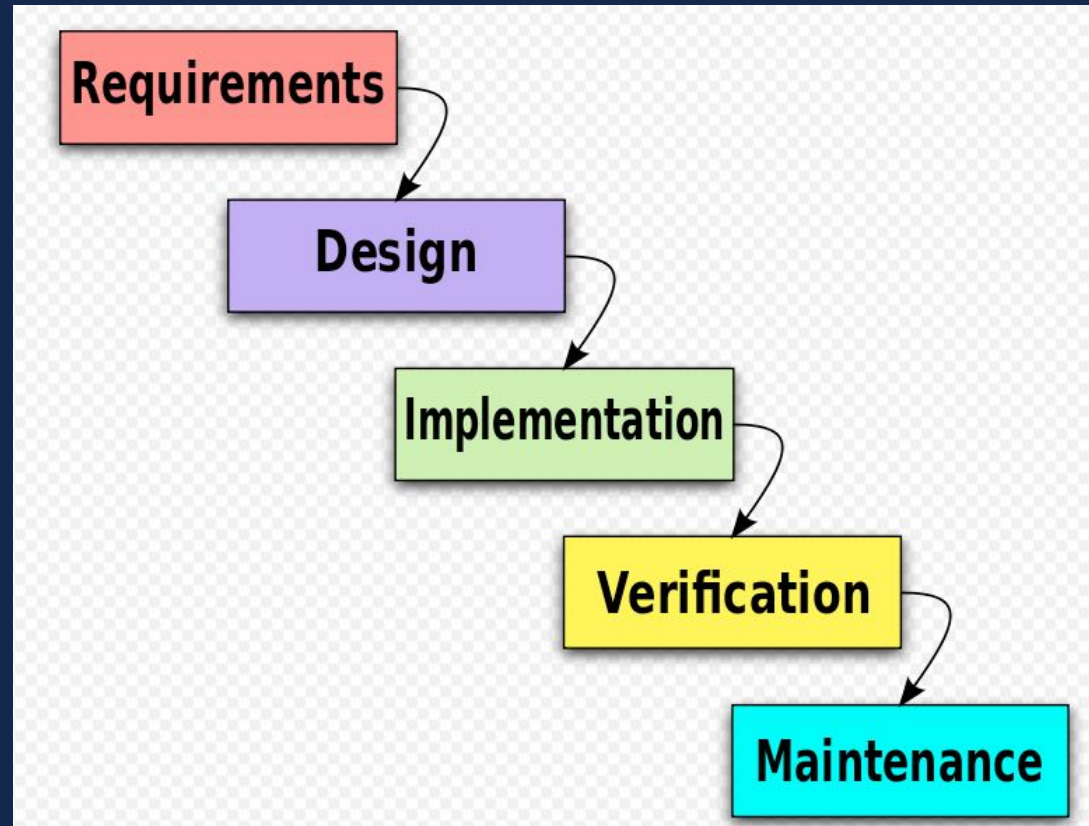
The temptation to be a hero and “solve the problem” is huge. What we all have to realize is that saying yes to dropping our professional disciplines is not the way to solve problems. Dropping those disciplines is the way you create problems.

The Clean Coder
Robert C. Martin



Waterfall Methods

Waterfalls



http://en.wikipedia.org/wiki/Waterfall_model

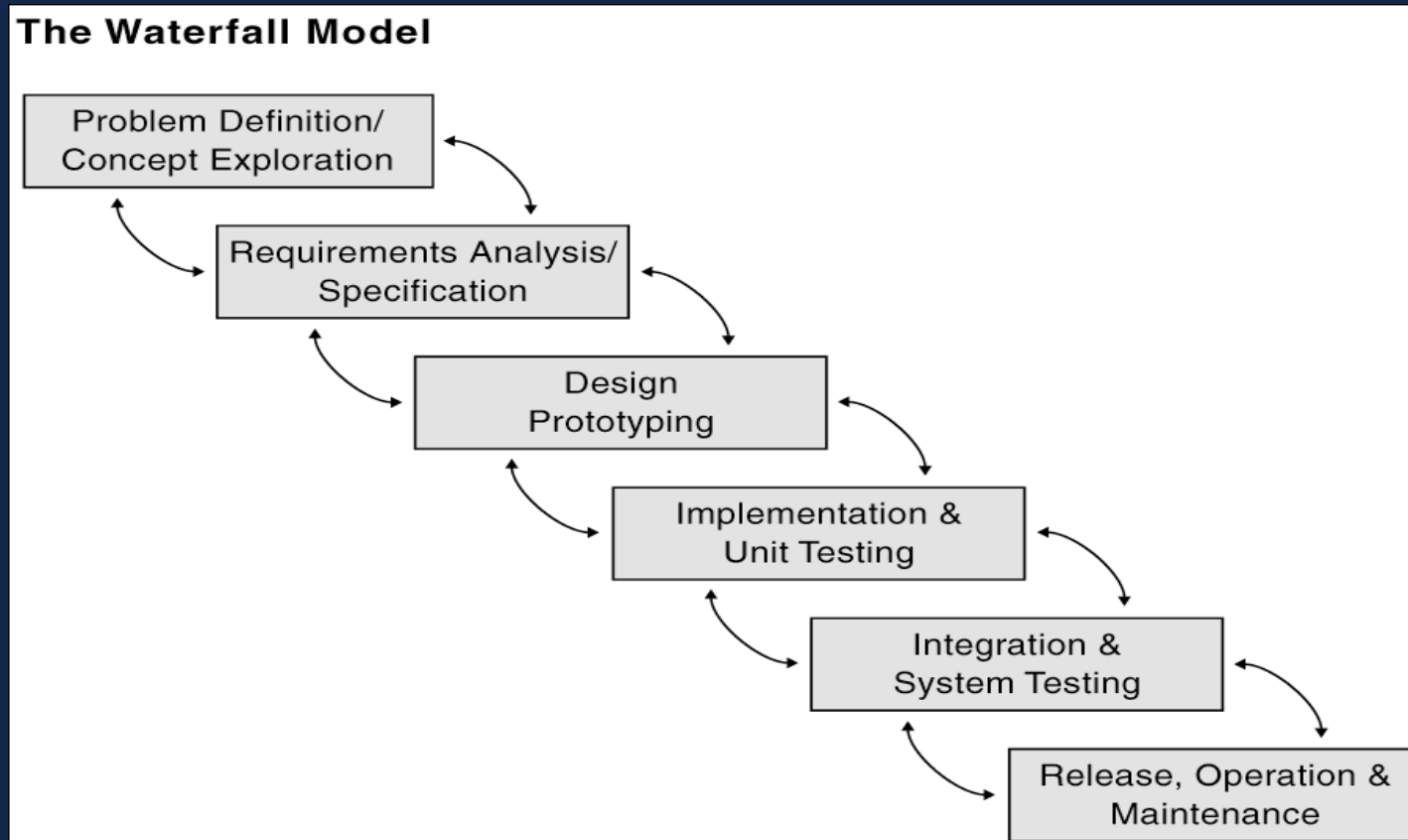
Waterfalls

Royce's Waterfall

1. Requirements Specification
2. Design
3. Construction aka “Coding”
4. Integration
5. Testing and Debugging
6. Installation
7. Maintenance

Waterfalls

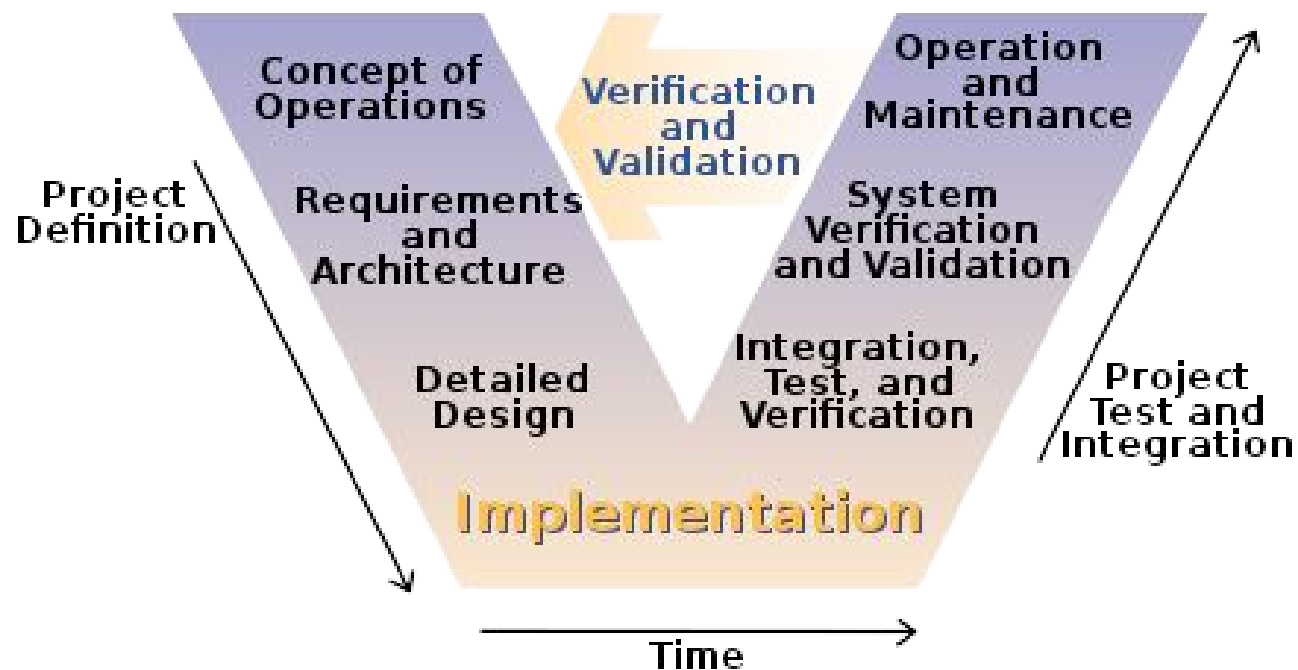
Web Site Engineering, Powell et al.



Waterfall Notes

- Number and name of steps is arbitrary
 - Unfortunately we will see discrete step boundaries not always clear
- Main point is a plan and document first before proceeding
 - Common methodology in “physical world”
 - Why?
 - In my experience with home building the idea of a pure waterfall is nonsense, there is plenty of “in field” modifications
 - Ike on the next slide says it all when it comes to plans!

Just Change the Shape!?



[https://en.wikipedia.org/wiki/V-Model_\(software_development\)](https://en.wikipedia.org/wiki/V-Model_(software_development))

Waterfall Notes

- Yes, the waterfall is simple to understand, that is the real reason I think it still works even in what you might consider in appropriate situations
 - It's a counter to the no process “make it up as we go along” and as such it is vast improvement
- Interestingly despite the Agile-istas thinking this feels right to those who cook, construct, etc. Maybe not it seems as if maybe they haven't seen the chaos in a kitchen or on a job site. So it might not be pure BTUF but at the very least ETUF

A black and white photograph of General Dwight D. Eisenhower, wearing a military cap and uniform, speaking to a group of soldiers. The soldiers are wearing helmets and combat uniforms, and are looking towards the general. The background shows more soldiers and some tents, suggesting a military camp or battlefield setting.

**“IN PREPARING FOR BATTLE I HAVE ALWAYS
FOUND THAT PLANS ARE USELESS, BUT
PLANNING IS INDISPENSABLE.”**

DWIGHT D. EISENHOWER

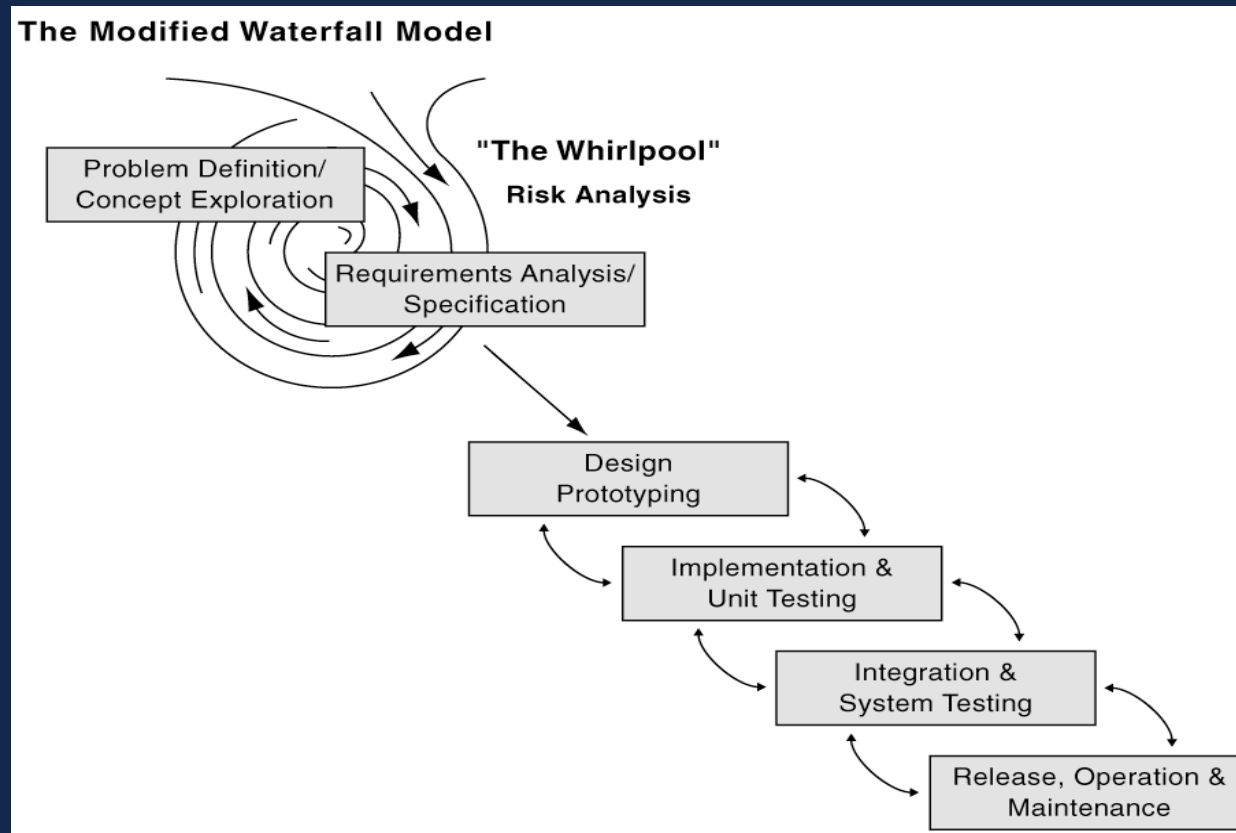
Modified Waterfalls

- Generally there is concern about risk reduction and dealing with future info
 - Try to avoid jumping in and doing the wrong thing
 - Try to incorporate things you learn on the way to adjust project
- A real stupid sounding truth

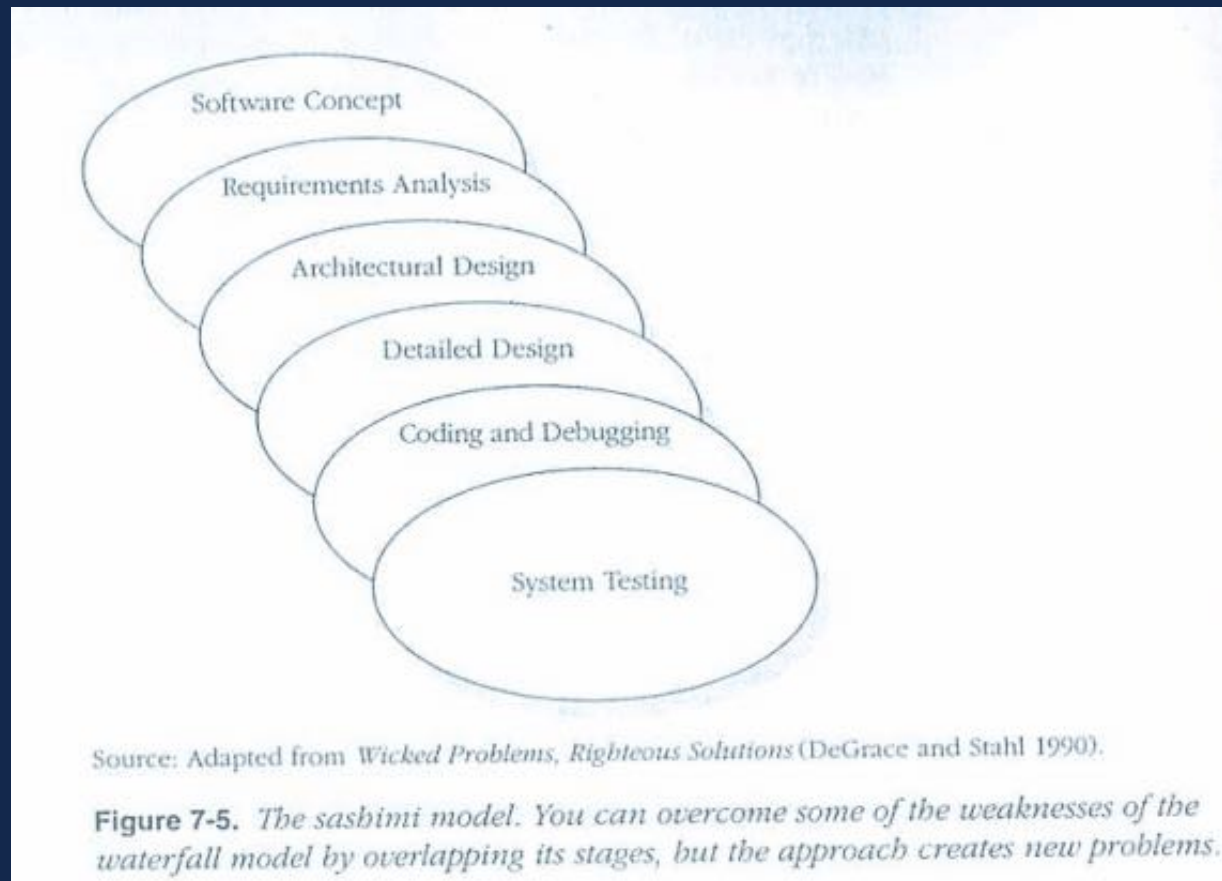
“You just don’t know what you don’t know until you know it.”

Modified Waterfall #1

Web Site Engineering Powell et al.

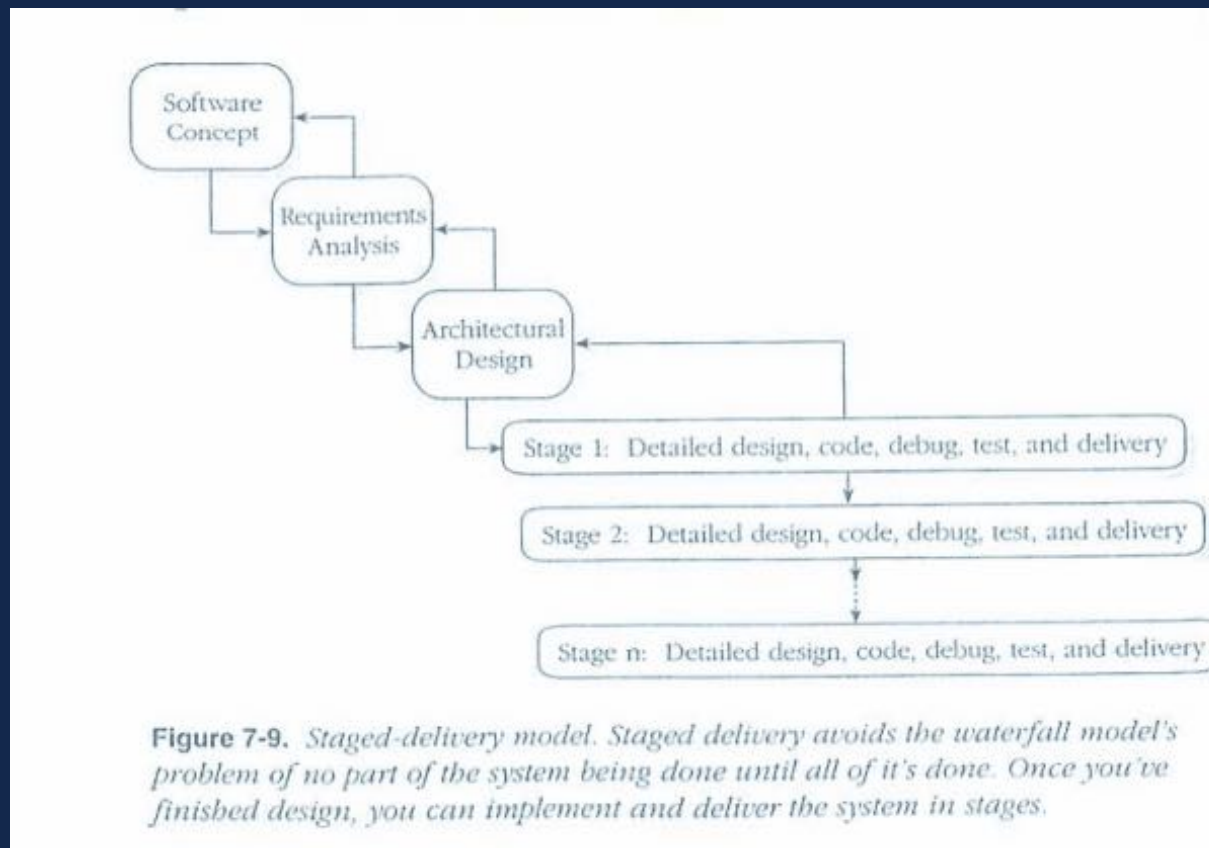


Modified Waterfall #2



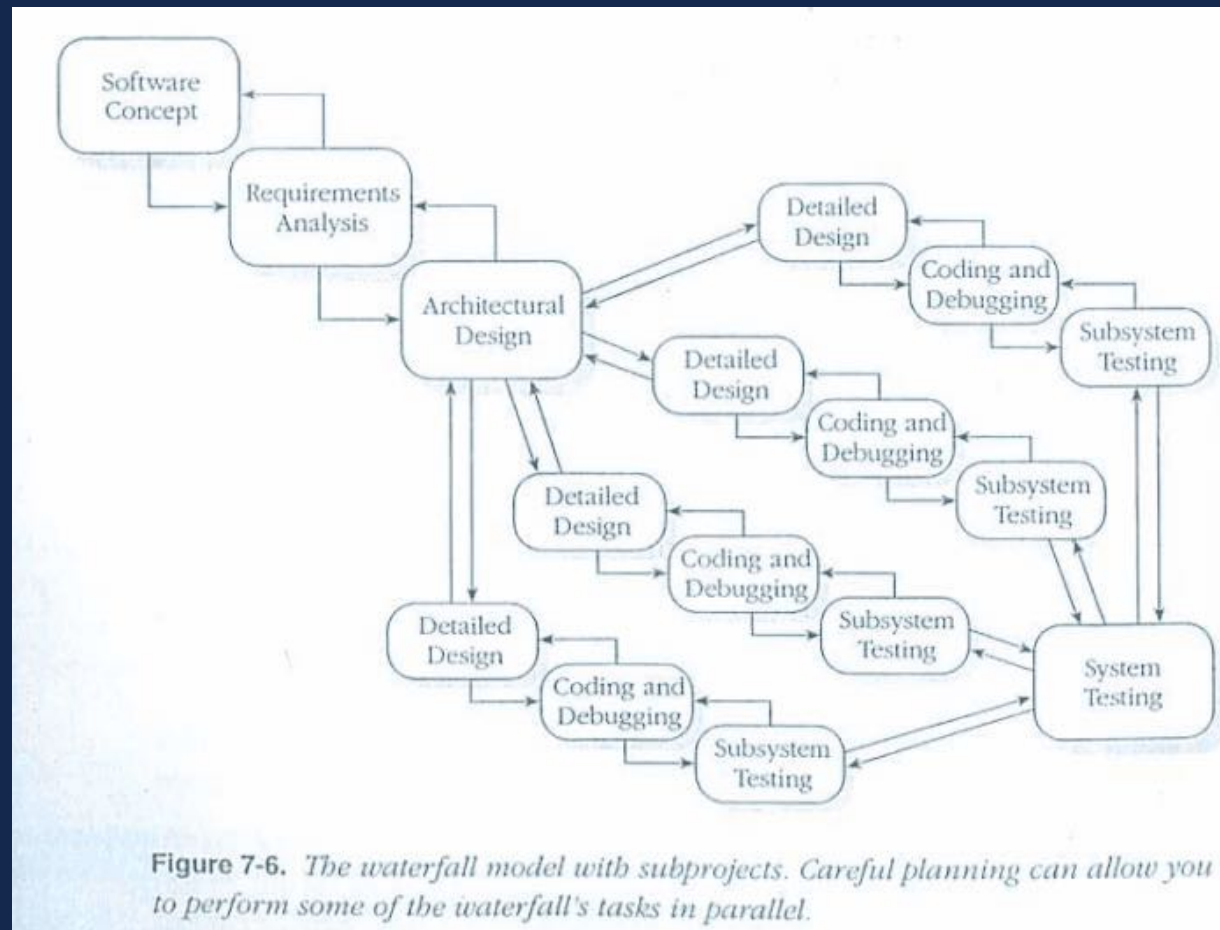
Ch 7, *Rapid Development*, Steve McConnell

Modified Waterfall #3



Ch 7, Rapid Development, Steve McConnell

Modified Waterfall #4



Ch 7, Rapid Development, Steve McConnell

These are Pragmatic Responses

- All these modifications seem like they are hinting about trying to figure out things incrementally before we start.
 - We can adjust our goals, desires, approaches based upon what we find
- Is there a good way to do this in a guided manner?

Towards Incrementalism

- Rough Design Up Front (RDUF) - plan enough to get going (or my EDUF)
 - What is enough? Temptation to start
- Clearly inching along seems safer than big plan on something with too many unknowns or just plain wrong
 - Unfortunately what happens if you inch your way into a corner?

Prototyping

- Evaluate an idea without full commitment
- UX in particular benefits greatly from prototyping
- Tempting thought - take a prototype and refine it to production quality
- Challenges
 - Boss: Pretty pixels on screen = near done

Prototype Tradeoffs

Pros

- Reduced time
- Reduce cost
- Better User Involvement
- Work out details and make pretty

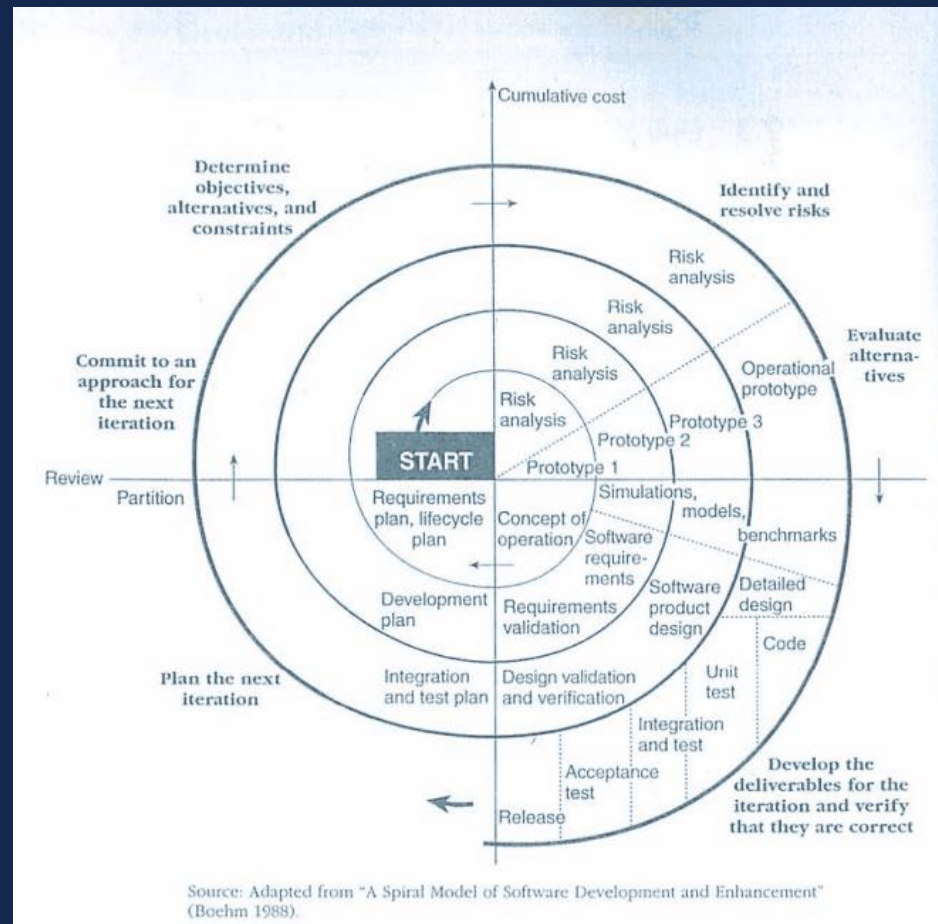
Cons

- User misunderstanding
- “Ugly kids”
- Longer time
- Bigger cost

Extreme Prototyping

- Static prototype
 - Visual mock-ups and/or HTML mocks
 - Feedback and adjust
- Simulated service layer
 - Full HTML/CSS/JS with mock calls
 - Feedback and adjust
- Services implemented
 - Working app
 - Hope feedback changes focus on service layer after up front work

Spiral Model

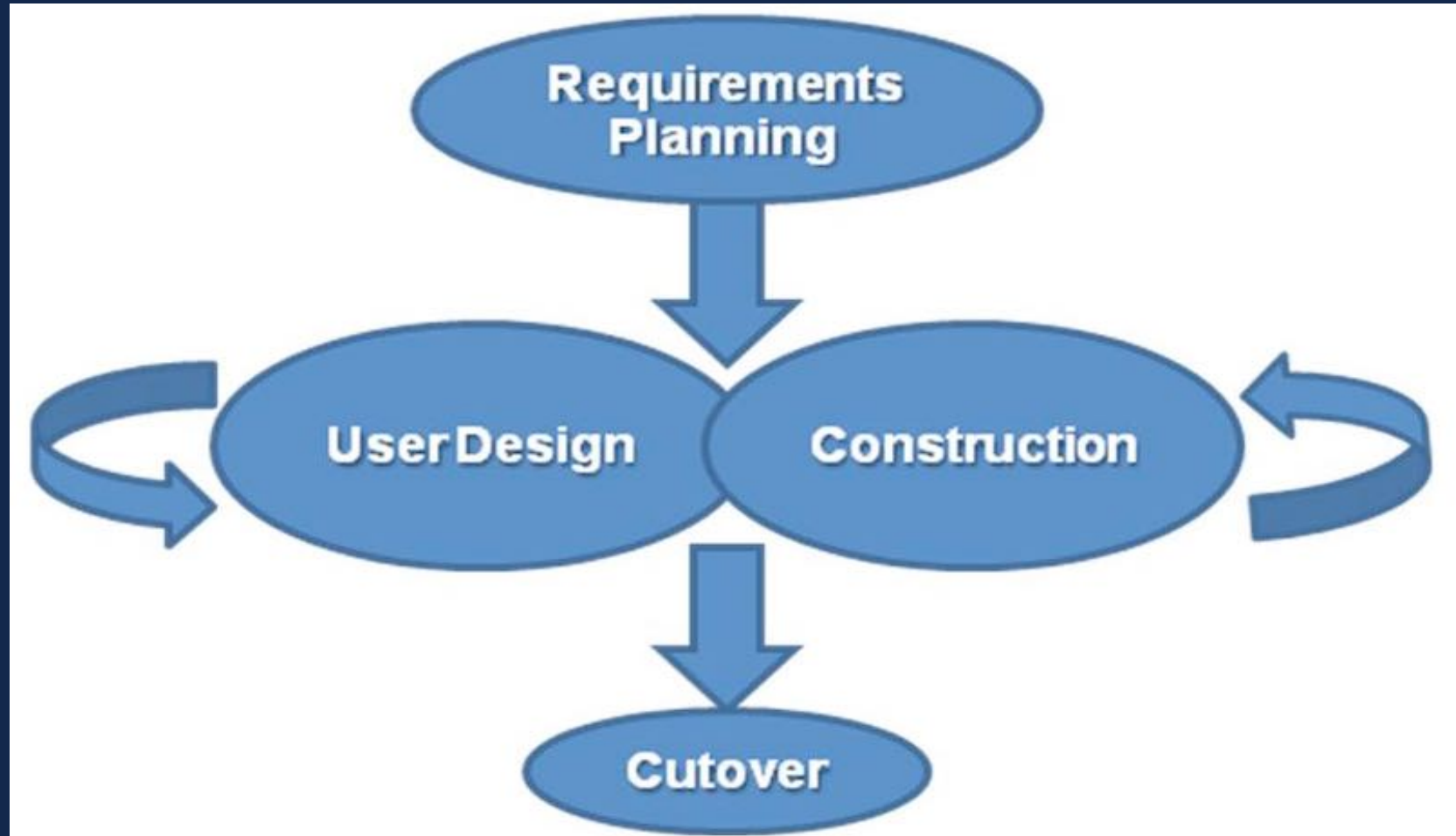


Iterative prototyping spiraling outward from small to big

Rapid Application Development

- Rapid Application Development (RAD) favors rapid prototyping over planning
- Like name aims for speed and tries to address the reality of change via iterative prototyping
- We change this name a lot over time: Agile, Extreme (XP), Lean, etc.

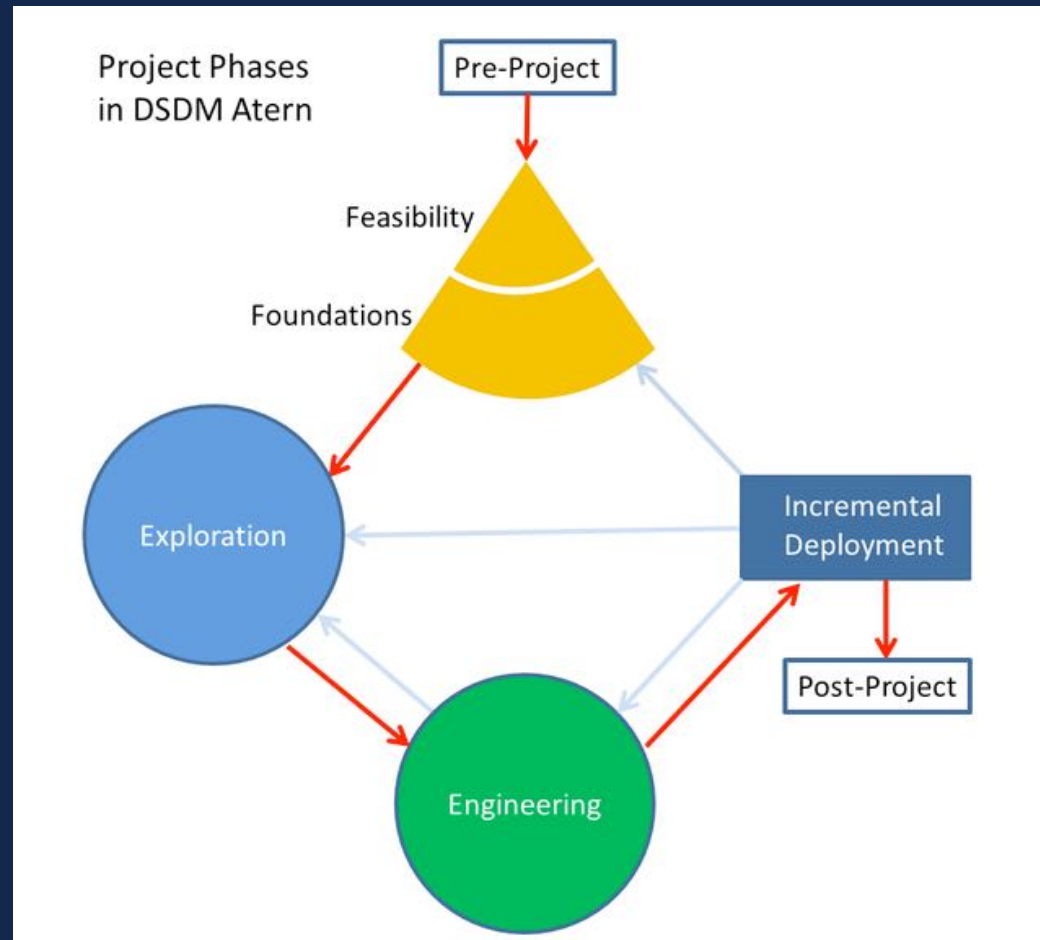
RAD Visualized



Joint Application Design

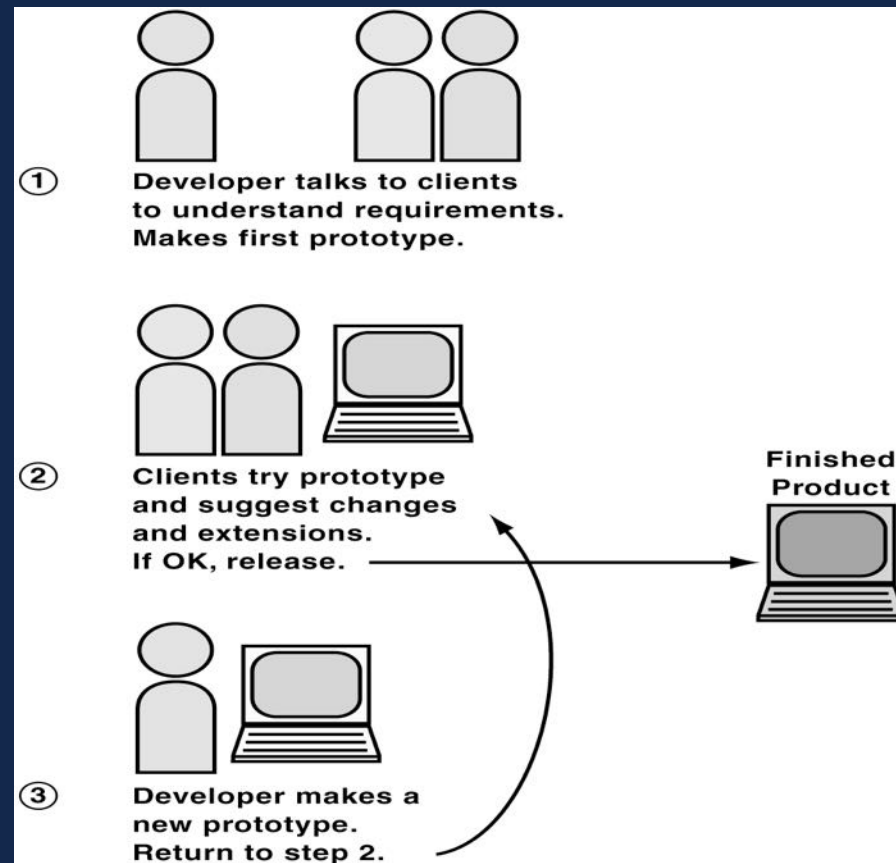
- JAD (Joint Application Design) - method to collect app requirements early on in iterative fashion with end users
 - Came from exploration in DSDM (Dynamic Systems Development Method) but is often applied wholesale to the whole project

DSDM Visualized



Sad JAD

Web Site Engineering Powell et al.



Agile Enters

AGILE



Agile Manifesto

“We are uncovering better ways of development software by doing it and helping others do it. Through this work we have come to value:

- Individuals and interactions over process and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

That is while value in the items on the right, we value the items on the left more.

12 Principles of Agile Manifesto

1. Customer Satisfaction by rapid delivery of software
2. Welcoming changing requirements even late in development
3. Working software is delivered frequently (weeks not months)
4. Working software is the principal measure of progress
5. Sustainable development, able to maintain a constant pace
6. Close, daily cooperation between business people and developers
7. Face-to-face conversation is the best form of communication (co-location)
8. Projects are built around motivated individuals, who should be trusted
9. Continuous attention to technical excellence and good design
10. Simplicity - the art of maximizing the amount of work not done - is essential
11. Self-organizing teams
12. Regular adaptation to changing circumstances

Agile Examples

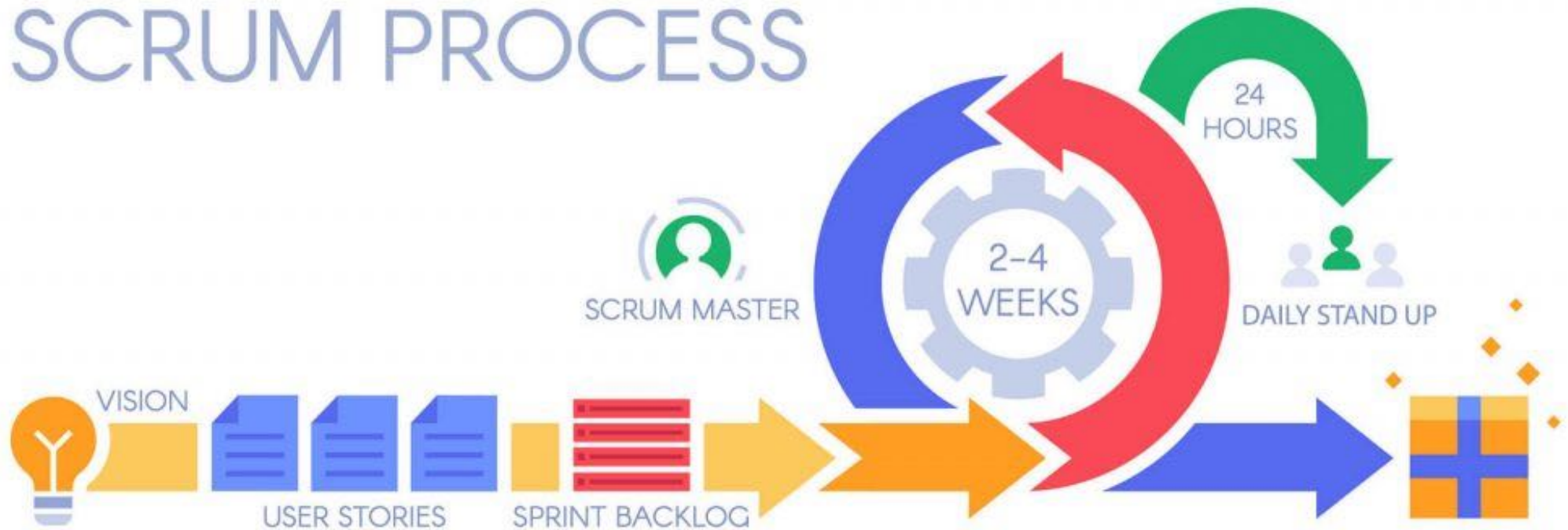
SCRUM!

Theory based on this Rugby move!?



Scrum Visualized

SCRUM PROCESS



SCRUM Values - Feels Familiar ?

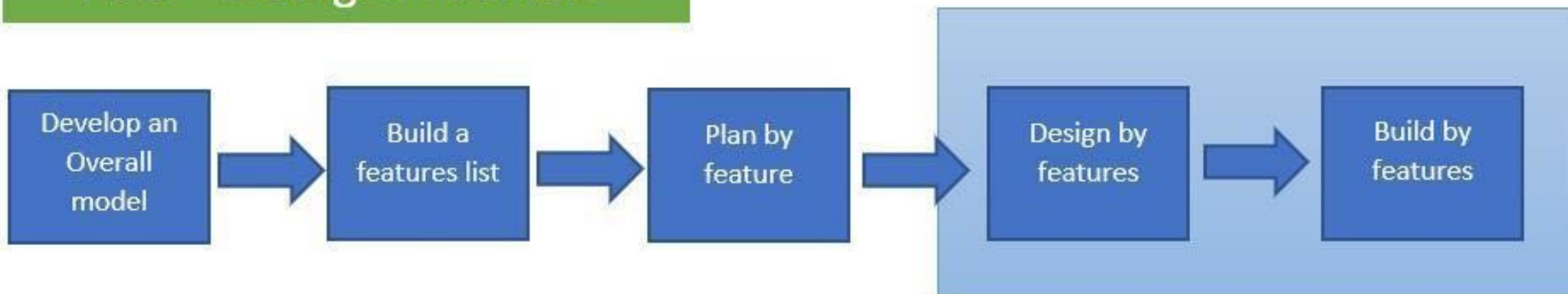
Successful use of Scrum depends on people becoming more proficient in living five values:

Commitment, Focus, Openness, Respect, and Courage

— *The Scrum Guide*, November 2020

Feature Driven Development Visualized

FDD – 5 Stage Processes



Scrum Details

- Scrum team size - 7 (really 7+/-2)
 - Enough diversity for roles not too big
- Three scrum roles:
 1. product owner
 2. scrum master
 3. team member

Product Owner

- Directs team to maximize ROI on project
 - Pick valuable work, avoid less valuable
 - Control development order
- Records requirements in form of “user stories”
 - As a <type of user>, I want to <do something>, so that <some value is created>

Product Owner Role

- Holds product vision
- Represent biz interest
- Represent customer interest
- Balance these (I added this)
- Owns product backlog and prioritizes it
- Answers and owns team questions

Scrum Master

- Kind of the sub-boss (but they claim not)
- Serve as coach, champion, facilitator, road-block remover, etc.
- Primary job is keep team efficient

Scrum Master Role

- Expert in all thing scrum
- Coach (and cheerleader)
- Impediment remover
- Facilitator
- Politician? (I added this)

Team Member

- Teams are self-organizing (people can switch roles, do many, etc.)
 - Team members get to pick work
 - Team members may not be homogenous and have specialties or they may be more interchangeable
- Team members ~~do the work~~ get things done

Team Member Role

- Responsible for completing user stories to incrementally improve product
- Self-organizes to get work done
- Creates and owns estimates
- Owns the how to do the work decision

Product Backlog

- Cumulative ordered list of desired deliverables of the product
 - Features, fixes, docs, etc.
- Should be ordered in priority
- Top items tend to be detailed (ex. Add underlining button) and bottom items are more amorphous (ex. Make drawing routine faster)

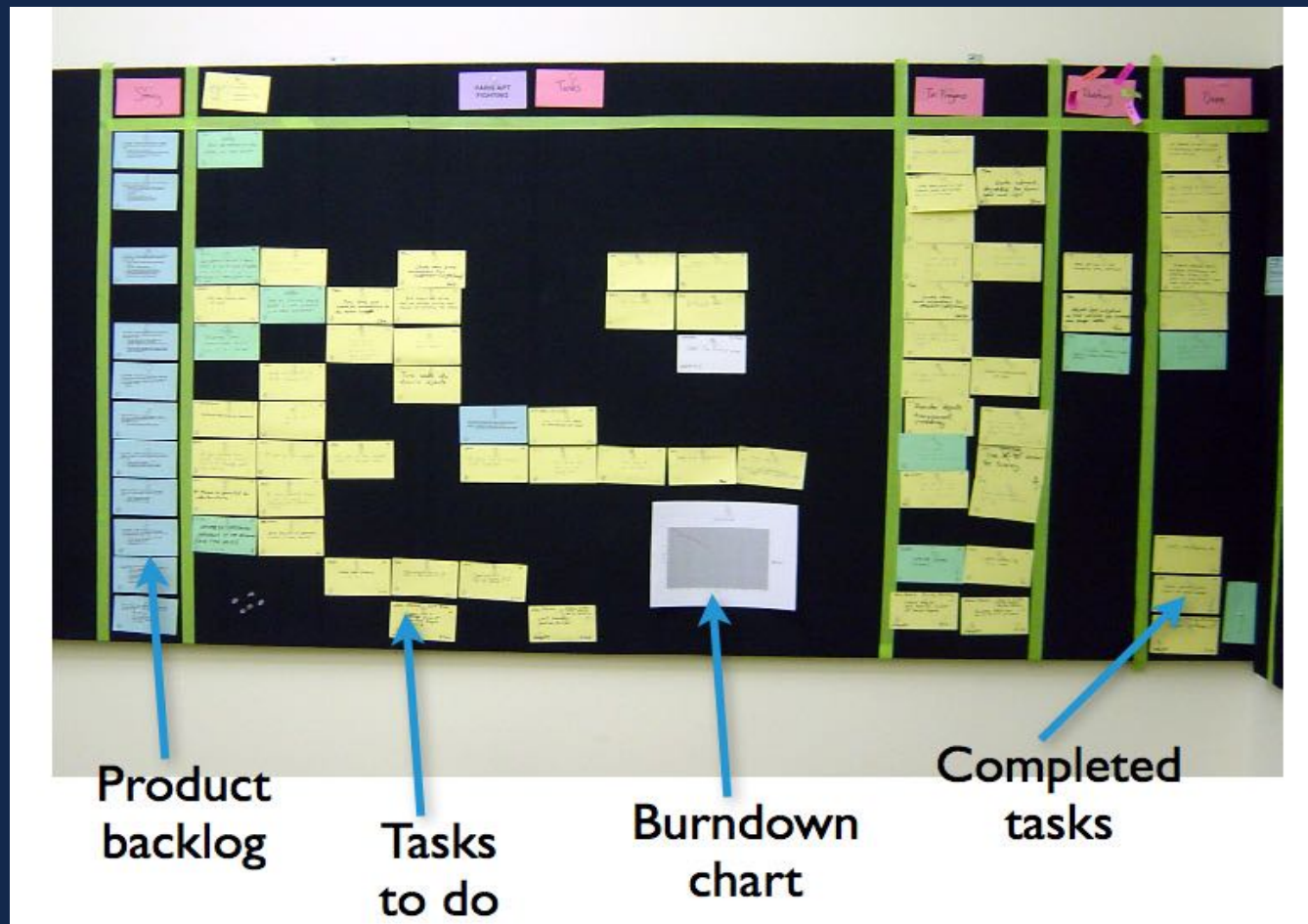
Prioritization of ideas

- There is nothing new here
- We try to formalize the process, but this trade-off effort is part of our famous triangle of features, time, cost
- Our choice of what makes sense is often quite reactive and inconsistent
- Many attempts are made to bring some order to this VERY important decision

MoSCoW Method* Not in Agile or Is It?

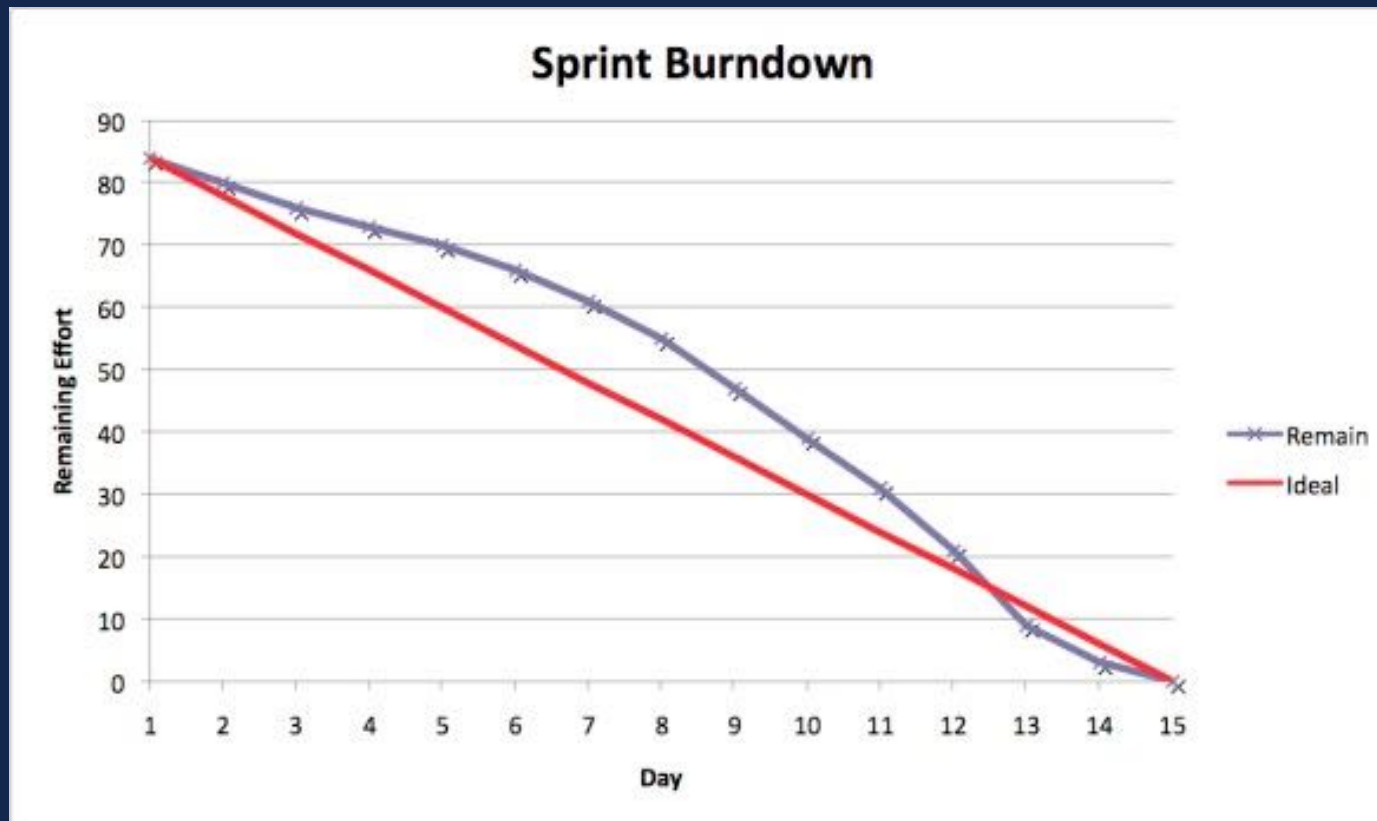
- M - Must: Requirement that must be in final solution
 - MUST - Minimum Usable SubseT
 - S - Should: High priority item that should be in final solution
 - C - Could: Desirable item but not mandatory
 - W - Won't: Item agreed not to be done
-
- The general idea of Moscow is to get you to categorize the things to do formally so you don't try to do everything.

Post-It Time!?



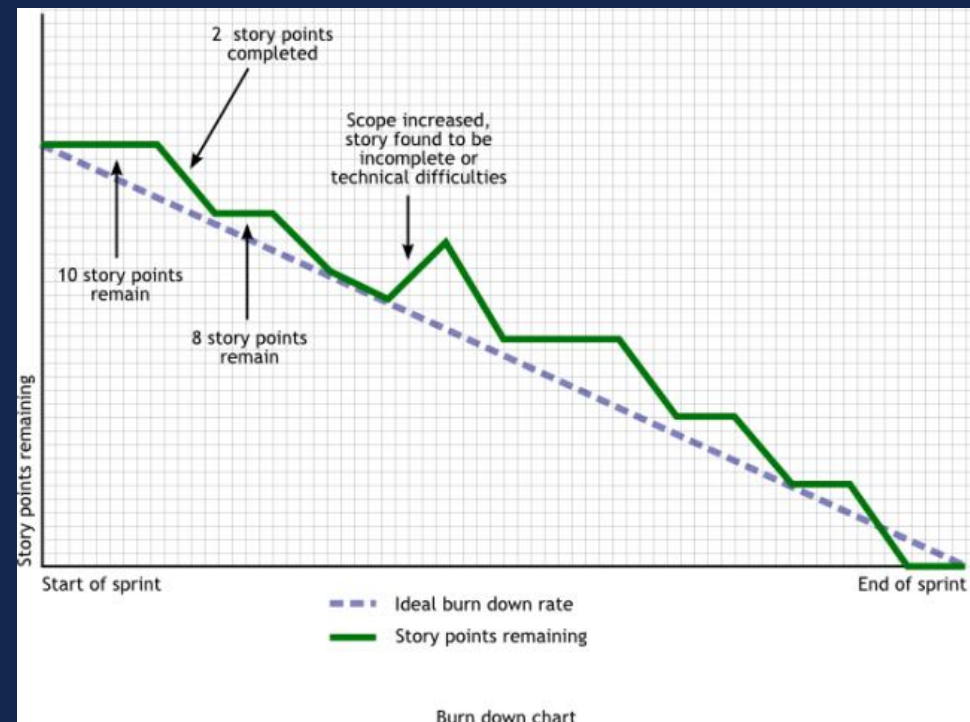
Burn Down Chart

A burn down chart plots work left to do versus time.

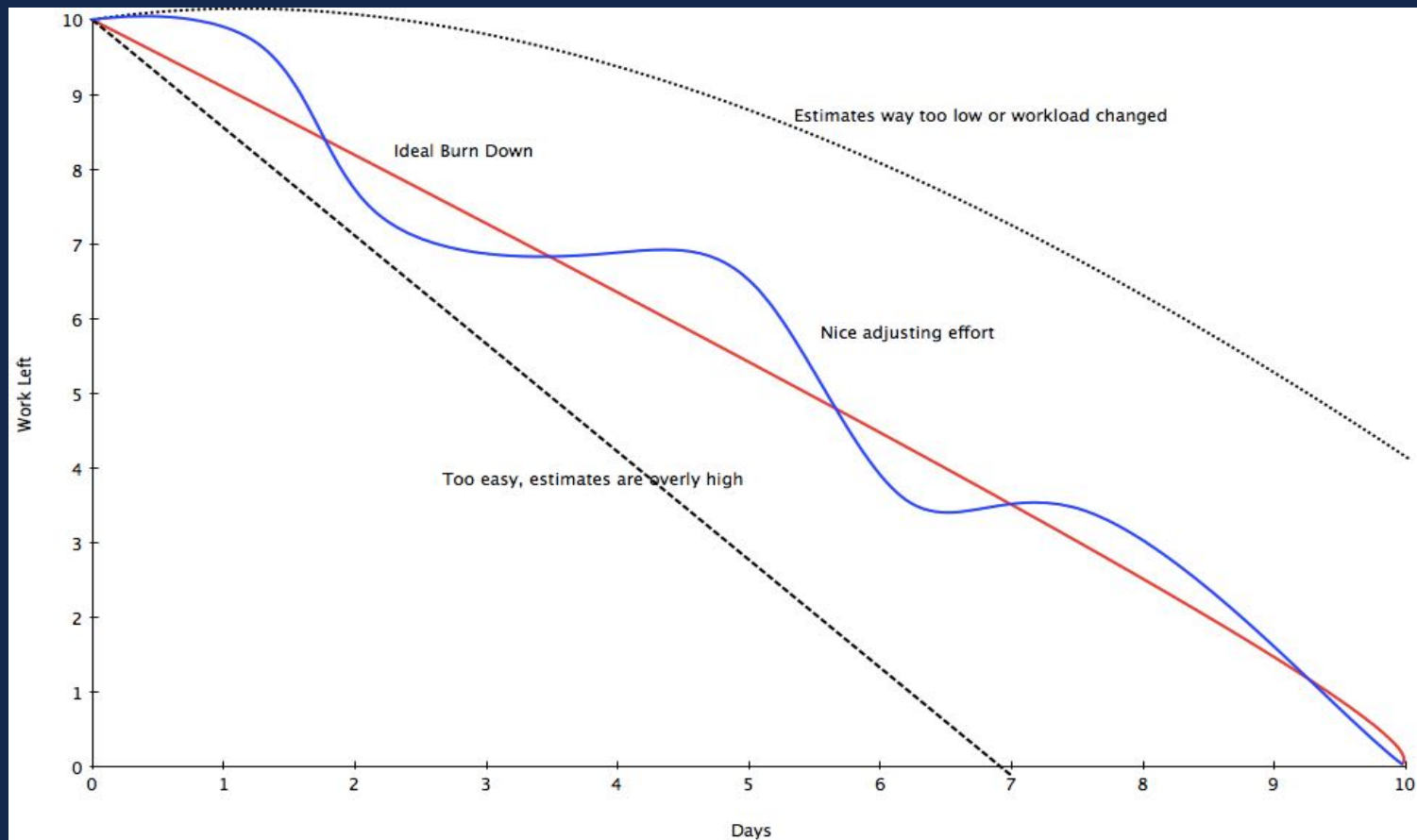


Note - Burn Down Chart

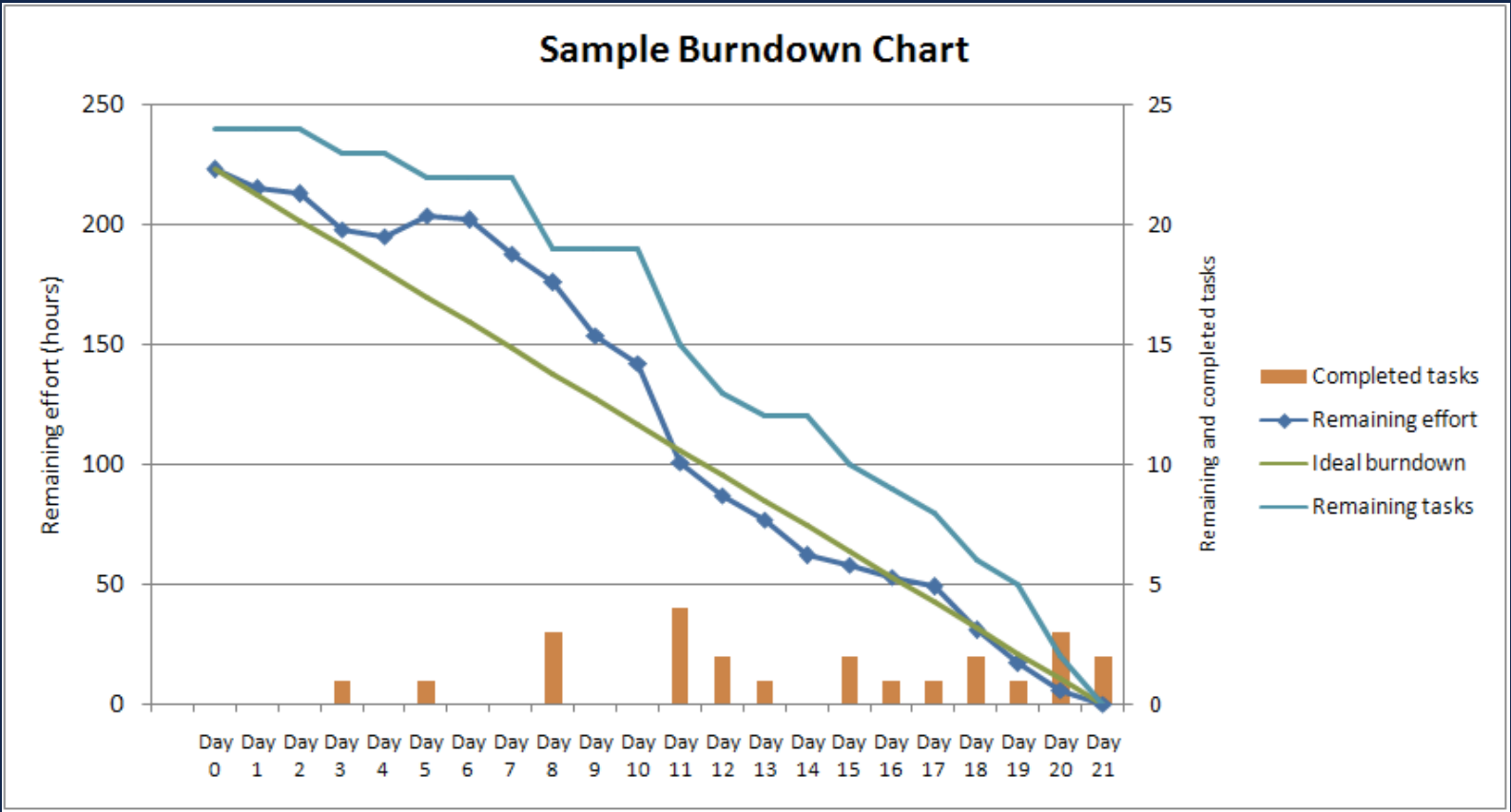
- Q: Are we getting closer?
- Helps keep you going running “1 yard plays”
- Note all the examples show charts like this ... slightly right ward and then magically leftward at the sprint end...hmmmm



Burn Down Ranges



Burning More Complexity In



What’s the point unless you are underserved by your BD chart?



Iteration Design

- Don't make them too long
 - 1 week, 2 week, 3 week, one big one at 6 weeks?
 - Short length helps you deal with change so you can switch focus if something comes up the next iteration
 - Progress of completing an iteration keeps us motivated
- Keep the iteration content in balance
 - Not just bugs, but features too for example
 - Variety keeps the interest up

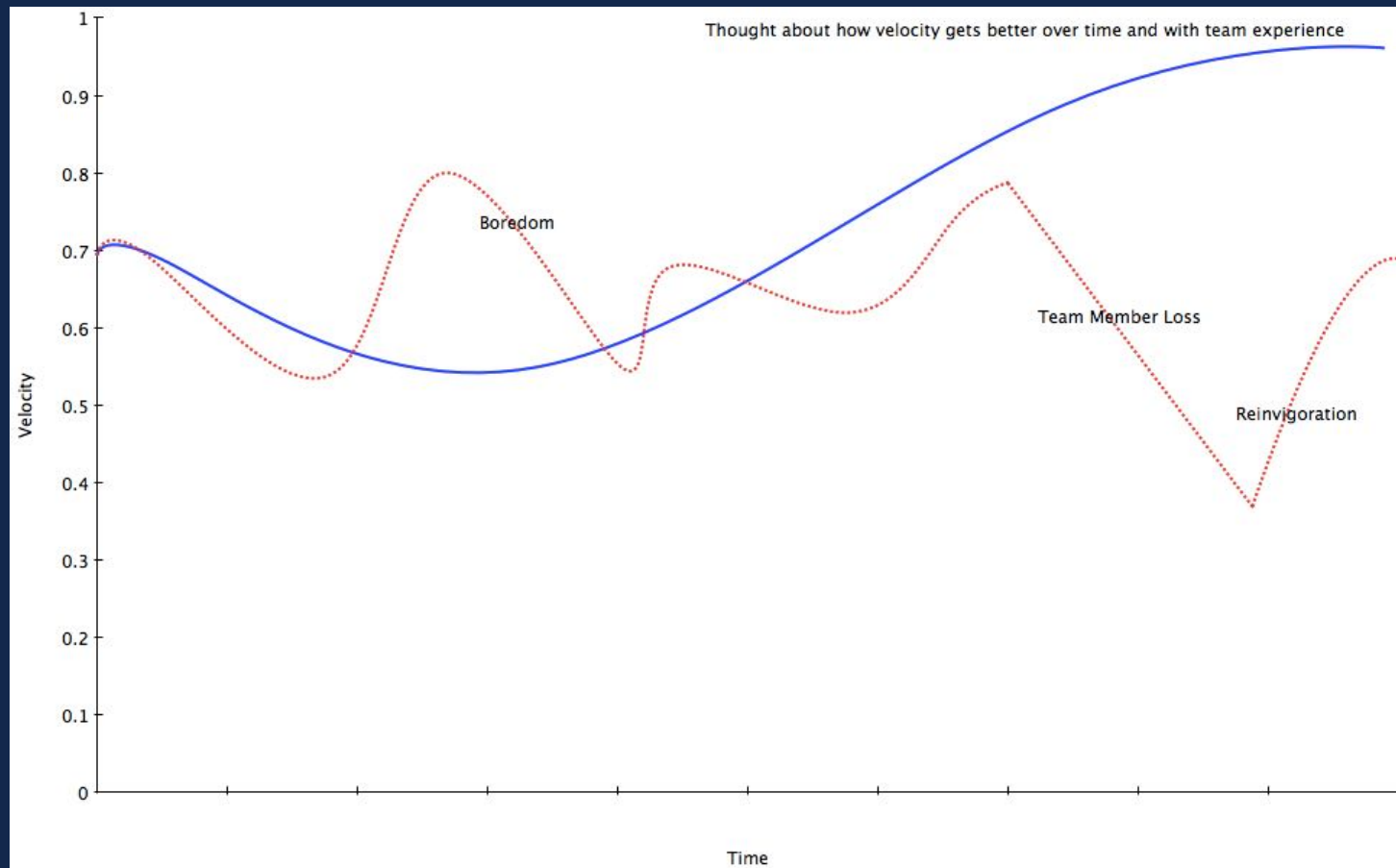
The Need to Account for Velocity

- Given X days, some value $X-O$ is how much is spent on work
 - O for overhead maybe? Whatever the point there is some sense that you can't have perfect days
- Maximum productivity as we know is a myth
- Novice Programmers schedule utopian days
- Experienced Programmers schedule in real world days

Velocity

- Some factor that indicates what % of time is working during a sprint
- Commonly 0.7 is used as a starting velocity value and we adjust it as we move along to account for reality
- Example:
 - January -> 31 days -> 20 workdays, actually 19 days (MLK)
 - Use 0.7 velocity and you get 14 workdays to schedule!
 - The extra 5 days we can consider slack

Velocity Changes Over Time



Accepting Reality

- **Your Ultimate Goal:** Always tell customers and other stakeholders what you know actually can be done as opposed to just what they want to hear
- **Rule:** Balance what customer wants $\approx$ what can be done
 - Degree of equality is related to customer reasonableness and your ability to be brutally honest with the customer
- If rule is out of balance then either the customer is going to be unhappy, the dev unhappy/over worked or both



SCOTTY SAYS ENGINEERS

**SHOULD UNDERPROMISE
AND ALWAYS DELIVER**

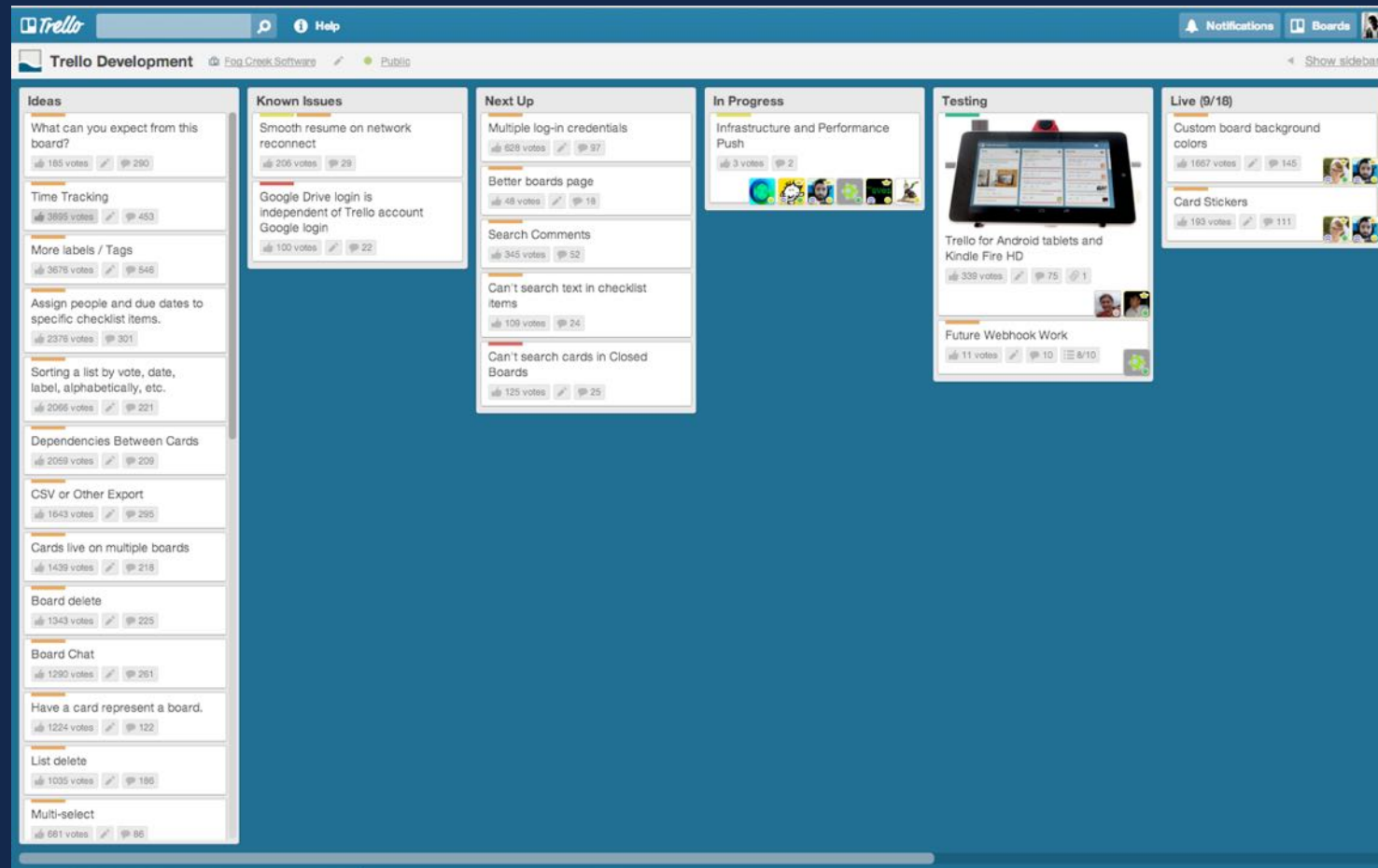
User Stories and Tasks

- Users stories != Tasks typically
- A User Story might be made of 1 to n tasks
 - Careful though how many tasks before you need to split your story?
- Each task should be a piece of work for a single developer and come with an estimate
- Tasks should be no smaller than .5 (4hrs) and 5 days (20hrs) assuming no velocity built in

User Story & Tasks Example

- User Story – Estimate 9
- Story is composed of 4 tasks
 - Task 1 – Estimate 2
 - Task 2 – Estimate 5
 - Task 3 – Estimate 3
 - Task 4 – Estimate 1
- Task estimate = 11 $\neq$ 9 the original story estimate
- Now we should go back and adjust our user story to match the cumulative task estimates

Tooling Example



<http://www.trello.com>

Tooling Example

The screenshot displays the Pivotal Tracker interface for the 'ZurmoCRM (Public)' project. The top navigation bar includes 'SIGN UP' and 'SIGN IN' links, and a velocity indicator set to 8. The main content area is divided into two panels: 'CURRENT' and 'BACKLOG'.

CURRENT View: This panel shows a list of 105 items for the current sprint, ending on September 30. It includes a 'Release 2.2.5' section with a 'Finish' button. Items are color-coded (green for 'In Progress', yellow for 'To Do', and red for 'Blocked') and have 'Accept' and 'Reject' buttons. Examples of items include:

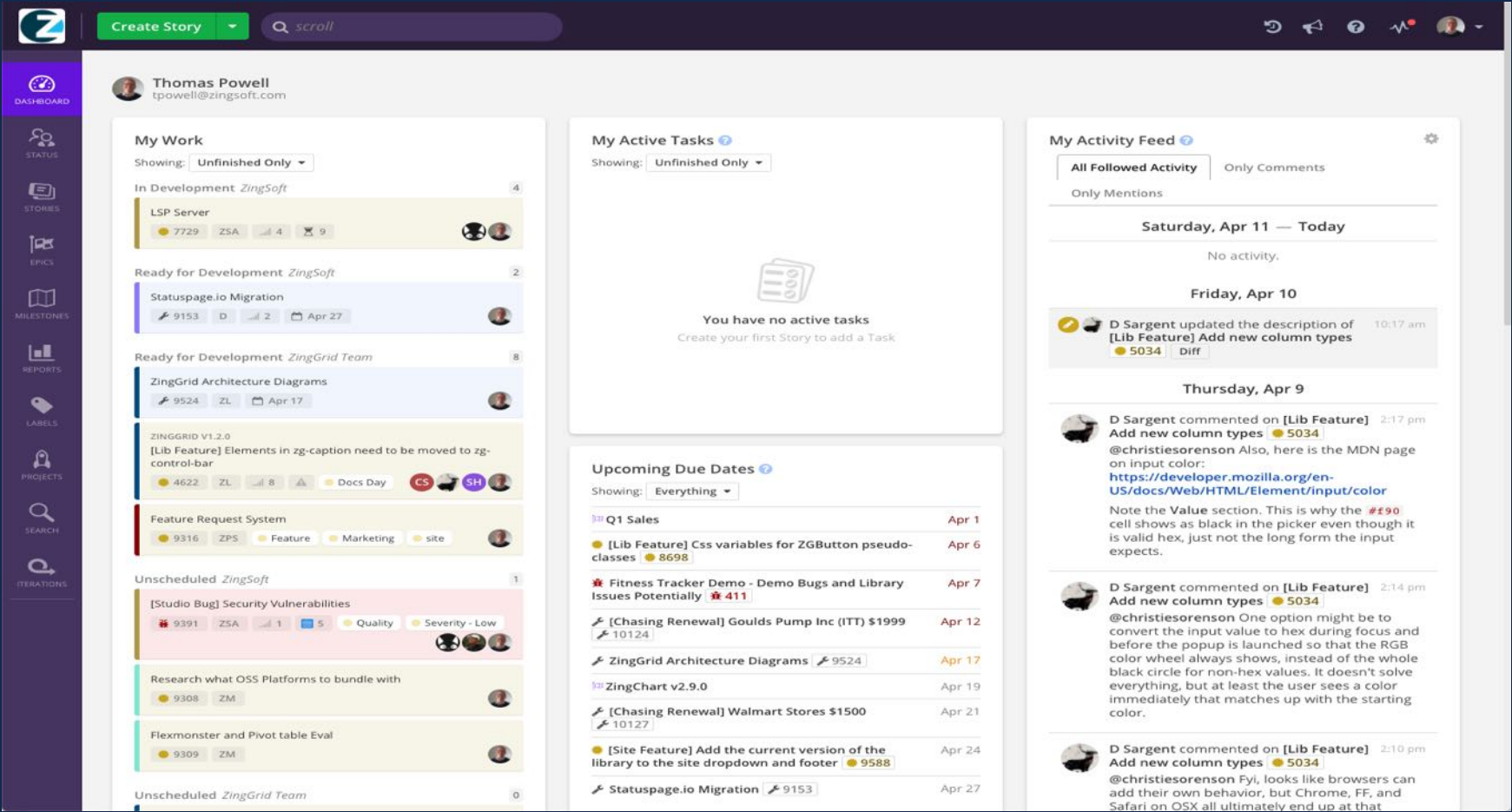
- 2.2.5 Bug with matrix reports not exporting all data (SP)
- 2.5, marketing Review messages (LA)
- 2.2.5 Workflow bug - Email template issue with attachments (SP)
- 2.2.5 Bug with default value of picklist fields in import mapping screen clearing (SP)
- 2.2.5, ramp-up change @zurmoalerts.com to be configurable in the global settings (JG)
- 2.5, branch-workflow Functional tests for mass delete (SM)
- 2.5, marketing engine, ramp-up The configuration modal window for campaign and marketing charts should show Start Date and End Date instead of Begin Date and End Date (BP)
- 2.5, branch-userinterface, workflow engine Workflow improvement - subscribe someone to a marketing list (JG)
- 2.5 create 6 more themes and backgrounds (AA)
- 2.5, branch-importrefactor Import complete - a row in last step part2 (JG)
- 2.5, bid, web forms Web Form Builder enhancement (NabilM)
- 2.5, ramp-up Add administrative option in opportunities module in designer to turn off probability mapping (BP)
- [Invalid, Performance Issues.] ReadPermissionsSubscriptionUtil should use RedBeanModel (MS)
- 2.5, branch-htmldemodatarefactor build richer full html demo data for email templates, Autoresponders and Campaigns and also better text template demo data (MS)
- 2.5, branch-readrefactorforredbeanmodel [Invalid, Performance Issues.] ReadPermissionsOptimizationUtil should use RedBeanModel (MS)
- 2.5, branch-htmldemodatarefactor HTML content unit tests (MS)
- 2.5.0, tasks v2 Testing for the notifications for task changes in case users email box is not configured. (IN)
- 2.5 Styling web forms selected attributes in edit view (AA)
- 2.5, branch-autobuildv2, branch-importrefactor Refactor import for

BACKLOG View: This panel shows a list of 106 items for the backlog, starting on October 7. Items are color-coded and have 'Start' and 'Stop' buttons. Examples of items include:

- 2.5 Fix Campaign icon sizing in list detailview (AA)
- 2.5 Bug with Merge tag in Tasks, Meetings, Notes (MS)
- 2.5 Possible bug with Catalog Items (SP)
- 2.5, marketing engine Pages with htmlContent should show help regarding what tags are not allowed and other related stuff
- 2.5-upgrade-script, branch-autobuildv2, branch-bleeding Generate Schema diff of default install and share it with ivica (MS)
- 2.5, branch-gamification User interface and logic for redeeming coins against a game reward (JG)
- 2.5 need google analytics module icon (AA)
- 2.5, marketing Email Details View and Email Recipients in Campaigns Styling (AA)
- 2.5.0, tasks v2 General list view for tasks (Mayank)
- 2.5.0, tasks v2 Integration with new task modal design for account/contact/opportunities/leads (Mayank)
- 2.5.0, tasks v2 Integration of new kanban view design for account/opportunity/lead/contact details page (Mayank)
- 2.5, branch-workflow style/css fixes on workflow in message queue search improvement (AA)
- 2.5 styling for group and roles intro page (AA)
- 2.5.0, tasks v2 Handle designer specific logic for task v2 (Mayank)
- 2.5 Refactor emailMessages to people/accounts to be MANY_MANY (SP)

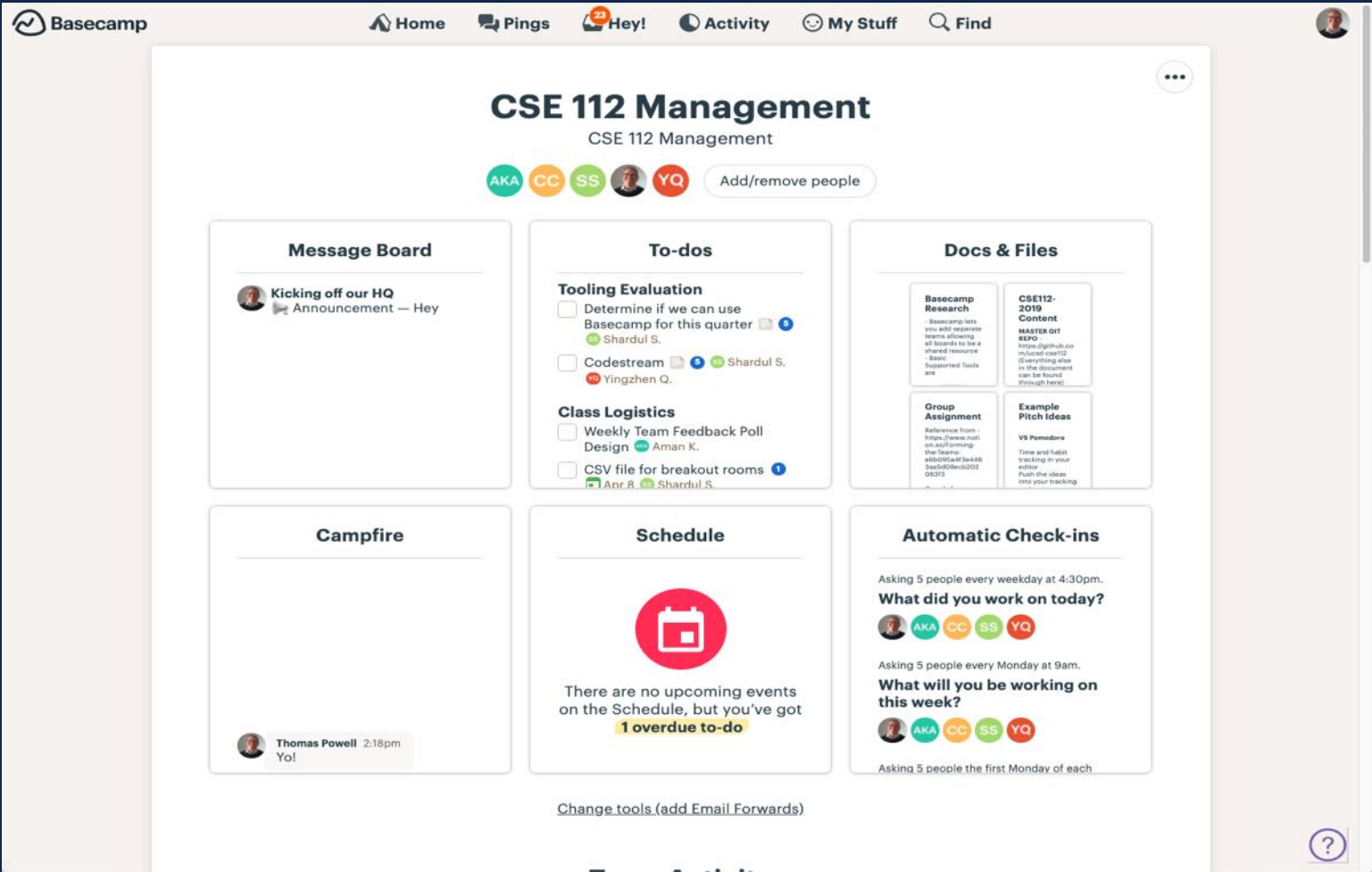
<http://www.pivotaltracker.com>

Tooling Example



<https://clubhouse.io/>

Tooling Example



Note the Tool Range

- Your tool range here seemed to range from simple to complex from visual to textual
- There is no single tool that is right
- Much of the tool choice has to do with preference, ease, cost, and often signaling
 - “Only real _____s use _____” is sadly a thing

Some Pot Shots

- Story points are like fake currency in games to me
 - I just spent X doughnuts
 - We just used X story points
- Intention is good
 - But it is somewhat window dressing on a cut and chew approach
- There have been a number of times I want to employ agile mechanisms but the nature of what needs to be done doesn't lend itself well even though the project itself is managed this way

Some Pot Shots

- Very often the tasks don't decompose well; they are large-ish but don't benefit from being broken into mini tasks.
- **Example:** We are porting ~96 classes to a messed up way of OOP to a standard ES6 way.
 - That is clearly not a single sprint effort but breaking the story cards into a single class at a time is nuts and just chunking it by sprint size is arbitrary.
 - The person(s) doing this every report for ~2-5 weeks annoyingly get to say “working on the class refactor”

Some Pot Shots

- **Conclusion:** Some sort of religious adherence is not useful, you'll be made to force things and it limits how you might solve the problem.
- **Remember:** At the conclusion/ship time people don't buy your process they buy your result!
 - The process has to help us get there and when it doesn't we should feel free to modify it or even dump parts of it

Some Other Thoughts

- Sprint length varies - 1 week, 2 weeks, 1 month
 - Too frequent overhead gets bad
 - Too infrequent - you're doing Waterfall even if you don't know it
 - Should it be static? Makes it simple but sometimes you might be making things arbitrary or awkward
- Product Owner is a key role, have a bad one and you're doomed.
- Q: What does a good product owner look like? Waterfall wise are they different?

Some Quotes

Quote #1

“The hardest single part of building a software system is deciding precisely what to build. No other part of the conceptual work is as difficult as establishing the detailed technical requirements, including all the interfaces to **people, to machines, and to other software systems**. No other part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify later.”

Quote #2

“...the most important function that the software builder performs for the client is the iterative extraction and refinement of the product requirements. For the truth is, the client does not know what he wants. The client usually does not know what questions must be answered, and he has almost never thought of the problem in the detail necessary for specification.”

Quote #3

“Much of present-day software-acquisition procedure rests upon the assumption that one can specify a satisfactory system in advance, get bids for its construction, have it built, and install it. I think this assumption is fundamentally wrong, and that many software-acquisition problems spring from that fallacy. Hence, they cannot be fixed without fundamental revision--revision that provides for iterative development and specification of prototypes and products.”

Quote #4

“Grow, don’t build, software.”

Nope, not modern dev, all said by

Fred Brooks, age 83

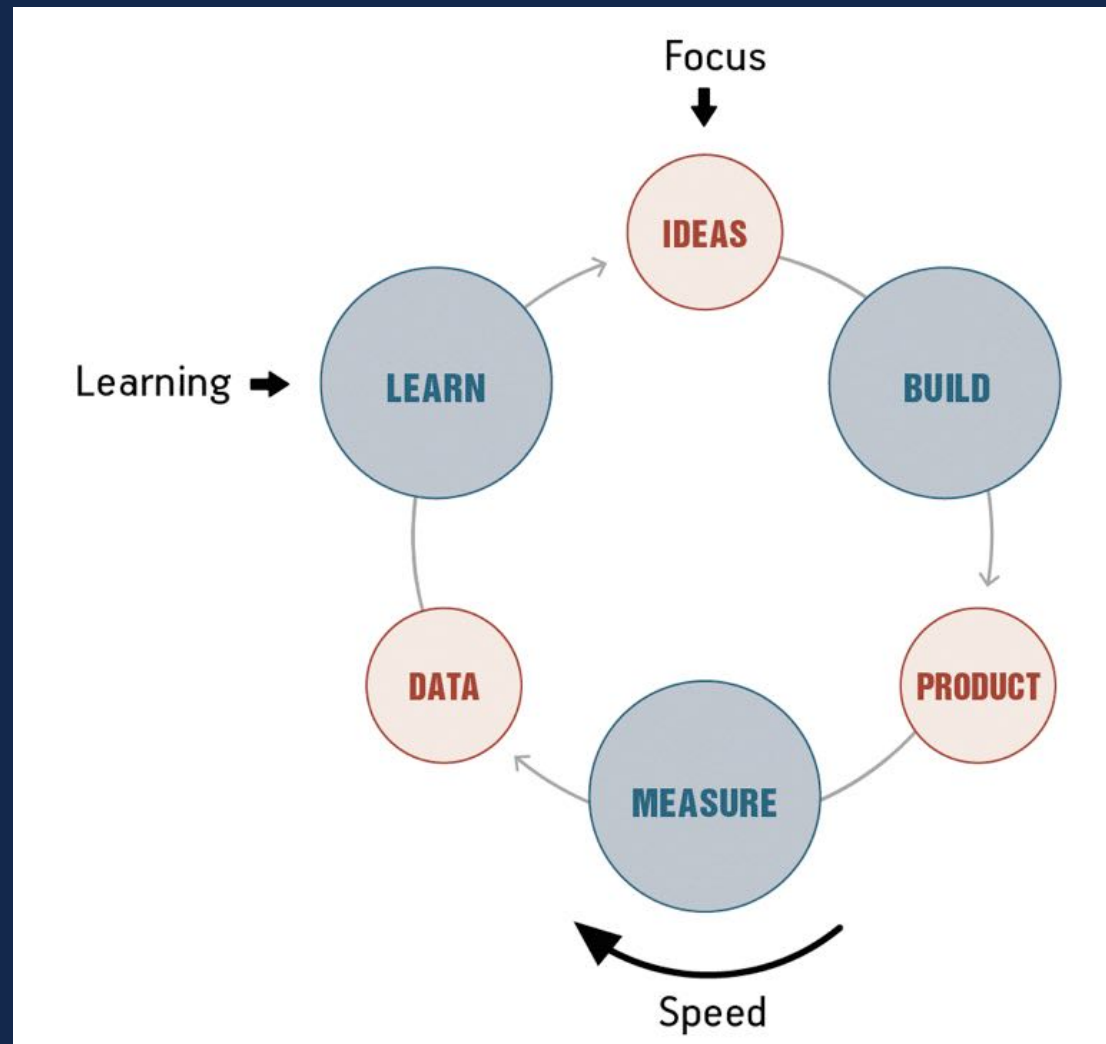
“Mr. Mythical Man Month,
Waterfall, etc.”

Seems like we keep
visiting the same ideas
with new names

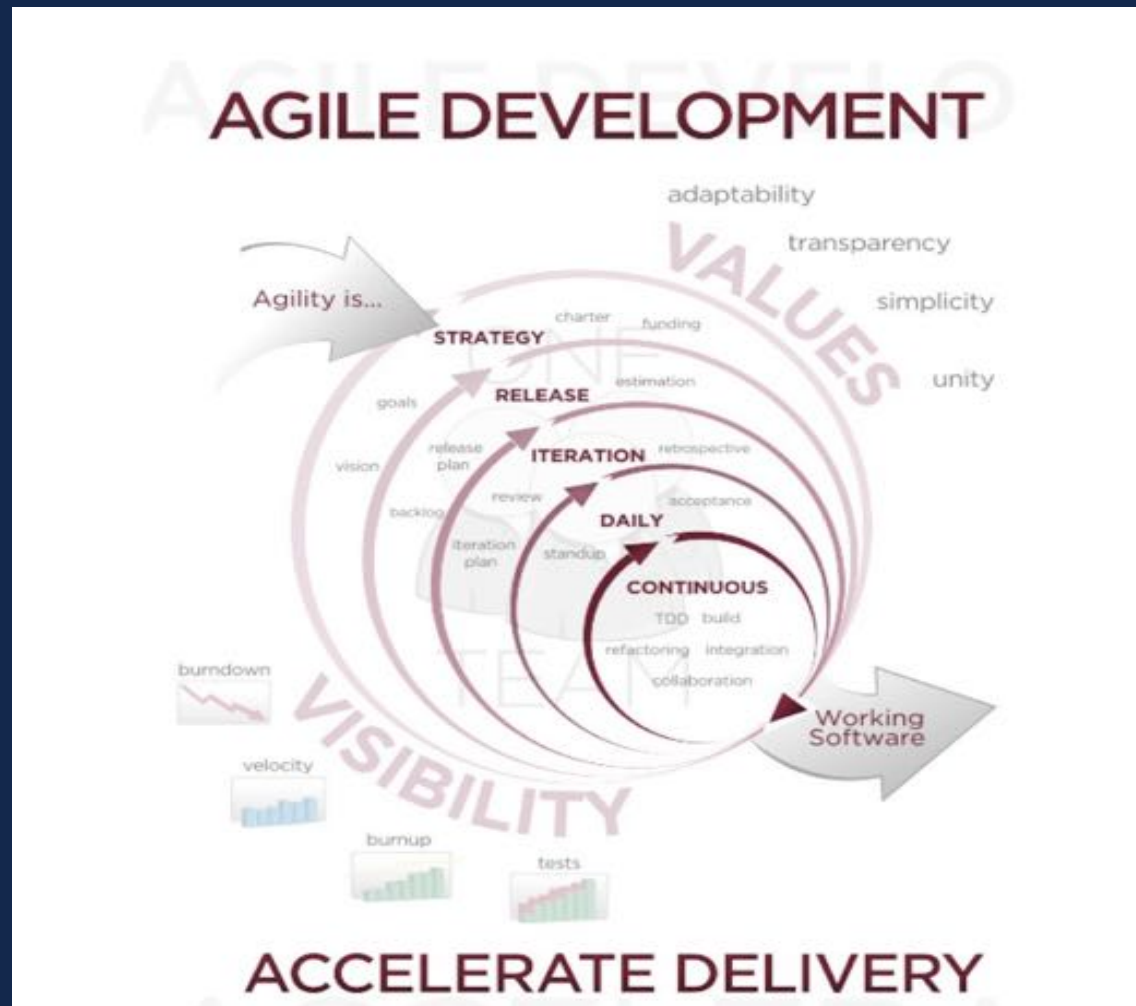


Keep Changing Model,
Keep Looking More Familiar

Example: Lean Loop from Manufacturing



Agile Visualized Seems Lean-y



Continuous Deployment is the Goal!

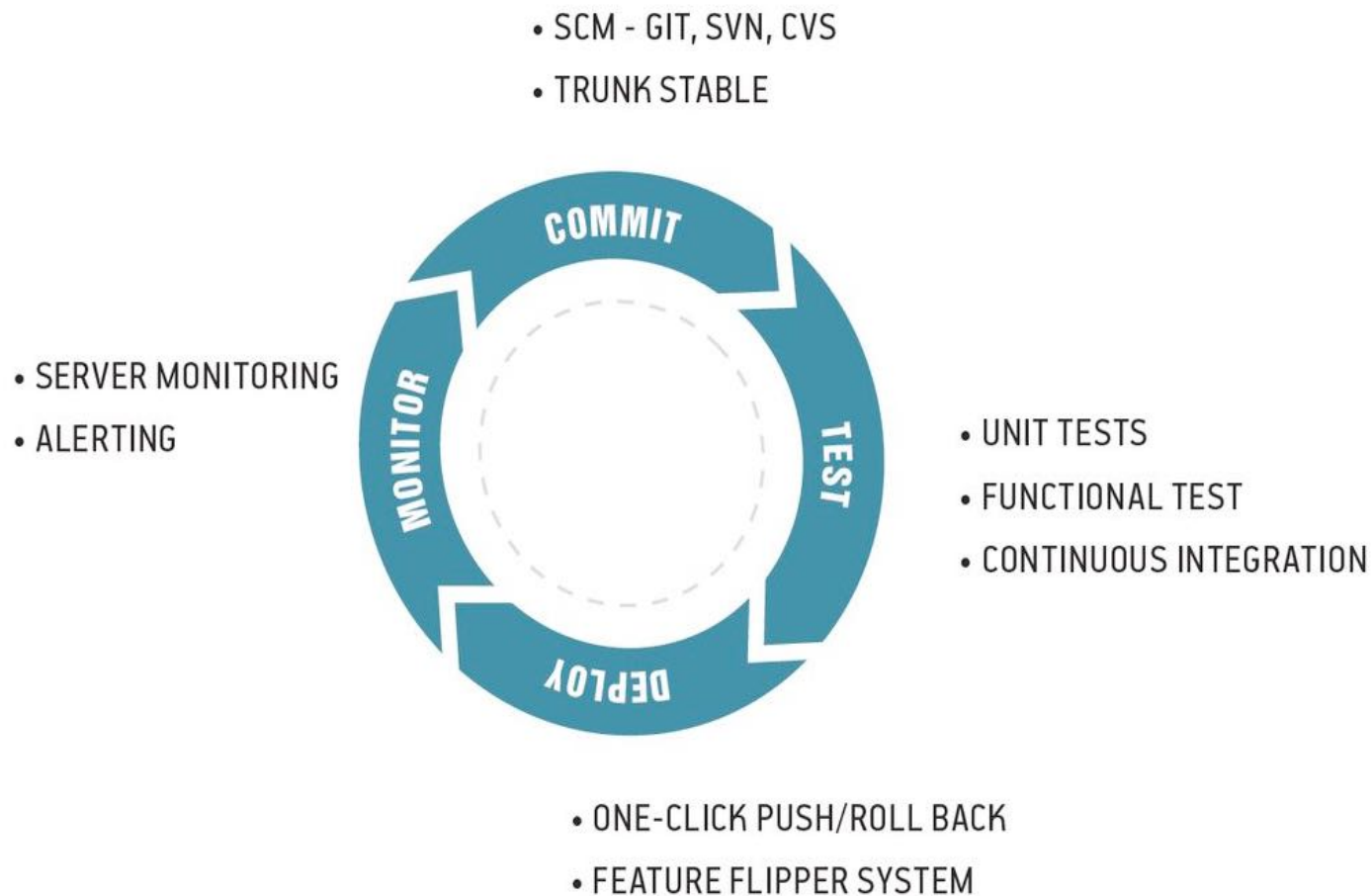
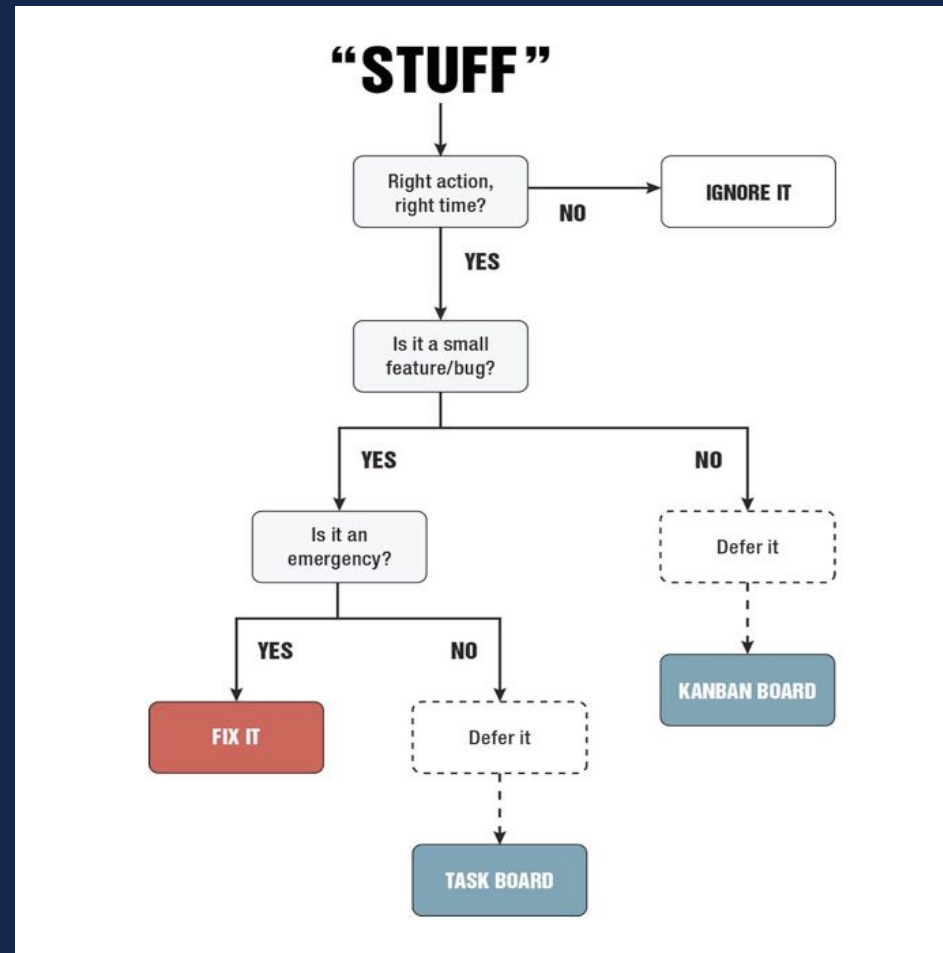


Figure A-1. Continuous deployment cycle

Pull don't push

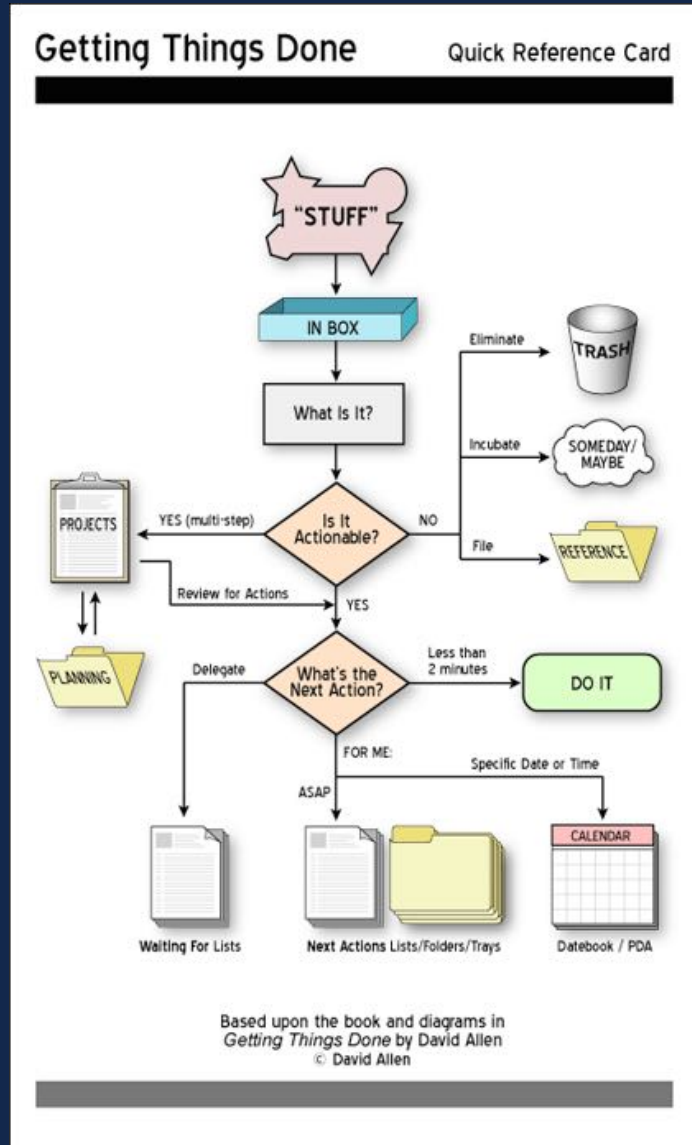
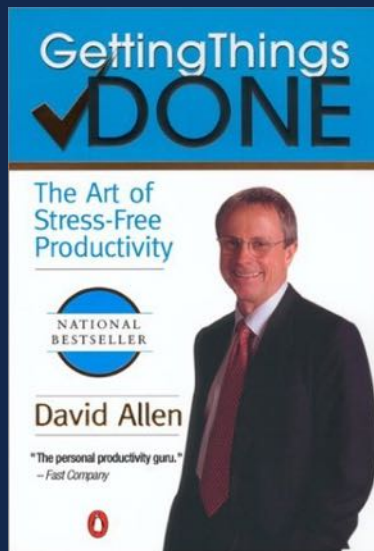
- So how do we know what to do and when?
- Good question!
- Thoughts?
 - Not my problem!!!!
 - Ask Users?
 - You Design?

Task Prioritization



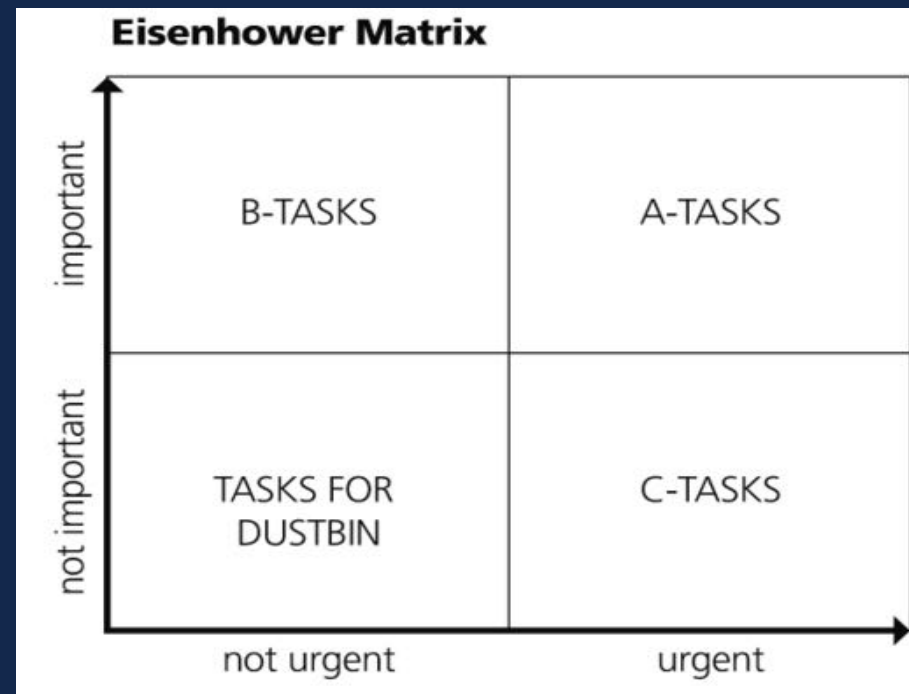
Just GTD for Programming?

- This seems to remind me of productivity schemes like Getting Things Done methodology
- Ok ok ok...we get it now!



Ike did this before them?

It seems there really isn't much new under the sun in terms of process management, but then how come we keep pushing here?

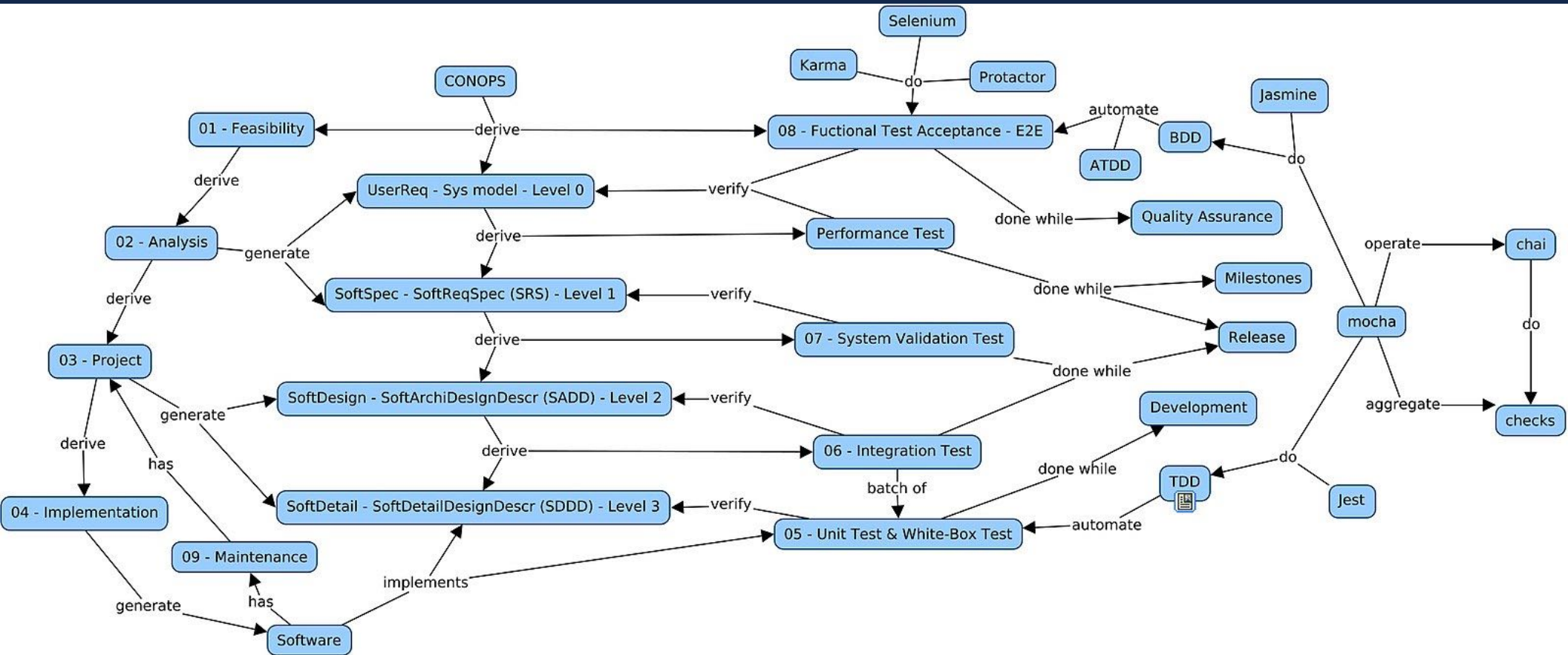


Iterative Truths

- While JAD, RAD, Lean, Agile... can deal with change and unclarity of thought it may...
 - Build wrong thing many times?
 - Suffer from “marry” prototype problem
 - Damage us with early releases
 - Be difficult to cost control or budget
 - Suffer from difficulty of org buy-in unless they deliver
 - Have its own arbitrary adherence problems like any other process model (see my example earlier)

Maturity of Models

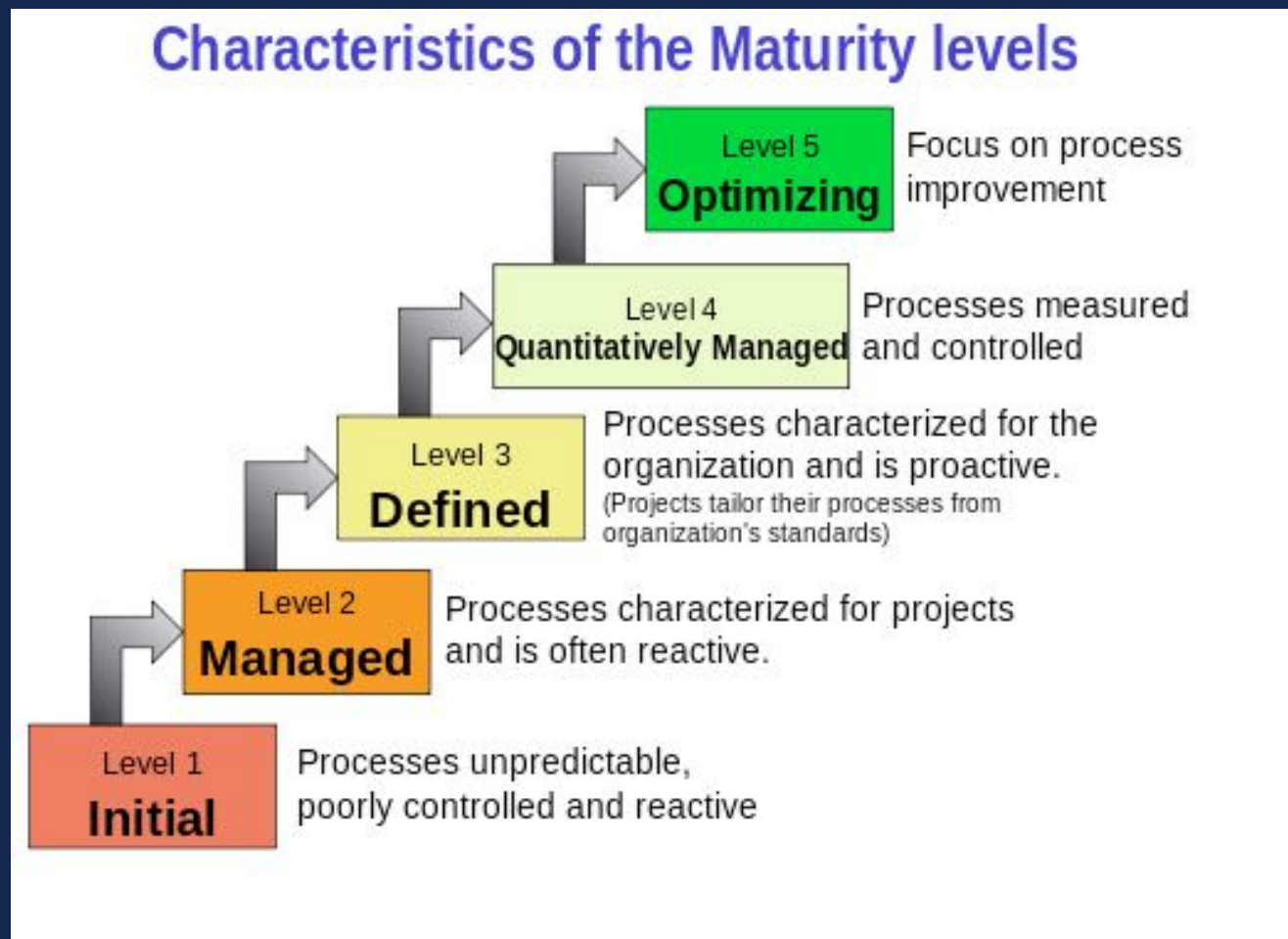
Over Processed? It Depends?



Process Maturity

- CMMI - Capability Maturity Model Integration
 - Required by many (ex. DoD, US Govt.)
- Basically talks about having formalism and repeatability to your SE processes with eventual focus on improvement
- Given the nature of where the industry is and the type of projects being run it is hard to tell how important this is (very or not very)
- I actually think it is very (eventually) important, but understand that the culture and situation drives the model and adoption not the other way around

CMMI Visualized



Over-Engineered

- Watch out for Architectural Astronauts
 - Afflicted with Analysis Paralysis
- Failure (or fear) to launch
- Just one more thing...
- “Spruce Goose” projects

Under-Engineered

- Quick and dirty
- Cowboy coding, magical thinking, no compromises
- Sometimes it makes sense for exploration
- Both these points are the extremist position to too much process or too little

Product over process

- Does process really matter how you did it if result is ok?
- To us or boss
 - Yes - if not repeatable
 - Yes - if it costs too much
 - Yes - if it was a miserable experience
- To customer
 - Who cares about sausage unless pricey or unavailable!

Worse is better?

- Counterintuitively sometimes this is true
 - Less functionality can be better quality wise
 - More functionality can be worse system wise, but may be better market wise?
- Consider the effect of feature/UX from market forces
- Human's employing simple heuristics to navigate the market will often introduce things which may sabotage the engineering purity
 - Ex: Having to jam in features for market acceptance only to see them unused

Always Be Pragmatic

- Principle of Good Enough
- 80/20 Rule - Pareto distribution, 80% of value from 20%
“Law of vital few”
- KISS - Keep it simple, stupid
- MVP - Minimal Viable Product
- More to be created

Remember Picks Don't Have to be Permanent

Complete

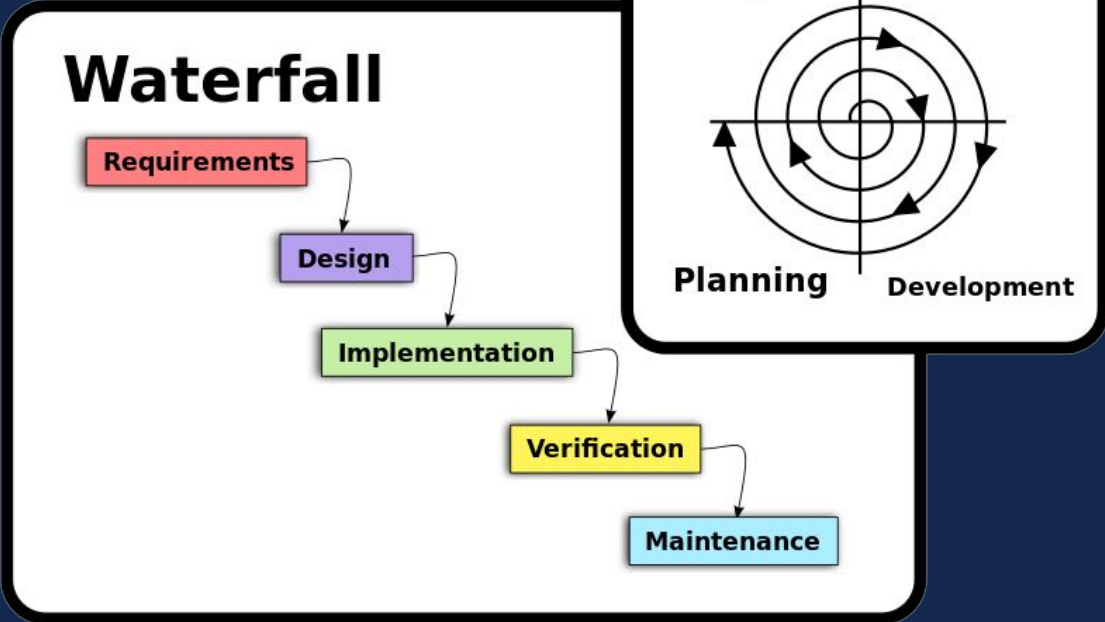
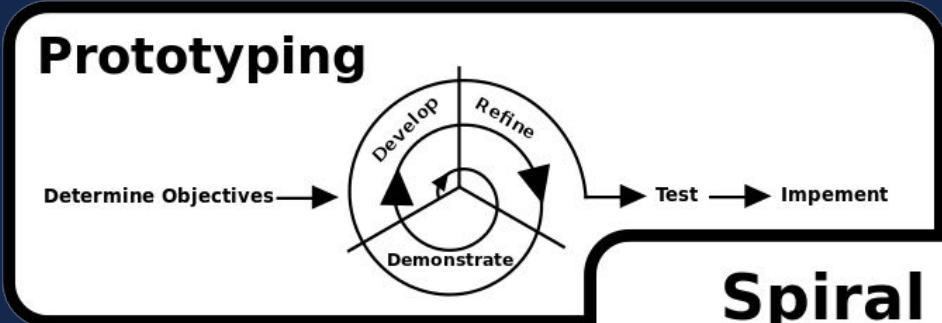
Incremental

Iterative

At Once

Formal

Ad Hoc



Theory and Practice

- There has been some sense that theory is well just theory
 - I've already noticed the perk ups in various lectures
- We will tell some war stories at various points in the course to back all this up

Where are we now?

**Grady Booch**  @Grady_Booch · Aug 10

The three golden ages of software engineering:

- Structured analysis and design
- Object oriented analysis and design
- Agile

We seem to be in a post-agile world, where a great deal of software involves selecting an assortment of frameworks then gluing them together.

Replying to [@Grady_Booch](#)



Software Engineering Post Modernism then or Post Agileism if we want our own term

From a cultural point of view, I think we are seeing consumerist&marketist effects permeating engineering Dev decisions are now like purchases and brand matters most to devs.

[Reply](#)

Summary

- Two basics approaches
 - Top Down - Waterfall
 - Bottom Up - Agile
- In appropriate third approach (mostly)
 - Middle Out - Just start coding
- We actually need to consider all models and move in and out of top down, bottom up and middle out thinking depending on what is called for
- The course will do things in an Agile fashion so we should get to know it but like playing a sport, we likely will be poor at the process until we practice it and every organization we go to may have a slightly different flavor to how they approach things.